高云 GW5A-25A FPGA 工程
project_2025_11_07

代码逐层解析与注释说明

器件型号	`GW5A-LV25UG324C2/I1`（`GW5A-25A`，UBGA324，速度等级 2）
目标板卡	AD/DA 采集 + 串口 + 数码管 + ADRC 控制演示板
顶层模块	`adda_scope_top`（`src/vga/adda_scope_top.v`，含 VGA 示波器）
工程文件	`project_2025_11_07.gprj`（Gowin EDA V1.9.9 64-bit）
对照工程	`FPGA_TD/project_2025_11_07_TD.al`（Anlogic TD 6.0.3 浏览用容器）
代码仓库	https://github.com/starfruitsz/FPGA
文档版本	v1.0
生成日期	2026-10-29

本报告覆盖工程中**全部** Verilog 源文件、IP 文件与物理约束文件；
每个文件按“模块接口 → 语句块 → 关键语句/信号”三级层次逐块解析，
并给出数值、时序与设计意图说明。所有源码均已在原位置添加中文注释，
注释前后代码的 Verilog 词法内容经脚本比对完全一致（仅新增注释）。

目录

1 工程总览 7

1.1 本报告的范围与方法 . . . 7
1.2 工程来源、工具链与“用 TD 打开”的说明 . . . 7
1.2.1 原始工程 . . . 7
1.2.2 关于用安路 TD 打开该工程 . . . 8
1.3 器件与资源 . . . 8
1.4 模块层次与文件对应关系 . . . 9
1.5 数据流与控制流 . . . 9
1.6 时钟与复位策略 . . . 9
1.7 参数与定点数约定汇总 . . . 9
1.8 代码风格与工程约定 . . . 12

2 顶层模块 adda（src/adda.v） 12

2.1 文件信息 . . . 12
2.2 第 1 层：模块接口 . . . 12
2.2.1 端口 . . . 12
2.2.2 参数 . . . 12
2.3 第 2 层：内部结构与语句块索引 . . . 12
2.4 第 3 层：语句块逐块解析 . . . 12
2.4.1 上电初值：po_flag2（106–113 行） . . . 12
2.4.2 发送数据拆分（115–116 行） . . . 15
2.4.3 块 1：ADC 去偏置（120–128 行） . . . 15
2.4.4 块 2：DA 输出选择（130–160 行） . . . 15
2.4.5 块 3/块 4：26ms 定时器与 pi_flag（162–185 行） . . . 16
2.4.6 块 5：串口 16bit 组帧（187–211 行） . . . 16
2.4.7 块 6：组帧完成脉冲（213–223 行） . . . 17
2.4.8 块 7：解析 B 帧（k 参数）（225–245 行） . . . 17
2.4.9 块 8：解析 A 帧（c 参数）（247–270 行） . . . 18
2.4.10 例化 1：key（272–282 行） . . . 18
2.4.11 例化 2：seg（283–297 行） . . . 19
2.4.12 例化 3：rtl（298–309 行） . . . 19
2.4.13 例化 4：eso（310–329 行） . . . 19
2.4.14 例化 5：uart_rx（330–346 行） . . . 20
2.4.15 例化 6：uart_tx（347–379 行） . . . 20

2.4.16 例化 7：divider（380–416 行） 21
2.5 时序与数值示例 21
2.6 本章小结 21
2.7 完整注释源码 22

3 中值滤波与电压换算模块 rtl（src/rtl.v） 29
3.1 文件信息 29
3.2 第 1 层：模块接口 29
3.3 第 2 层：内部信号 30
3.4 第 3 层：语句块解析 30
3.4.1 语句 1：符号判断（46–51 行） 30
3.4.2 块 1：中值使能（53–65 行） 30
3.4.3 块 2：累加计数器（67–76 行） 30
3.4.4 块 3：1024 点累加（78–90 行） 31
3.4.5 块 4：求平均（92–103 行） 31
3.4.6 语句 2：正/负向电压刻度（105–115 行） 31
3.4.7 块 5：电压换算（117–136 行） 32
3.4.8 语句 3：输出（138–141 行） 32
3.5 本章小结与提示 32
3.6 完整注释源码 33

4 扩展状态观测器 eso（src/eso.v） 35
4.1 文件信息 35
4.2 第 1 层：模块接口 36
4.3 算法：离散扩张状态观测器 36
4.4 第 2 层：内部信号 36
4.5 第 3 层：语句块解析 36
4.5.1 块 1：状态打拍（86–111 行） 36
4.5.2 块 2：参数锁存（113–136 行） 37
4.5.3 语句组 1：$u\beta_0$ 项（138–146 行） 38
4.5.4 语句组 2：hz_2、hz_3（148–154 行） 38
4.5.5 语句组 3/3’/4：观测误差与三个增益（156–180 行） 38
4.5.6 语句组 5/6：状态递推（182–200 行） 39
4.5.7 语句组 7–11：状态误差反馈与扰动补偿（202–239 行） 39
4.5.8 语句组 12：输出标度（241–246 行） 39
4.6 验证与调参建议 40

4.7 完整注释源码 40

5 时钟分频模块 divider（src/divider.v） 45

5.1 文件信息 45
5.2 第 1 层：接口 45
5.3 第 2 层：内部结构 45
5.4 第 3 层：语句块解析 45
5.4.1 块 1：分频计数器（32–52 行） 45
5.4.2 块 2/3：输出翻转（54–80 行） 46
5.5 完整注释源码 46

6 按键消抖模块 key（src/key.v） 48

6.1 文件信息 48
6.2 第 1 层：接口 48
6.3 第 3 层：语句块解析 48
6.3.1 块 1：消抖计时（29–44 行） 48
6.3.2 块 2：翻转输出（46–59 行） 49
6.4 本章小结 49
6.5 完整注释源码 49

7 串口接收模块 uart_rx（src/uart_rx.v） 51

7.1 文件信息 51
7.2 第 1 层：接口与参数 51
7.3 第 3 层：语句块解析 51
7.3.1 块 1–3：三级同步器（57–82 行） 51
7.3.2 块 4：起始位检测（84–94 行） 51
7.3.3 块 5：接收忙标志（96–108 行） 52
7.3.4 块 6/7：位内计数与中点采样（110–131 行） 52
7.3.5 块 8/9：位计数与数据移位（133–152 行） 53
7.3.6 块 10–12：完成脉冲与输出（154–184 行） 53
7.4 完整注释源码 54

8 串口发送模块 uart_tx（src/uart_tx.v） 57

8.1 文件信息 57
8.2 第 1 层：接口 57
8.3 帧结构 58
8.4 第 3 层：语句块解析 58

8.4.1 块 1：忙标志（48–58 行） . . . 58
8.4.2 块 2：位内计数（60–69 行） . . . 58
8.4.3 块 3：位起始脉冲（71–81 行） . . . 59
8.4.4 块 4：位序号（83–92 行） . . . 59
8.4.5 块 5：发送数据查表（94–134 行） . . . 59
8.5 完整注释源码 . . . 60

9 数码管扫描显示模块 seg（src/seg.v） 62
9.1 文件信息 . . . 62
9.2 第 1 层：接口 . . . 62
9.3 第 2 层：内部结构 . . . 62
9.4 第 3 层：语句块解析 . . . 62
9.4.1 例化：bcd（37–50 行） . . . 62
9.4.2 块 1：显示内容组装（71–98 行） . . . 63
9.4.3 块 2/3：位扫描节拍（100–122 行） . . . 64
9.4.4 块 4/5：扫描位序号与一热码（124–152 行） . . . 64
9.4.5 块 6/7：当前位数字与小数点（154–188 行） . . . 65
9.4.6 块 8：段码译码（190–215 行） . . . 66
9.4.7 块 9：位选输出（217–229 行） . . . 66
9.5 本章小结 . . . 67
9.6 完整注释源码 . . . 67

10 二进制转 BCD 模块 bcd（src/bcd.v） 72
10.1 文件信息 . . . 72
10.2 第 1 层：接口 . . . 72
10.3 算法原理 . . . 72
10.4 第 3 层：语句块解析 . . . 72
10.4.1 块 1：相位计数（32–43 行） . . . 72
10.4.2 块 2：移位/修正主过程（45–67 行） . . . 73
10.4.3 块 3：相位标志（69–76 行） . . . 73
10.4.4 块 4：结果锁存（78–96 行） . . . 73
10.5 转换实例 . . . 74
10.6 完整注释源码 . . . 74

11 PLL IP：Gowin_PLL（src/gowin_pll/gowin_pll.v） 76
11.1 文件信息 . . . 76

11.2 第 1 层：接口 . . . 76
11.3 第 2 层：结构与硬件参数 . . . 77
11.3.1 PLLA 原语例化（49–77 行） . . . 77
11.3.2 关键 defparam（87 行起） . . . 77
11.4 为什么它没有进入网表 . . . 77
11.5 完整注释源码 . . . 78

12 FIFO IP：fifo_top（src/fifo_top/fifo_top.v） 81
12.1 文件信息 . . . 81
12.2 IP 配置（fifo_top.ipc） . . . 82
12.3 文件结构：加密 IP 的三段式 . . . 82
12.3.1 加密声明段（30–49 行） . . . 82
12.3.2 明文模块声明（491–535 行） . . . 83
12.4 完整注释源码（仅明文部分） . . . 84

13 物理约束文件 project_2025_11_07.cst 86
13.1 文件信息 . . . 86
13.2 语法说明 . . . 86
13.3 引脚分配总表 . . . 86
13.3.1 用户约束的 30 个引脚 . . . 86
13.3.2 未约束、由工具自动分配的 12 个引脚 . . . 86
13.4 端口数与实际 Buffer 数的差异 . . . 88
13.5 完整注释源码 . . . 88

14 设计审查、缺陷清单与改进建议 90
14.1 设计意图复原 . . . 90
14.2 缺陷与风险清单 . . . 90
14.3 重点问题详解 . . . 92
14.3.1 P1：参数更新优先级导致 k 失效 . . . 92
14.3.2 P2：符号项的位宽陷阱 . . . 92
14.3.3 P4：BCD 位宽不足 . . . 93
14.4 工程性改进建议 . . . 93
14.5 上板验证清单 . . . 93

15 VGA 示波器子系统：架构与配置 94
15.1 概览与仓库地址 . . . 94
15.2 分层结构与文件清单 . . . 94

15.3 特性开关 94
15.4 IO（引脚）配置方式 96
15.4.1 引脚表格式 96
15.4.2 生成约束 96
15.4.3 本工程实际分配的 VGA 引脚 96
15.5 视频格式配置 97
15.6 运行时配置 97
15.6.1 按键 97
15.6.2 串口命令（特性 4） 97
15.6.3 显示内容 97
15.7 显示原理与资源占用 98
15.8 验证状态与限制 98

A 附录 **99**
A.1 附录 A：文件清单与统计 99
A.2 附录 B：模块与关键信号速查 99
A.3 附录 C：复现步骤 99
A.3.1 C.1 用安路 TD 打开（代码浏览用容器工程） 99
A.3.2 C.2 编译本报告 101
A.4 附录 D：术语表 101

1 工程总览

1.1 本报告的范围与方法

本报告针对 project_2025_11_07 这一 Gowin（高云）FPGA 工程，逐文件、逐语句块地解析其 RTL 设计。分析对象为工程文件中列出的全部 11 个 Verilog 源文件、2 个 IP 生成文件、1 个物理约束文件，以及工程目录中附带的 2 个 IP 例化模板：

类别	文件	是否进入网表
顶层	src/adda.v	是（Top）
算法	src/rtl.v、src/eso.v	是
外设	src/uart_rx.v、src/uart_tx.v	是
显示	src/seg.v、src/bcd.v	是
基础	src/key.v、src/divider.v	是
IP	src/gowin_pll/gowin_pll.v	否（未例化）
IP	src/fifo_top/fifo_top.v	否（未例化）
约束	src/project_2025_11_07.cst	引脚/电平
模板	src/gowin_pll/gowin_pll_tmp.v、src/fifo_top/fifo_top_tmp.v	否（仅供查阅）

“进入网表”一列并非主观判断，而是由三处证据共同确认：综合资源报告 syn_rsc.xml 中只出现 7 个子模块；PnR 报告 rpt.txt 中 PLL 占用为 0/6、BSRAM 占用为 0%；以及 adda.v 中 Gowin_PLL、fifo_top 的例化代码被注释符包裹（完整路径见第 1.2 节的目录说明）。

解析方法遵循“自顶向下、逐块下钻”的原则，每一章的结构固定为：

1. **文件信息**：路径、模块名、实例名、行数、时序模板；
2. **模块接口（第 1 层）**：端口表、参数表；
3. **内部结构（第 2 层）**：寄存器/连线清单与语句块索引；
4. **语句块解析（第 3 层）**：每块给出现行代码、逐信号解释、数值与时序推演；
5. **要点与风险**：设计意图、隐含假设、潜在缺陷。

1.2 工程来源、工具链与“用 TD 打开”的说明

1.2.1 原始工程

工程由 Gowin EDA（云源软件）V1.9.9 64-bit 生成，工程主文件为 project_2025_11_07.gprj，其 <Device> 节点写明了器件：

```
<Device name="GW5A-25A" pn="GW5A-LV25UG324C2/I1">gw5a25a-008</Device>
```

综合工程文件还给出了封装与速度等级：

```
<Device id="GW5A-25A" package="UBGA324" speed="2" partNumber="GW5A-LV25UG324C2/I1"/>
```

1.2.2 关于用安路 TD 打开该工程

这里需要明确一个前提：**安路 TangDynasty（TD）无法直接打开高云的.gprj 工程**。两者是不同厂商的私有工程格式（高云.gprj / 安路.al），器件库、原语、IP 格式、约束语言（高云.cst 与安路.adc/.fdc）均不兼容。经检查，本机安装的是 **TD 6.0.3 PHX SP2**，其器件数据库 D:\textbackslash FPGA\textbackslash arch\textbackslash device_resource\textbackslash device_resource.csv 只包含 ph1（13 个）与 ph2（2 个）两个 Phoenix 系列，不含 GW5A 或任何高云器件。

因此本次采用“**同名工程容器**”的方案：用 TD 的 Tcl 接口（td_commands_prompt.exe -nogui）新建了一个与源码同名的 TD 工程 FPGA_TD/project_2025_11_07_TD.al，器件选用 TD 6.0.3 中规模最小、封装同为 324 球的 PH1A60GEG324-3，再把 11 个 Verilog 源文件以**引用方式**（不复制）加入工程。这样 TD 的层次树、源码浏览器、交叉引用功能都能用于阅读和注释本工程的代码，而**原始高云工程完全未被改动**。

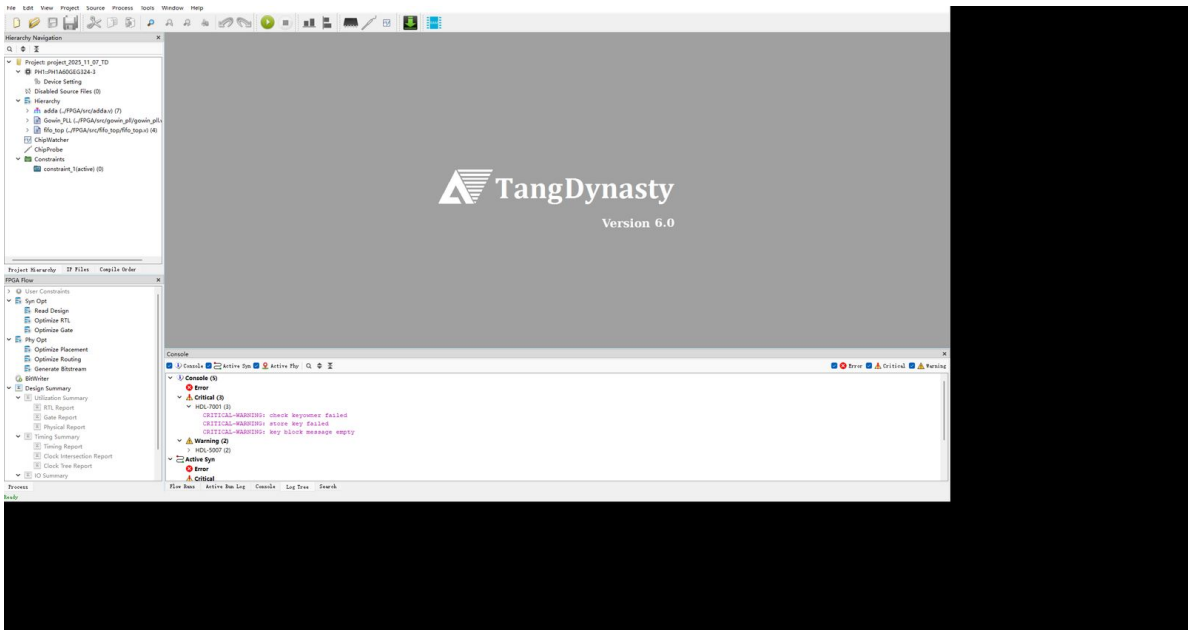


图 1： TD 6.0.3 打开同名工程容器后的界面：左侧 Hierarchy Navigation 已经识别出 adda (7)、Gowin_PLL (1)、fifo_top (4) 三个层次根，说明全部源文件均被正确解析。

为什么层次树里 adda 只有 7 个子节点？因为顶层只例化了 7 个模块（key、seg、rtl、eso、uart_rx、uart_tx、divider）。Gowin_PLL 与 fifo_top 虽然被 TD 解析并单独列在层次树中，但它们是“未被例化的孤立模块”，不会出现在 adda 之下，也不会进入综合网表——这与高云综合报告的结论一致。

注意：TD 容器工程只用于“阅读/结构化浏览”代码，**不要**用它做综合或生成码流。一方面它选用的是安路 PH1 器件，与板上的 GW5A-25A 引脚、电气属性完全不同；另一方面 gowin_pll.v 里的 PLLA 原语和 fifo_top.v 里加密的 FIFO 都不属于安路器件库。若需要重新编译，应回到 Gowin EDA 打开原始.gprj。

1.3 器件与资源

该器件属于 Arora V（GW5A）系列 22nm 工艺的入门型号，324 球 UBGA 封装，速度等级 2，工业级/商业级双标注（C2/I1）。工程中真正消耗资源的核心是 eso.v 的 9 个乘法器（映射为 7 个 MULTALU27X18）与 rtl.v 的两个除法器，其余模块都是控制/IO 逻辑。

表 1: GW5A-LV25UG324C2/I1 的容量与本工程占用

资源	器件容量	本工程占用
LUT / ALU / ROM16（逻辑）	23 040	575（2%）
寄存器（FF）	23 685	375（2%）
CLS（逻辑簇）	11 520	405（4%）
DSP（MULTALU27X18）	28	7（25%）
PLL	6	0（0%）
BSRAM	若干	0（0%）
I/O 端口	—	44（其中 42 个实际布出 Buffer）

1.4 模块层次与文件对应关系

```
adda  (src/adda.v)                                  <- 顶层 / Top Module
 |-- key_inst      : key      (src/key.v)          按键消抖 20ms
 |-- seg_inst      : seg      (src/seg.v)          数码管动态扫描
 |    `-- bcd_8421 : bcd      (src/bcd.v)          二进制 -> 3 位 BCD
 |-- adc_inst      : rtl      (src/rtl.v)          中值滤波 + 电压换算
 |-- eso_inst      : eso      (src/eso.v)          扩展状态观测器 (ADRC)
 |-- uart_rx_inst  : uart_rx  (src/uart_rx.v)      串口接收 (8N1)
 |-- uart_tx_inst  : uart_tx  (src/uart_tx.v)      串口发送 (16bit)
 `-- divider_inst  : divider  (src/divider.v)      1kHz 分频

未例化(只在工程文件列表里): Gowin_PLL (gowin_pll.v) / fifo_top (fifo_top.v)
```

图 2: 模块层次树。带 -- 前缀的是实例名，括号内为源文件。

1.5 数据流与控制流

系统是一套“AD 采样 → 数字化处理 → DA 回放 + 显示 + 串口上报 + 上位机在线调参”的闭环演示平台：

1.6 时钟与复位策略

全设计只有一个异步复位 sys_rst_n（低有效，引脚 A15），所有时序块都采用“posedge clk or negedge rst_n”的异步复位、同步释放风格（除 eso 用 ad_clk 作为时钟外）。由于 ad_clk1 由 sys_clk 分频而来，它与 sys_clk 是同源的，1kHz 下不存在实际跨时钟域问题，但用“分频时钟”而非“时钟使能”驱动的写法在更高频率下需要谨慎（见第 14 章）。

1.7 参数与定点数约定汇总

工程中所有运算均为“整数 + 移位”的定点数，没有浮点。约定如下：

表 2: 例化关系与实例名一览

父模块	子模块	实例名	作用
adda	key	key_inst	按键消抖
adda	seg	seg_inst	数码管扫描显示
seg	bcd	bcd_8421	二进制转 BCD
adda	rtl	adc_inst	中值/电压换算
adda	eso	eso_inst	扩展状态观测器
adda	uart_rx	uart_rx_inst	串口接收
adda	uart_tx	uart_tx_inst	串口发送
adda	divider	divider_inst	1kHz 分频

```
[外部 ADC] --ad_date[9:0]--> rtl (中值/电压) --sign--> seg(+bcd) --> 数码管
[外部 ADC] --ad_date[9:0]--> date_y(=ad_date-512) --> eso(1kHz)
              --date_u--> (~u + 512) --> DAC ad_out[9:0]
[上位机 rx] --Axxx/Bxxx--> uart_rx --> 16bit 组帧 --> c_date/k_date --> eso
[按键 key_in] --> key --> key_out --> 选择 ad_out 的数据来源
[ad_date] --> uart_tx --> tx (115200, 8N1, 每帧 182us)
[sys_clk 50MHz] --> divider --> ad_clk1/da_clk1 (1kHz) --> eso 采样时钟
```

图 3: 数据流与控制关系（省略复位线与时钟树细节）。

表 3: 关键信号与跨模块连接

信号	位宽/类型	说明
ad_date[9:0]	10bit 无符号	外部 ADC 并行码值（偏移二进制，中点 512）
date_y	10bit 有符号	ad_date-512，把中点移到零，送 ESO 的误差输入
date_u	10bit 有符号	ESO 输出的控制量，同时回送 ESO 的 u_out 形成反馈
ad_out[9:0]	10bit 无符号	送 DAC 的偏移二进制码，由按键选择“直通”或“控制量”
c_date/k_date	12bit	串口下发的两个 ESO 参数（A 帧、B 帧的低 12 位）
po_data/po_flag	8bit/1bit	串口接收字节与单周期有效脉冲
sign	1bit	当前码值低于中值，数码管据此显示负号

表 4: 时钟域一览

时钟	频率	来源	使用者
sys_clk	50MHz	板载有源晶振，引脚 T9	adda、key、seg、uart_rx、uart_tx、divider
ad_clk1	1kHz	divider 由 sys_clk 分频	AD 芯片转换时钟、eso 运算时钟
da_clk1	1kHz	divider（与 ad_clk1 同相）	DA 芯片转换时钟

表 5: 全工程参数/常数

位置	名称	取值	含义
adda	CNT_DATA_MAX	1024	中值样本数（转发给 rtl 的约定值）
adda	CNT_MAX	1 299 999	26ms 定时器（仅驱动死信号 pi_flag）
divider	cnt_max	24 999	$50\text{MHz}/(2\times 25000)=1\text{kHz}$
key	CNT_MAX	999 999	20ms 消抖时间
seg	CNT_MAX	23 999	0.48ms 位扫描间隔
bcd	—	22 拍	一次二进制转 BCD 的时钟数
uart_rx/tx	UART_BPS	115 200	波特率
uart_rx/tx	CLK_FREQ	50 000 000	系统时钟，用于算分频
eso	B01/B02/B03	90 / 2700 / 27000	观测器增益（配合 $h=2^{-10}$）
eso	b0	5000	控制增益（配合 >> 13）
eso	c / k	150 / 20	状态误差反馈参数（可串口在线修改）

- rtl 中电压量的标度为 100×（即 1 LSB = 0.01V），满量程 ±2.0V；
- eso 中采样步长 $h = 1/1024$ 由 >>>10 实现，系数 B0x 直接乘后右移；
- eso 输出经 >>>15 压缩后取低 10 位，映射到 DAC 的 0∼1023 量程。

1.8 代码风格与工程约定

- **时序模板**：几乎所有状态块都是“复位分支 + else if 条件 + else 保持”结构，属于安全的默认保持型写法，综合后不会产生锁存器；
- **位宽**：大量使用 <= 1'b0 给多位寄存器赋初值（例如 data_reg<=1'b0），Verilog 会自动零扩展，功能正确但不是严谨写法；
- **命名**：沿用“中文注释 + 拼音变量”（date、pingj、seg_date），阅读时需要对照注释理解；
- **重复常量**：50_000_000 在 uart 例化处出现两次，115200 同样，若更换时钟需同步修改 PLL 参数、divider 参数与 uart 参数三处。

2 顶层模块 adda（src/adda.v）

2.1 文件信息

项目	内容
文件路径	FPGA/src/adda.v（注释后 417 行）
模块名 / 实例名	adda / 顶层（Top Module）
工程中的角色	数据通路的汇聚点：连接 ADC/DAC、串口、数码管、按键
参数	CNT_DATA_MAX=1024、CNT_MAX=1299999
端口	44 个（16 输入 + 28 输出）
时钟/复位	sys_clk=50MHz、sys_rst_n 低有效异步复位
综合结果	575 LUT / 375 REG / 7 DSP（含子模块）

2.2 第 1 层：模块接口

2.2.1 端口

2.2.2 参数

2.3 第 2 层：内部结构与语句块索引

2.4 第 3 层：语句块逐块解析

2.4.1 上电初值：po_flag2（106–113 行）

```
initial
    begin
```

表 6: adda 端口表

端口	方向	位宽	说明 / 连接去向
sys_clk	input	1	50MHz 系统时钟（引脚 T9）
sys_rst_n	input	1	异步复位，低有效（引脚 A15）
ad_date	input	10	外部 ADC 并行码值（偏移二进制 0∼1023）
key_in	input	1	按键，1= 松开，0= 按下（引脚 B16）
ad_clk	input	1	预留的外部 AD 时钟输入（**本设计未使用**）
da_clk	input	1	预留的外部 DA 时钟输入（**本设计未使用**）
rx	input	1	串口接收（引脚 P18）
ad_clk1	output	1	送 AD 的转换时钟（1kHz，来自 divider）
da_clk1	output	1	送 DA 的转换时钟（1kHz，来自 divider）
ad_out	output	10	送 DAC 的数据（偏移二进制）
sel	output	4	数码管位选（低有效）
seg	output	8	数码管段选（bit7= 小数点，“1”点亮）
tx	output	1	串口发送（引脚 N18）
led0	output	1	按键状态指示（1= 直通 AD，0= 输出控制量）
led6	output	1	每收到一帧 B 参数翻转一次
led7	output	1	每收到一帧 A 参数翻转一次

参数	值	用途
CNT_DATA_MAX	11'd1024	与 rtl 的内部常数一致；**本模块内未使用**
CNT_MAX	25'd12_999_9=1 299 999	26ms 定时器上限，仅用于产生 pi_flag

表 7: adda 的语句块索引（行号对应本章末尾的完整源码）

行号	类型	功能
106	initial	po_flag2 上电清 0
115	assign	把 ad_date 拆成低 8 位 / 高 2 位供串口发送
120	always	块 1：ADC 码值去偏置（减 512）
130	always	块 2：DA 输出数据选择 + led0
162	always	块 3：26ms 自由计数器
174	always	块 4：pi_flag 单周期脉冲（死逻辑）
187	always	块 5：串口 16bit 指令组帧
213	always	块 6：组帧完成脉冲 po_flag3
225	always	块 7：解析 B 帧 → k 参数
247	always	块 8：解析 A 帧 → c 参数
272	例化	key → key_inst
283	例化	seg → seg_inst
298	例化	rtl → adc_inst
310	例化	eso → eso_inst
330	例化	uart_rx → uart_rx_inst
347	例化	uart_tx → uart_tx_inst
380	例化	divider → divider_inst

表 8: adda 的内部信号

信号	类型/位宽	作用
key_out	wire	消抖后的按键电平（1= 未按，0= 按下）
volt	wire [9:0]	rtl 输出的电压值（本模块未使用）
sign	wire	码值低于中值，送 seg 显示负号
date_y	reg signed [9:0]	ad_date-512，ESO 的误差输入
date_u	wire signed [9:0]	ESO 输出控制量，回送 ESO 的 u_out
po_data/po_flag	wire [7:0]/wire	串口接收字节与有效脉冲
ad_pingj0/1	wire [7:0]	串口发送数据的低 8 位、高 2 位
po_flag2	reg	16bit 组帧的奇偶标志
po_flag3	reg	16bit 完整接收脉冲
pi_flag	reg	26ms 脉冲，**无负载（死逻辑）**
rx_date	reg [15:0]	收到的 16bit 指令字
c_date/k_date	reg [11:0]	指令字中的 12bit 参数
c_flag/k_flag	reg	参数更新标志（置 1 后不清零）

```
        po_flag2=1'b0;
    end
```

上电时把 po_flag2 清 0，保证串口收到的第一个字节写入 rx_date[15:8]。Gowin 器件支持 FF 上电初值（综合为 INIT=0），因此可综合；但更规范的写法是在 po_flag2 所在的时序块复位分支里清零（当前该块 187 行并没有复位分支）。

2.4.2 发送数据拆分（115–116 行）

```
assign ad_pingj0={ad_date[7:0]};
assign ad_pingj1={6'b000000,ad_date[9:8]};
```

把 10bit 码值拆成两个字节，高位字节只用低 2 位。347 行的 uart_tx 例化把二者拼成 16bit：{ad_pingj0，ad_pingj1}。由于 uart_tx 先发 pi_data[7:0]，上位机收到的顺序是“低字节在前、高字节在后”，解析时应按小端处理。

2.4.3 块 1：ADC 去偏置（120–128 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        date_y<=10'd0;
    else
        date_y<=ad_date-10'd512; //0-1024 变 -512-512
```

ADC 输出 0∼1023 的偏移二进制，中点 512 对应 0V。减去 512 后得到 $-512 \sim +511$ 的有符号量，作为 ESO 的误差输入 e_in（eso 内部再取负号得到 $y = 512 - \mathrm{ADC}$）。10bit 减法溢出回绕正好等价于补码，因此无需显式符号扩展，date_y 声明为 signed [9:0] 即可。

2.4.4 块 2：DA 输出选择（130–160 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        ad_out<=10'd0;
    else if(key_out==1'b0)
        begin
            ad_out<=~ad_date; //按下输出 ad 读数
            led0<=1'b1;
        end
    else
        begin
            ad_out<=~date_u+10'd512; //按下输出控制量，0 灯亮
            led0<=1'b0;
        end
```

两种工作模式由按键状态 key_out 选择：

key_out	ad_out	led0	含义
0（按下）	~ad_date	1	直通：把 ADC 原始码取反后送 DAC
1（松开）	~date_u+10'd512	0	控制：把 ESO 的控制量整形成偏移二进制

~x 在 10bit 上等于 $1023-x$，因此第一行是“反相直通”，第二行 $\sim u+512=511-u$ 也是“反相 + 加偏置”。这个反相构成 ADC→ESO→DAC 回路的负反馈方向，是否与被控对象实际极性匹配，需要结合模拟板子确认（若方向相反，表现为自激或发散）。led0 与 ad_out 在同一时钟块中赋值，二者严格同拍。

~date_u 中 date_u 是 signed 而 10'd512 是 unsigned，Verilog 会把整个表达式按无符号解释。当前写法在数值上等价于 $511-u$（模 1024），但建议显式写成 10'd511 - date_u 或统一为无符号，避免可读性陷阱。

2.4.5 块 3/块 4：26ms 定时器与 pi_flag（162–185 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt <= 25'b0;
    else if(cnt == CNT_MAX)
        cnt <= 25'b0;
    else
        cnt <= cnt + 1'b1;
```

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        pi_flag <= 1'b0;
    else if(cnt == CNT_MAX-1'b0) //tx 发送过快后分配
        pi_flag <= 1'b1;
    else
        pi_flag <= 1'b0;
```

cnt 从 0 数到 CNT_MAX=1 299 999 后归零，周期为 $(1\,299\,999+1)/50\text{MHz}=26$ ms；在计满前一拍产生一个周期的 pi_flag。

但 pi_flag 在本模块中没有任何负载：uart_tx 的 pi_flag 端口在 347 行被固定接成 1'b1。因此块 3、块 4 综合时会被整体优化，只留下一个悬空网络。这属于历史遗留（注释里写着“tx 发送过快后分配”），如确实需要“每 26ms 发一帧”，应把 pi_flag 接到 uart_tx 的 pi_flag，并把常值 1 去掉。

2.4.6 块 5：串口 16bit 组帧（187–211 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        rx_date<=16'd0;
```

```
    else if((po_flag == 1'b1)&&(po_flag2==1'b0))
        begin
            rx_date<={po_data,8'd0}; //先赋高八位
            po_flag2<=1'b1;
        end
    else if((po_flag == 1'b1)&&(po_flag2==1'b1))
        begin
            rx_date<={rx_date[15:8],po_data}; //再赋低八位拼接十六位
            po_flag2<=1'b0;
        end
    else
        begin
            rx_date<=rx_date;
            po_flag2<=po_flag2;
        end
```

状态机只有两个状态，由 po_flag2 表示：0 表示“等高字节”，1 表示“等低字节”。上位机必须**严格按两字节一帧、连续发送**，且第一字节是高字节（含 A/B 标志），第二字节是低字节；中间若丢字节会造成整帧错位，直到下一次 po_flag2 复位为止。

2.4.7 块 6：组帧完成脉冲（213–223 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        po_flag3 <= 1'b0;
    else if((po_flag==1'b1)&&(po_flag2==1'b1))
        po_flag3 <= 1'b1;
    else
        po_flag3 <= 1'b0;
```

当第 2 个字节到达的同一拍（此时 po_flag2 仍为 1），po_flag3 置 1 且只保持一个周期，作为块 7/块 8 的解析使能。这种“用前一状态的标志做使能”的写法依赖非阻塞赋值的同拍读取特性，可读性一般但功能正确。

2.4.8 块 7：解析 B 帧（k 参数）（225–245 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        led6<=1'b1;
    else if((rx_date[15:12]==4'HB)&&(po_flag3==1'B1))
        begin
            led6<=~led6;
            k_flag<=1'b1;
            k_date<=rx_date[11:0];
        end
```

```
    else
        begin
            led6<=led6;
            //k_flag<=1'b0;
            k_date<=k_date;
        end
```

帧头高 4bit 为 4'hB 时判为 k 参数帧，取低 12bit 存入 k_date，同时翻转 led6 作接收提示。注意两点：其一，k_flag 置 1 后**没有清零分支**（被注释掉了），因此它一旦为 1 就永久保持；其二，k_date 没有复位分支，复位后是 X（实际由上电初值决定），不会影响功能，但仿真时需注意。

2.4.9 块 8：解析 A 帧（c 参数）（247–270 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        led7<=1'b1;
    else if((rx_date[15:12]==4'HA)&&(po_flag3==1'B1))
        begin
            led7<=~led7;
            c_flag<=1'b1;
            c_date<=rx_date[11:0];
        end
    else
        begin
            led7<=led7;
            //c_flag<=1'b0;
            c_date<=c_date;
        end
```

结构与块 7 完全对称，只是帧头为 4'hA，目标参数是 c。

重要缺陷：c_flag 与 k_flag 都是“置 1 后永不清零”，而 eso.v 中参数更新写成了 if(c_flag) c<=c_date; else if(k_flag) k<=k_date; 的**优先级结构**。后果是：只要收到过一帧 A 参数，c_flag 就恒为 1，此后 **k 参数永远无法再更新**。上位机表现为“先发 A 帧调 c 正常，之后发 B 帧调 k 完全没反应”。修复方式：把两个标志都改成单周期脉冲（解析时置 1，下一拍清 0），或把 eso 中的两个分支改为互不排斥的并行 if。详见第 14 章 P1。

2.4.10 例化 1：key（272–282 行）

```
key key_inst
(
    .key_in(key_in),
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    .key_out(key_out)
);
```

按键消抖，输出翻转式电平 key_out，供块 2 选择 DA 输出模式（详见第 6 章）。

2.4.11 例化 2：seg（283–297 行）

```
seg seg_inst
(
    .clk(sys_clk),
    .rst_n(sys_rst_n),
    .point(4'b0100),
    .data(ad_date), //gai
    .seg_en(1'b1),
    .sign(sign),
    .sel(sel),
    .seg(seg)
);
```

显示 ad_date 的十进制值。point=4'b0100 表示小数点固定在第 3 位（百位），所以显示格式形如“1.23V”？——注意 point 只控制小数点位置，并不显示单位；而 bcd 只能转 3 位十进制，因此 1000 以上会丢失千位（见第 10 章）。seg_en=1'b1 常开。

2.4.12 例化 3：rtl（298–309 行）

```
rtl adc_inst
(
    .clk(sys_clk),
    .rst_n(sys_rst_n),
    .ad_data(ad_date),
    .sign(sign),
    .volt(volt)
);
```

中值提取与电压换算，输出 sign 供数码管显示负号；volt 虽有连接，但在顶层没有任何负载，综合后会连内部逻辑一起被裁剪掉绝大部分（因此实际资源里并没有完整的 31bit 电压通路）。

2.4.13 例化 4：eso（310–329 行）

```
eso eso_inst
(
    .ad_clk(ad_clk1),
    .sys_rst_n(sys_rst_n),
    .e_in(date_y),
    .u_out(date_u),
    .out_da(date_u),
```

```
    .c_flag(c_flag),
    .k_flag(k_flag),
    .c_date(c_date),
    .k_date(k_date)
);
```

这是全设计的“算法核心”，共 9 个乘法器。注意三个细节：

1. 时钟用的是 ad_clk1（1kHz），即控制律每 1ms 运算一次，与 AD/DA 转换同步；
2. out_da 与 u_out 接在**同一根 wire** date_u 上：out_da 是输出（驱动），u_out 是输入（负载），因此 date_u 既是控制量输出、又是 ESO 内部的 $\beta_0 u$ 反馈项，构成一个延迟一拍的闭环；
3. z1k0/z2k0/z3k0 三个观测状态输出**悬空未连接**，只在本模块内部使用。

2.4.14 例化 5：uart_rx（330–346 行）

```
uart_rx
#(
    .UART_BPS(115200),
    .CLK_FREQ(50_000_000)
)
uart_rx_inst
(
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    .rx(rx),
    .po_data(po_data),
    .po_flag(po_flag)
);
```

115200bps、8N1。50MHz 下每位 434 个时钟，接收端在位中点采样（详见第 7 章）。

2.4.15 例化 6：uart_tx（347–379 行）

```
uart_tx
#(
    .UART_BPS(115200),
    .CLK_FREQ(50_000_000)
)
uart_tx_inst
(
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    //.pi_data(rx_date),//tx 先发低八位，所以 tx 值跟实际 rx_date 顺序颠倒，不影响 rx_date数
    值
```

```
    .pi_data({ad_pingj0,ad_pingj1}),
    .pi_flag(1'b1),

    .tx(tx)
);
```

pi_flag 接常值 1'b1，意味着发送器**自由连续发送**：一帧 21 个位时隙（≈ 182 μs）发完立即开始下一帧，串口线被本设计独占。发送内容是 {ad_date[7:0], 6'h0, ad_date[9:8]}，即 16bit 小端数值。

2.4.16 例化 7：divider（380–416 行）

```
divider
#(
    .cnt_max(24999)
)
divider_inst(
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    .clk0(ad_clk1),
    .clk1(da_clk1)
);
```

分频产生 1kHz 的 AD/DA 转换时钟：50MHz/$(2\times(24999+1))=1$kHz。两路输出同相同频。注意 divider 的参数注释写“500 分频”，实际是 50000 分频，以代码中的 24999 为准。

2.5 时序与数值示例

表 9: adda 中的时间基准

事件	周期公式	实际值
26ms 定时器	$(1299999+1)/50$MHz	26.0 ms
AD/DA 转换时钟	$2\times25000/50$MHz	1 kHz
一帧串口数据	$21\times434/50$MHz	182.3 μs
中值标定	$1024/50$MHz	20.5 μs

2.6 本章小结

- 顶层是一个“三出口”结构：AD 数据同时送 seg（看）、uart_tx（传）、以及经 rtl/eso（算）后送 DAC；
- 控制通道的参数由串口下发，帧约定为 16bit、大端发送（高字节在前），高 4bit 是 A/B 标签，低 12bit 是参数值；

- pi_flag、ad_clk/da_clk、volt、seg_date 等属于历史遗留或预留信号，读者不必在它们身上寻找功能；
- 两个 IP（PLL、FIFO）作为“工程装饰”存在于文件列表中，不参与综合。

2.7 完整注释源码

```
//============================================================================
// 文件: adda.v            模块: adda            工程: project_2025_11_07
// 功能: AD/DA 采集与 ADRC(自抗扰控制)演示的顶层模块(Top Module)。
// 时钟: sys_clk = 50MHz(系统时钟), sys_rst_n = 低电平有效异步复位。
//
// 模块层级(TD/Gowin 综合后实际编译到的 7 个子模块):
//   adda
//    +-- key_inst      : key      按键消抖(20ms), 切换 ad_out 输出内容
//    +-- seg_inst      : seg      数码管 6 位动态扫描, 显示 AD 原始码值
//    |    +-- bcd_8421 : bcd      二进制 -> 3 位 BCD(移位加 3 法)
//    +-- adc_inst      : rtl      AD 中值滤波 + 正/负向电压分辨率换算
//    +-- eso_inst      : eso      扩展状态观测器(ESO), 由误差产生控制量
//    +-- uart_rx_inst  : uart_rx  串口接收 16bit 指令帧(Axxx/Bxxx)
//    +-- uart_tx_inst  : uart_tx  串口发送 16bit AD 码值(两帧 8N1)
//    +-- divider_inst  : divider  分频, 产生 1kHz 的 AD/DA 转换时钟
//
// 另有 Gowin_PLL、fifo_top 两个 IP 文件被加入工程, 但在本模块中被注释掉,
// 未参与综合(综合报告 PLL 0/6、BSRAM 0% 可佐证), 详见文件末尾。
//============================================================================
module adda
#(
//----------------------------------------------------------------------------
// 参数区:
//   CNT_DATA_MAX : rtl 中值累加样本数(1024 次), 本模块仅作占位, 未使用
//   CNT_MAX      : 26ms 定时器上限 1299999; 50MHz 下 (1299999+1)/50MHz = 26ms
//                  该值用于产生 pi_flag, 但 pi_flag 在本模块中无任何负载(见下文)
//----------------------------------------------------------------------------
    parameter CNT_DATA_MAX=11'd1024,
    parameter CNT_MAX = 25'd12_999_9
)

(
//----------------------------------------------------------------------------
// 端口区
//   时钟/复位 : sys_clk(50MHz)、sys_rst_n(低有效)
//   AD 数据   : ad_date[9:0]  外部 ADC 并行输出, 无符号 0~1023
//   按键      : key_in        接板载按键, 1=松开, 0=按下
//   AD/DA 外部时钟: ad_clk、da_clk 为输入端口, 但本设计中未使用(悬空被裁剪)
//----------------------------------------------------------------------------
    input wire sys_clk,
    input wire sys_rst_n,
    input wire [9:0] ad_date,
    input wire key_in,
    input ad_clk,
    input da_clk,
    input wire rx,

//   输出时钟  : ad_clk1/da_clk1  由 divider 分频产生, 送给 AD/DA 芯片做转换时钟
//   DA 数据   : ad_out[9:0]      送 DAC 的偏移二进制码
```

```
//   数码管    : sel[3:0] 位选(低有效), seg[7:0] 段选(含小数点位)
//   串口      : tx 发送, rx 接收
//   LED       : led0 按键状态; led6/led7 分别指示 B 帧/A 帧参数接收
//----------------------------------------------------------------------------
    output wire ad_clk1,
    output wire da_clk1,
    output reg [9:0] ad_out,
    output wire[3:0] sel,
    output wire[7:0] seg,
    output wire tx,
    output reg led0,
    output reg led6,
    output reg led7
);

//----------------------------------------------------------------------------
// 内部信号声明
//   key_out     : 消抖后的按键电平(1=未按, 0=按下)
//   volt        : rtl 计算出的电压值, 本模块未使用
//   sign        : rtl 输出的符号位(1=低于中值), 送数码管显示负号
//   date_y      : ad_date-512 的有符号值, 把 ADC 中点 512 移到零点
//   date_u      : ESO 输出的有符号控制量, 同时作为 ESO 的 u_out 反馈输入
//   po_data/po_flag : 串口接收字节与单周期有效脉冲
//   seg_date    : 预留, 未使用
//   ad_pingj0/1 : 把 ad_date 拆成低 8bit + 高 2bit, 供 16bit 串口帧填写
//   po_flag2    : 16bit 组帧的奇偶标志(0=待收高字节, 1=待收低字节)
//   po_flag3    : 16bit 完整接收的单周期脉冲
//   c_date/k_date : 串口下发的 12bit 参数(Axxx=c, Bxxx=k)
//   c_flag/k_flag : 参数接收标志, 置 1 后**不再清零**(见下方附注)
//----------------------------------------------------------------------------
wire key_out;
wire [9:0] volt;
wire sign;
reg signed [9:0] date_y;
wire signed [9:0] date_u;
wire [7:0] po_data;
wire po_flag;
reg [7:0] seg_date;
wire [7:0] ad_pingj0;
wire [7:0] ad_pingj1;
reg po_flag2;
reg po_flag3; //16位信号完整接收标志位

reg pi_flag;
reg [24:0] cnt;
reg [15:0] rx_date;

reg [11:0] c_date;
reg [11:0] k_date;
reg c_flag;
reg k_flag;

//----------------------------------------------------------------------------
// 上电初值: po_flag2 用 initial 赋初值(TD/Gowin 支持 FF 上电初值),
// 保证第一帧数据先写入高 8bit。综合后该寄存器带 INIT=0 属性。
//----------------------------------------------------------------------------
initial
```

```
    begin
        po_flag2=1'b0;
    end

//----------------------------------------------------------------------------
// 拆包: ad_date[7:0] -> ad_pingj0, ad_date[9:8] -> ad_pingj1(高位补 0)。
// 串口帧格式 = {高 8bit(有效 2bit), 低 8bit}, 见 uart_tx 例化处的拼接。
//----------------------------------------------------------------------------
assign ad_pingj0={ad_date[7:0]};
assign ad_pingj1={6'b000000,ad_date[9:8]};
//assign ad_pingj2={8'b1010_1010};

//----------------------------------------------------------------------------
// [块 1] ADC 码值去偏置: 0~1023 的二进制偏移码 -> -512~+511 的补码(有符号)。
// 10bit 减法自然回绕实现符号扩展, date_y 为 signed 声明。
//----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        date_y<=10'd0;
    else
        date_y<=ad_date-10'd512; //0-1024 变 -512-512

//----------------------------------------------------------------------------
// [块 2] DA 输出数据选择 + 工作指示灯
//   key_out=0(按键按下) : 输出 ~ad_date, 即把 ADC 原始码取反回环到 DAC
//   key_out=1(松开)     : 输出 ~date_u+512, 即 ESO 控制量整形成偏移二进制码
// 注意: ~x+512 在 10bit 无符号运算下等价于 511-x, 即对 ESO 输出做了反相,
//       其方向构成 DAC->ADC 环路的负反馈, 方向正确性需结合被控对象确认。
// led0 与 ad_out 同步赋值, 高电平表示处于'直通显示'状态。
//----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        ad_out<=10'd0;
    else if(key_out==1'b0)
        begin
            ad_out<=~ad_date; //按下输出 ad 读数
            led0<=1'b1;
        end
    else
        begin
            ad_out<=~date_u+10'd512; //按下输出控制量, 0 灯亮
            led0<=1'b0;
        end

//----------------------------------------------------------------------------
// (以下为已废弃的 seg_date 计算代码, 保留作历史记录, 不参与综合)
//----------------------------------------------------------------------------
//always@(posedge sys_clk or negedge sys_rst_n)
// if(sys_rst_n==1'b0)
// seg_date<=8'd0;
// else
// seg_date<=po_data/256*100+po_data%256/16*10+po_data%16;
// //八位最大 255, 因转换 10 进制最大 99

//----------------------------------------------------------------------------
// [块 3] 26ms 自由计数器: 0~CNT_MAX 循环, 作为下一个块的节拍。
// (仅驱动 pi_flag, 而 pi_flag 无负载, 综合时与块 4 一并被优化掉)
```

```
164 //----------------------------------------------------------------------------
165 always@(posedge sys_clk or negedge sys_rst_n)
166     if(sys_rst_n == 1'b0)
167         cnt <= 25'b0;
168     else if(cnt == CNT_MAX)
169         cnt <= 25'b0;
170     else
171         cnt <= cnt + 1'b1;
172 
173 //----------------------------------------------------------------------------
174 // [块 4] pi_flag: 每 26ms 产生一个单周期脉冲。
175 // 实际 uart_tx 的 pi_flag 端口被固定接 1'b1(见例化处), 此信号悬空被裁剪,
176 // 属历史遗留逻辑; 若改为'每 26ms 发一帧', 应把 pi_flag 接到 uart_tx。
177 //----------------------------------------------------------------------------
178 always@(posedge sys_clk or negedge sys_rst_n)
179     if(sys_rst_n == 1'b0)
180         pi_flag <= 1'b0;
181     else if(cnt == CNT_MAX-1'b0) //tx 发送过快后分配
182         pi_flag <= 1'b1;
183     else
184         pi_flag <= 1'b0;
185 
186 //----------------------------------------------------------------------------
187 // [块 5] 16bit 串口指令组帧
188 //   po_flag 是 uart_rx 收到 1 个字节的脉冲; 用 po_flag2 区分奇偶次:
189 //   第 1 次: rx_date[15:8] <= po_data(高字节), po_flag2<=1
190 //   第 2 次: rx_date[7:0]  <= po_data(低字节), po_flag2<=0
191 // 因此上位机必须按 {高字节, 低字节} 顺序连续发 2 字节, 中间不能丢字节。
192 //----------------------------------------------------------------------------
193 always@(posedge sys_clk or negedge sys_rst_n)
194     if(sys_rst_n == 1'b0)
195         rx_date<=16'd0;
196     else if((po_flag == 1'b1)&&(po_flag2==1'b0))
197         begin
198             rx_date<={po_data,8'd0}; //先赋高八位
199             po_flag2<=1'b1;
200         end
201     else if((po_flag == 1'b1)&&(po_flag2==1'b1))
202         begin
203             rx_date<={rx_date[15:8],po_data}; //再赋低八位拼接十六位
204             po_flag2<=1'b0;
205         end
206     else
207         begin
208             rx_date<=rx_date;
209             po_flag2<=po_flag2;
210         end
211 
212 //----------------------------------------------------------------------------
213 // [块 6] 组帧完成标志: 第 2 个字节到达的同一拍置 1, 仅保持 1 个周期,
214 // 作为块 7/块 8 解析 16bit 内容的使能。
215 //----------------------------------------------------------------------------
216 always@(posedge sys_clk or negedge sys_rst_n)
217     if(sys_rst_n == 1'b0)
218         po_flag3 <= 1'b0;
219     else if((po_flag==1'b1)&&(po_flag2==1'b1))
220         po_flag3 <= 1'b1;
```

```
    else
        po_flag3 <= 1'b0; //16 位拼接标志

//----------------------------------------------------------------------------
// [块 7] 解析 'Bxxx' 帧(高 4bit=4'hB): 后 12bit 为 ESO 的 k 参数
//   led6 每收到一帧 B 参数翻转一次, 作为接收指示;
//   k_flag 置 1 后**没有清零分支**, k_date 亦无复位, 属'锁存式'标志;
//   该标志在 eso.v 中与 c_flag 构成优先级(见 eso.v 块 3 的说明)。
//----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        led6<=1'b1;
    else if((rx_date[15:12]==4'HB)&&(po_flag3==1'B1)) //前四位为参数标志位, 后12位为数据位
        begin
            led6<=~led6; //K 接收标志位有效后 led6 跳变
            k_flag<=1'b1;
            k_date<=rx_date[11:0]; //Bxxx, xxx 为 adrc 的 k
        end
    else
        begin
            led6<=led6;
            //k_flag<=1'b0;
            k_date<=k_date;
        end

//----------------------------------------------------------------------------
// [块 8] 解析 'Axxx' 帧(高 4bit=4'hA): 后 12bit 为 ESO 的 c 参数
//   同样 led7 翻转指示, c_flag 置 1 后不清零。
// 附注(设计隐患): c_flag 与 k_flag 一旦置 1 永不清除, 而 eso.v 中
//   'if(c_flag) c<=c_date; else if(k_flag) k<=k_date;' 为优先级结构,
//   所以一旦收到过 A 帧, 之后的 B 帧将**永远无法更新 k**。
//   若要两个参数都能反复在线修改, 应在解析块中把两个 flag 都做成单周期脉冲。
//----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        led7<=1'b1;
    else if((rx_date[15:12]==4'HA)&&(po_flag3==1'B1)) //前四位为参数标志位, 后12位为数据位
        begin
            led7<=~led7; //C 接收标志位有效后 led7 跳变
            c_flag<=1'b1;
            c_date<=rx_date[11:0]; //Axxx, xxx 为 adrc 的 c
        end
    else
        begin
            led7<=led7;
            //c_flag<=1'b0;
            c_date<=c_date;
        end


//----------------------------------------------------------------------------
// [例化 1] 按键消抖
//----------------------------------------------------------------------------
key key_inst
(
    .key_in(key_in),
    .sys_clk(sys_clk),
```

```
278     .sys_rst_n(sys_rst_n),
279     .key_out(key_out)
280 );
281 
282 //------------------------------------------------------------------------------
283 // [例化 2] 数码管显示: 显示 ad_date 的十进制值, 小数点位置 4'b0100(第 3 位),
284 // seg_en 恒为 1(常开), sign 来自 rtl 的符号判断, 用于显示负号。
285 //------------------------------------------------------------------------------
286 seg seg_inst
287 (
288     .clk(sys_clk),
289     .rst_n(sys_rst_n),
290     .point(4'b0100),
291     .data(ad_date), //gai
292     .seg_en(1'b1),
293     .sign(sign),
294     .sel(sel),
295     .seg(seg)
296 );
297 //------------------------------------------------------------------------------
298 // [例化 3] AD 中值滤波与电压换算(模块名 rtl, 实例名 adc_inst)
299 // 输出 sign 用于显示负号, volt 电压值本模块未使用(仅作为内部计算)。
300 //------------------------------------------------------------------------------
301 rtl adc_inst
302 (
303     .clk(sys_clk),
304     .rst_n(sys_rst_n),
305     .ad_data(ad_date),
306     .sign(sign),
307     .volt(volt)
308 );
309 //------------------------------------------------------------------------------
310 // [例化 4] ESO 扩展状态观测器
311 //   ad_clk   : 用 ad_clk1(1kHz)作为观测器采样节拍;
312 //   e_in     : date_y(去偏置的 ADC 误差), 注意 eso 内部会再取负号 -e_in;
313 //   u_out    : date_u 回送, 构成 u(k) 的一拍延迟;
314 //   out_da   : date_u 的另一驱动源, 即控制量输出(与 u_out 共用一个 wire)。
315 //------------------------------------------------------------------------------
316 eso eso_inst
317 (
318     .ad_clk(ad_clk1),
319     .sys_rst_n(sys_rst_n),
320     .e_in(date_y),
321     .u_out(date_u),
322     .out_da(date_u),
323     .c_flag(c_flag),
324     .k_flag(k_flag),
325     .c_date(c_date),
326     .k_date(k_date)
327 );
328 
329 //------------------------------------------------------------------------------
330 // [例化 5] 串口接收(115200, 8N1): 50MHz 时钟下分频计数值 = 434
331 //------------------------------------------------------------------------------
332 uart_rx
333 #(
334     .UART_BPS(115200),
```

```
    .CLK_FREQ(50_000_000)
)
uart_rx_inst
(
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    .rx(rx),
    .po_data(po_data),
    .po_flag(po_flag)
);

//-------------------------------------------------------------------------------
// [例化 6] 串口发送(115200, 8N1)
//   pi_data = {ad_pingj0, ad_pingj1} = {ad_date[7:0], 6'b0, ad_date[9:8]},
//   即一帧 16bit: 先低字节后高字节(uart_tx 内部先发 bit[7:0] 再发 bit[15:8]);
//   pi_flag = 1'b1 常高 => 发送器自由连续发送, 每 20 个位时隙(约 174us)发一帧,
//   由内部 work_en/bit_cnt 自行握手机制保证一帧一帧连续输出。
//-------------------------------------------------------------------------------
uart_tx
#(
    .UART_BPS(115200),
    .CLK_FREQ(50_000_000)
)
uart_tx_inst
(
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    //.pi_data(rx_date),//tx 先发低八位, 所以 tx 值跟实际 rx_date 顺序颠倒, 不影响 rx_date数值
    .pi_data({ad_pingj0,ad_pingj1}),
    .pi_flag(1'b1),

    .tx(tx)
);

//-------------------------------------------------------------------------------
// [已注释] PLL 例化: 原设计用 Gowin_PLL 产生 ad_clk1/da_clk1(6.25MHz),
// 后被 divider(1kHz)替代, 因此 gowin_pll.v 虽在工程中但未例化。
//-------------------------------------------------------------------------------
/*Gowin_PLL pll_inst(
    .clkout0(ad_clk1), //output clkout0
    .clkout1(da_clk1), //output clkout1
    .clkin(sys_clk) //input clkin
); */

//-------------------------------------------------------------------------------
// [例化 7] 分频器: cnt_max=24999, 50MHz/(2*25000) = 1kHz,
// 同时输出 clk0->ad_clk1、clk1->da_clk1, 两路同频同相。
//-------------------------------------------------------------------------------
divider
#(
    .cnt_max(24999)
)
divider_inst(
    .sys_clk(sys_clk),
    .sys_rst_n(sys_rst_n),
    .clk0(ad_clk1),
    .clk1(da_clk1)
```

```
);



//------------------------------------------------------------------------------
// [已注释] fifo_top 例化：原计划用 10bit x 128 深度的异步 FIFO 缓存 AD 数据，
// 因未使用(且相关信号 WrEn/RdEn/Q/Empty/Full 未声明)，故 fifo_top.v 只是被
// 包含在工程中，并没有进入网表。
//------------------------------------------------------------------------------
/*fifo_top fifo_top_inst1(
        .Data(ad_date), //input [9:0] Data
        .WrClk(ad_clk1), //input WrClk
        .RdClk(ad_clk1), //input RdClk
        .WrEn(WrEn), //input WrEn
        .RdEn(RdEn), //input RdEn
        .Q(Q), //output [9:0] Q
        .Empty(Empty), //output Empty
        .Full(Full) //output Full
    );*/

//------------------------------------------------------------------------------
// 附注：本模块共 42 个 I/O(B 参数 44 个引脚约束中含 2 个未使用端口)，
// 综合后逻辑占用 575 LUT / 375 REG / 7 个乘法器，资源占用极低。
//------------------------------------------------------------------------------
endmodule
```

3 中值滤波与电压换算模块 rtl（src/rtl.v）

3.1 文件信息

文件路径	FPGA/src/rtl.v（注释后 143 行）
模块 / 实例	rtl / adc_inst（在 adda 中）
功能	(1) 用 1024 点平均值标定 ADC 零点；(2) 换算正/负向电压
时钟	clk=50MHz，rst_n 低有效
实现代价	2 个 30bit 除法器、1 个 14bit 除法、若干乘加

3.2 第 1 层：模块接口

表 10: rtl 端口

端口	方向	位宽	说明
clk	input	1	系统时钟 50MHz
rst_n	input	1	异步复位，低有效
ad_data	input	10	ADC 码值（0~1023 无符号）
sign	output	1	1 表示码值低于零点（负电压）
volt	output	10	电压值，单位 0.01V（100× 标度）

3.3 第 2 层：内部信号

信号	位宽	作用
clk_sample	1	预留，未使用
median_en	1	标定完成标志，置 1 后保持
cnt_median	11	累加计数 0∼1024
data_sum_m	30	1024 点累加和
data_median	14	零点（平均值）
volt_reg	31	电压中间量
data_p	31	正向每 LSB 电压刻度（定点 $2^{13} \times 100$ 倍）
data_n	31	负向每 LSB 电压刻度

3.4 第 3 层：语句块解析

3.4.1 语句 1：符号判断（46–51 行）

```
assign sign=(ad_data<data_median)? 1'b1:1'b0;
//assign sign=1'b1;
```

当前码值低于标定零点即判为负,输出 sign 给 seg 决定是否显示负号。注释掉的 assign sign=1'b1 是调试时强制显示正号的开关。

3.4.2 块 1：中值使能（53–65 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        median_en<=1'b0;
    else if(cnt_median==CNT_DATA_MAX)
        median_en<=1'b1;
    else
        median_en<=median_en;
```

cnt_median 数到 1024 时把 median_en 置 1 并保持。由于块 2 在 median_en=1 后停止计数，cnt_median 永远停在 1024，所以 median_en 一旦置位就不会回零——这是一次性的“上电自校准”。

3.4.3 块 2：累加计数器（67–76 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt_median<=11'b0;
    else if(!median_en)
        cnt_median<=cnt_median+1'b1;
```

注意此处**没有 else 分支**：当 median_en=1 时 cnt_median 保持不变（推断为寄存器，不会产生锁存器，因为复位分支存在）。这个“计数到上限后自锁”的写法与块 1 配合完成单次标定。

3.4.4 块 3：1024 点累加（78–90 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_sum_m<=30'b0;
    else if(cnt_median==CNT_DATA_MAX)
        data_sum_m<=30'b0;
    else
        data_sum_m<=data_sum_m+ad_data;
```

在计数未满 1024 时把 ad_data 累加；计满的那一拍清零。最大累加和 $1023 \times 1024 = 1\,047\,552 < 2^{20}$，用 30bit 有充足余量。

3.4.5 块 4：求平均（92–103 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_median<=12'b0;
    else if(cnt_median==CNT_DATA_MAX)
        data_median<=data_sum_m/CNT_DATA_MAX;
    else
        data_median<=data_median;
```

累加和除以 1024。综合工具会把“除以 2 的整数次幂”优化成右移 10 位，不消耗除法器。由于非阻塞赋值，data_median 取到的是清零**之前**的累加和，结果正确。得到的平均值即 ADC 的零点（板级直流偏置），典型值约 512。

3.4.6 语句 2：正/负向电压刻度（105–115 行）

```
assign data_p=(median_en==1'b1)?3276800/((1024-data_median)*2):0;
assign data_n=(median_en==1'b1)?3276800/((data_median+1)*2):0;
```

把满量程 ±2.0V 按“零点两侧各自的码值区间”等分，得到每 LSB 的电压（定点数）。$3276800 = 2^{13} \times 400$，于是

$$\text{data_p} = \frac{2^{13} \times 400}{2(1024 - m)} = \frac{2^{13} \times 200}{1024 - m}, \tag{1}$$

$$\text{data_n} = \frac{2^{13} \times 400}{2(m + 1)} = \frac{2^{13} \times 200}{m + 1}, \tag{2}$$

其中 $m = \text{data_median}$。随后块 5 再右移 13 位，恰好抵消 2^{13}，得到 $100 \times V$ 的定点电压。两个除法器都是 30bit 组合除法，需要多级移位减法，是本模块 LUT 的主要来源。

3.4.7 块 5：电压换算（117–136 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        volt_reg<=1'b0;
    else if(median_en)
        if((ad_data>(data_median-3)) && (ad_data<(data_median+3)))
            volt_reg<=1'b0;
        else if(ad_data<data_median)
            volt_reg<=(data_n*(data_median-ad_data)) >> 13;
        else if(ad_data>data_median)
            volt_reg<=(data_p*(ad_data-data_median)) >> 13;
    else
        volt_reg<=1'b0;
```

三条规则：

1. **死区**：$|ad_data - m| \le 2$ 时输出 0，避免零点附近显示末位跳动；
2. **负向**：data_n$\times(m - ad_data)$ 后右移 13；
3. **正向**：data_p$\times(ad_data - m)$ 后右移 13。

volt 的有效范围是 0∼1023（对应 0∼10.23V 显示量程），而实际物理量程是 ±2.0V，因此数值上 volt $= 100 \times$ |电压|，例如 AD 满码时约 199（1.99V）。

数值验算：$m = 512$，$ad_data = 1023$ 时 data_p $= 2^{13} \times 200/512 = 3200$，volt $= (3200 \times 511) >> 13 = 199$，即 1.99V；ad_data=0 时同理得 199。与 ±2V 满量程、10bit 分辨率（约 3.9mV/LSB）一致。

3.4.8 语句 3：输出（138–141 行）

```
assign volt=volt_reg;
```

31bit 中间量直接连到 10bit 端口，综合时会隐式截断（取低 10 位）。由于设计保证结果不超过 1023，实际不会截断出错。

3.5 本章小结与提示

- 本模块的核心思想是“**上电自校准 + 分段线性换算**”，用一个平均值把 ADC 的直流量化偏置消掉，再把码值差换算成电压；
- 中值只在复位后 20.5μs 内计算一次，之后全程锁存；若温度漂移大，可改成“周期性重标定”（例如每 1s 重算一次）；
- 两个除法器占用了较多 LUT，若资源紧张，可把 $1/(1024 - m)$ 改为“查表 + 线性插值”或直接用移位近似；
- 输出 volt 在顶层没有负载，属“演示性输出”，实际综合时会被大幅裁剪。

3.6 完整注释源码

```
//================================================================================
// 文件: rtl.v            模块: rtl            实例名: adc_inst(位于 adda 顶层)
// 功能: AD 码值的中值(基准)提取与正/负向电压换算。
//   1) 上电后用 1024 个采样点求平均值, 作为 ADC 的'零点'(直流偏置);
//   2) 以该零点为界, 分别计算正向/负向每 LSB 对应的电压刻度(定点数);
//   3) 用当前码值与零点之差乘以刻度, 得到以 0.01V 为单位的电压值。
// 时钟: clk = sys_clk = 50MHz, 复位 rst_n 低有效。
// 注意: 中值只在复位后前 1024 个时钟一次性算出并锁存, 之后不再更新。
//================================================================================
module rtl
(
    input wire clk,
    input wire rst_n,
    input wire [9:0]ad_data,

    output wire sign,
    output wire [9:0]volt
);

//--------------------------------------------------------------------------------
// 参数: 求中值的采样点数 = 1024(2^10, 除以它等价于右移 10 位, 无需除法器)
//--------------------------------------------------------------------------------
parameter CNT_DATA_MAX=11'd1024;

//--------------------------------------------------------------------------------
// 内部寄存器
//   clk_sample : 预留(未使用)
//   median_en  : 中值就绪标志, 累加满 1024 点后置 1 且一直保持
//   cnt_median : 累加计数器, 0~1024
//   data_sum_m : 1024 点累加和(最大 1023*1024=1047552, 20bit 足够, 这里用 30bit)
//   data_median: 中值 = 累加和/1024, 即 ADC 直流零点
//   volt_reg   : 换算后的电压值(0.01V 单位)
//   data_p/data_n : 正向/负向每 LSB 的电压刻度, 定点表示
//--------------------------------------------------------------------------------
reg clk_sample;
reg median_en;
reg [10:0]cnt_median;
reg [29:0]data_sum_m;
reg [13:0]data_median;
reg [30:0]volt_reg;

wire [30:0]data_p;
wire [30:0]data_n;

//--------------------------------------------------------------------------------
// [语句 1] 符号判断: 当前码值低于零点 => sign=1(负数), 送数码管显示负号。
// (下一行 assign sign=1'b1 是调试用强制正号, 已注释)
//--------------------------------------------------------------------------------
assign sign=(ad_data<data_median)? 1'b1:1'b0;
//assign sign=1'b1;

//--------------------------------------------------------------------------------
// [块 1] 中值使能: cnt_median 计到 1024 时置 median_en=1 并保持。
// 由于 cnt_median 在 median_en=1 后停止计数(见下块), 该标志不会回落,
// 因此这是一次性的'上电自校准'。
```

```
56 //------------------------------------------------------------------------------
57 //中值使能信号
58 always @(posedge clk or negedge rst_n)
59     if(!rst_n)
60         median_en<=1'b0;
61     else if(cnt_median==CNT_DATA_MAX)
62         median_en<=1'b1;
63     else
64         median_en<=median_en;
65 
66 //------------------------------------------------------------------------------
67 // [块 2] 累加计数器: median_en=0 期间每拍 +1, 计到 1024 保持。
68 // (没有 else 分支, 计满后 cnt_median 保持 1024, 与块 1 共同形成自锁)
69 //------------------------------------------------------------------------------
70 //中值数据累加计数器
71 always @(posedge clk or negedge rst_n)
72     if(!rst_n)
73         cnt_median<=11'b0;
74     else if(!median_en)
75         cnt_median<=cnt_median+1'b1;
76 
77 //------------------------------------------------------------------------------
78 // [块 3] 1024 点累加: 未满 1024 时对 ad_data 累加, 满 1024 的那一拍清零,
79 // 为块 4 的除法提供'最后一拍的累加和'(注意清零与取商同拍, 由非阻塞赋值保证
80 // data_median 拿到的是清零前的和)。
81 //------------------------------------------------------------------------------
82 //1024 次中值数据累加总和
83 always @(posedge clk or negedge rst_n)
84     if(!rst_n)
85         data_sum_m<=30'b0;
86     else if(cnt_median==CNT_DATA_MAX)
87         data_sum_m<=30'b0;
88     else
89         data_sum_m<=data_sum_m+ad_data;
90 
91 //------------------------------------------------------------------------------
92 // [块 4] 求中值: 累加和 / 1024, 综合工具会自动实现为 >>10。
93 // 结果 data_median 只在计满的那一拍更新一次, 之后保持不变(零点锁存)。
94 //------------------------------------------------------------------------------
95 //中值数据
96 always @(posedge clk or negedge rst_n)
97     if(!rst_n)
98         data_median<=12'b0;
99     else if(cnt_median==CNT_DATA_MAX)
100        data_median<=data_sum_m/CNT_DATA_MAX;
101    else
102        data_median<=data_median;
103
104 //------------------------------------------------------------------------------
105 // [语句 2] 正/负向电压刻度(定点, 2^13*100 倍放大, 便于后续右移 13 位):
106 //   正向: data_p = 2^13*100*2/(1024-median)   (码值 >= 中值 时用)
107 //   负向: data_n = 2^13*100*2/(median+1)      (码值 <  中值 时用)
108 // 直观理解: 把 ±2V 满量程按各自的有效码值区间等分, 得到每 LSB 的电压。
109 // median_en=0 时输出 0, 避免除法在未标定时得到无意义值。
110 // 实现代价: 2 个 30bit 除法器, 综合后映射为多级流水/移位减法逻辑(占用一定 LUT)。
111 //------------------------------------------------------------------------------
112 //根据中值计算出的正向电压 AD 和负向电压 AD 的分辨率（放大 2^13*100 倍）
```

```
assign data_p=(median_en==1'b1)?3276800/((1024-data_median)*2):0;
assign data_n=(median_en==1'b1)?3276800/((data_median+1)*2):0;

//------------------------------------------------------------------------------
// [块 5] 电压换算: volt = 刻度 * |ad_data - 中值| >> 13
//   1) 码值在中值 ±3 以内视为 0(去抖/死区, 防止零漂显示跳动);
//   2) 码值低于中值: 用负向刻度 data_n * (median-ad_data) 再 >>13;
//   3) 码值高于中值: 用正向刻度 data_p * (ad_data-median) 再 >>13;
// 结果单位: 0.01V(即真实电压 x100), 例: AD=1023、中值=512 -> 约 199 = 1.99V。
//------------------------------------------------------------------------------
//处理后的稳定数据
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        volt_reg<=1'b0;
    else if(median_en)
        if((ad_data>(data_median-3)) && (ad_data<(data_median+3)))
            volt_reg<=1'b0;
        else if(ad_data<data_median)
            volt_reg<=(data_n*(data_median-ad_data)) >> 13;
        else if(ad_data>data_median)
            volt_reg<=(data_p*(ad_data-data_median)) >> 13;
    else
        volt_reg<=1'b0;

//------------------------------------------------------------------------------
// [语句 3] 输出: volt_reg(31bit) 赋给 10bit 端口 volt 时被寄存器化并截断,
// 实际有效范围 0~1023(0~10.23V 显示量程)。
//------------------------------------------------------------------------------
assign volt=volt_reg;

endmodule
```

4 扩展状态观测器 eso（src/eso.v）

4.1 文件信息

文件路径	FPGA/src/eso.v（注释后 252 行）
模块 / 实例	eso / eso_inst
功能	ADRC 的 ESO（扩张状态观测器）+ 状态误差反馈 + 扰动补偿
时钟	ad_clk=ad_clk1=1kHz（由 divider 分频得到）
实现代价	9 个乘法器 → 7 个 MULTALU27X18（占全设计 25%）

表 11: eso 端口

端口	方向	位宽	说明
ad_clk	input	1	观测器运算时钟（1kHz）
sys_rst_n	input	1	异步复位，低有效
e_in	input	10 signed	误差输入（date_y = ADC − 512）
u_out	input	10 signed	上一拍控制量反馈
c_flag、k_flag	input	1	参数更新标志
c_date、k_date	input	12	参数值
z1k0、z2k0、z3k0	output	20 signed	观测状态（顶层未连接）
out_da	output	10	控制量输出（接 date_u→DAC）

4.2 第 1 层：模块接口

4.3 算法：离散扩张状态观测器

设被控对象的离散模型为一阶/二阶带扰动的形式，ESO 用三阶线性观测器同时估计“输出 z_1、一阶导 z_2、总扰动 z_3”：

$$e(k) = z_1(k) - y(k), \tag{3}$$

$$z_1(k+1) = z_1(k) + h\, z_2(k) - \beta_{01} h\, e(k), \tag{4}$$

$$z_2(k+1) = z_2(k) + h\, z_3(k) - \beta_{02} h\, e(k) + b_0 h\, u(k), \tag{5}$$

$$z_3(k+1) = z_3(k) - \beta_{03} h\, e(k), \tag{6}$$

其中 $h = 1/1024$，$\beta_{01} = 90$、$\beta_{02} = 2700$、$\beta_{03} = 27000$，$b_0 = 5000/8192$。状态误差反馈（线性 ADRC 的 PD 部分）与扰动补偿为

$$u(k) = k\,(c\, z_1(k) + z_2(k)) + \mathrm{sgn}(c z_1 + z_2) + c\, z_2(k) - z_3(k), \tag{7}$$

最后右移 15 位压缩动态范围、取低 10 位送 DAC。

4.4 第 2 层：内部信号

4.5 第 3 层：语句块解析

4.5.1 块 1：状态打拍（86–111 行）

```
always@(posedge ad_clk or negedge sys_rst_n)
    begin
        if(sys_rst_n==1'b0)
            begin
                z1k1 <= 'd0;
                z2k1 <= 'd0;
                z3k1 <= 'd0;
```

信号	位宽	作用
z1k1/z2k1/z3k1	20 signed	上一拍观测状态
y	10 signed	测量值（= −e_in）
uk	10 signed	上一拍控制量
c / k	10/11 signed	反馈参数，默认 150 / 20
uk_b0、uk_b0_h	36 signed	$u\beta_0$ 及其 >> 13
z2k1_h、z3k1_h	20 signed	hz_2、hz_3（>> 10）
sub	21 signed	观测误差 $z_1 - y$
sub_B01..03	42 signed	误差乘三个增益
sub_B01_h..	42 signed	再 >> 10 得到 $\beta_{0i}he$
z1k0_reg..	44 signed	三阶状态递推的组合结果
err1/err2	32 signed	cz_1、cz_2
she	33 signed	$cz_1 + z_2$
kshe	43 signed	$k\,s$
sgn	5 signed	符号项（见风险说明）
v_1	42 signed	$ks + \mathrm{sgn} + cz_2$
viout	40 signed	$v_1 - z_3$
b	40 signed	viout >> 15

```
                y <= 'd0;
                uk <= 'd0;
            end
        else
            begin
                z1k1 <= z1k0; //z(k)=z(k+1)
                z2k1 <= z2k0;
                z3k1 <= z3k0;
                y <= -e_in;
                uk <= u_out;
            end
    end
```

这是 ESO 的“状态寄存器”层：把组合逻辑算出的 $k+1$ 拍状态锁存为 k 拍状态，并同时锁存测量值（取负：$y = -e_{in}$）和上一拍控制量。复位后全部为 0，保证观测器从零状态开始收敛。

4.5.2 块 2：参数锁存（113–136 行）

```
always@(posedge ad_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        begin
            c<=10'd150;
            k<=11'd20;
```

```
        end //初始化参数
    else if(c_flag==1'b1)
        c<=c_date;
    else if(k_flag==1'b1)
        k<=k_date;
    else
        begin
            c<=c;
            k<=k;
        end
```

复位时参数取经验值 $c = 150$、$k = 20$；工作期间由串口下发的 `c_flag/k_flag` 更新。这里的 `if/else-if` 是**互斥优先级**结构，配合顶层“标志置 1 后不清零”的问题，会导致 `k` 永远无法在 `c` 之后更新（详见第 2 章块 7/8 与第 14 章 P1）。

4.5.3 语句组 1：$u\beta_0$ 项（138–146 行）

```
wire signed [WIDTH + 16 - 1 : 0] uk_b0;
wire signed [WIDTH + 16 - 1 : 0] uk_b0_h;
assign uk_b0 = uk * b0;
assign uk_b0_h = uk_b0 >>> 13;
```

$u\beta_0$ 用 36bit 保存乘积，再右移 13 位。由于 $\beta_0 = 5000$、$2^{13} = 8192$，有效系数为 $5000/8192 \approx 0.6104$。

4.5.4 语句组 2：hz_2、hz_3（148–154 行）

```
wire signed [WIDTH - 1: 0] z2k1_h;
wire signed [WIDTH - 1: 0] z3k1_h;
assign z2k1_h = z2k1 >>> 10;
assign z3k1_h = z3k1 >>> 10;
```

右移 10 位即乘以采样步长 $h = 1/1024$。这两项是观测器的“积分通道”。

4.5.5 语句组 3/3’/4：观测误差与三个增益（156–180 行）

```
wire signed [WIDTH - 1 + 1: 0] sub;
assign sub = z1k1 - y;

wire signed [WIDTH - 1 + 1 + 21: 0] sub_B01;
wire signed [WIDTH - 1 + 1 + 21: 0] sub_B02;
wire signed [WIDTH - 1 + 1 + 21: 0] sub_B03;
assign sub_B01 = sub * B01;
assign sub_B02 = sub * B02;
assign sub_B03 = sub * B03;
```

```
wire signed [WIDTH - 1 + 1 + 21: 0] sub_B01_h;
assign sub_B01_h = sub_B01 >>> 10;
```

$e = z_1 - y$（21bit），分别乘 90、2700、27000（42bit），再右移 10 位得到 $\beta_{01}he$、$\beta_{02}he$、$\beta_{03}he$。三个乘法器共用一个被乘数 sub，综合工具可共享部分积。

4.5.6 语句组 5/6：状态递推（182–200 行）

```
assign z1k0_reg = z1k1 + z2k1_h - sub_B01_h; //前向差分 ESO
assign z2k0_reg = z2k1 + z3k1_h - sub_B02_h + uk_b0_h;
assign z3k0_reg = z3k1 - sub_B03_h;

assign z1k0 = z1k0_reg[21: 0];
assign z2k0 = z2k0_reg[21: 0];
assign z3k0 = z3k0_reg[21: 0];
```

这三行就是第 4.3 节的三个差分方程，一一对应。结果在 44bit 中间量上计算后，取低 22 位输出并回灌到块 1。取低位属于“环绕截断”，正常收敛时高位是符号扩展、不损失信息；但若观测器发散，会静默回绕而不是饱和。

4.5.7 语句组 7–11：状态误差反馈与扰动补偿（202–239 行）

```
assign err1 = z1k0 * c;
assign err2 = z2k0 * c;

assign she = err1 + z2k0;              // s = c*e1 + e2
assign kshe = she * k;

assign sgn= (she>1'd0)?1'd1:4'b1111;

assign v_1 = kshe +sgn + err2;
assign viout = v_1 - z3k0;             // ks+sgn+cz2-f
```

组合过程为

$$s = c\,z_1 + z_2 \;\rightarrow\; ks \;\rightarrow\; v_1 = ks + \mathrm{sgn}(s) + c\,z_2 \;\rightarrow\; u = v_1 - z_3. \tag{8}$$

4.5.8 语句组 12：输出标度（241–246 行）

```
wire signed [40 -1: 0] b;
assign b = viout >>> 15;
assign out_da = b[9 : 0];
```

右移 15 位（/32768）把大位宽控制量压到 10bit 量程，再取低 10 位送 DAC。这一步等价于给控制量加了一个固定的标度因子，实际增益需结合 DAC 的量化基准理解：上位机通过调 k、c 改变闭环

增益，而 2^{-15} 是设计者选定的全局标度。

位宽与符号问题（详见第 14 章 P2/P3）：

- sgn 声明为 wire signed [4:0]，但赋值为 4'b1111。4bit 无符号量补零扩展到 5bit 后是 **+15**，而不是 −1。若本意是负单位符号，应写 -5'sd1。
- she 声明 33bit，而 err1（32bit）加 z2k0（22bit）最大需要 34bit，极端值会丢失最高位。

4.6 验证与调参建议

1. **先用仿真验证收敛：**给 e_in 阶跃，观察 z1k0 是否跟随、z3k0 是否稳定；如果 z3 持续增长说明增益过大；
2. **用 ChipWatcher 观测：**Gowin EDA 的在线逻辑分析仪可以抓 z1k0..z3k0，注意它们没有引到顶层端口，需要先在顶层加观测输出或使用内部节点抓取；
3. **参数范围：**c/k 是 12bit 串口参数，写入 10/11bit 寄存器时会被截断，上位机应把 c 限制在 0∼1023、k 限制在 0∼2047。

4.7 完整注释源码

```
//================================================================================
// 文件: eso.v            模块: eso            实例名: eso_inst(adda 顶层)
// 功能: 线性/非线性扩展状态观测器(ESO, Extended State Observer), 与状态误差
//       反馈组合成 ADRC(自抗扰控制)的核心运算: 观测被控量 z1、微分 z2、
//       总扰动 z3, 再按 u = [k*(c*z1+z2) + sign + c*z2 - z3] 生成控制量。
//
// 离散化形式(前向差分, 采样步长 h = 1/1024):
//   e(k)   = z1(k) - y(k)
//   z1(k+1) = z1(k) + h*z2(k) - B01*h*e(k)
//   z2(k+1) = z2(k) + h*z3(k) - B02*h*e(k) + b0*h*u(k)
//   z3(k+1) = z3(k)           - B03*h*e(k)
//
// 定点实现: 乘常数后用 >>>10 实现 *h, 用 >>>13/>>>15 做标度归一。
// 所有运算均为有符号补码, 位宽在每级手工扩展(见下方各 wire 声明)。
//================================================================================
`timescale 1ns / 1ps
//--------------------------------------------------------------------------------
// `define WIDTH 20: 内部状态 z1/z2/z3 的位宽(20bit 有符号, 约 ±52 万)
//--------------------------------------------------------------------------------
`define WIDTH 20

module eso
(
//--------------------------------------------------------------------------------
// 端口
//   ad_clk   : 观测器运算时钟(接 adda 的 ad_clk1 = 1kHz)
//   sys_rst_n: 异步复位, 低有效
//   e_in     : 测量误差输入(adda 送 date_y = AD 码值-512)
//   u_out    : 上一拍的控制量反馈(接 date_u), 用于 b0*u 项
//   c_flag/k_flag/c_date/k_date : 串口下发的两个可调参数 c、k
//   z1k0..z3k0 : 观测状态输出(本工程 adda 中未连接, 仅内部使用)
//   out_da     : 控制量输出(10bit), 接 adda 的 date_u -> DAC
```

```
33 //------------------------------------------------------------------------------
34     input ad_clk,
35     input sys_rst_n,
36     input signed [9:0] e_in,
37     input signed [9:0] u_out,
38     input wire c_flag,
39     input wire k_flag,
40     input wire [11:0] c_date,
41     input wire [11:0] k_date,
42 
43     output signed [`WIDTH-1: 0] z1k0,
44     output signed [`WIDTH-1: 0] z2k0,
45     output signed [`WIDTH-1: 0] z3k0,
46     output signed [9:0] out_da
47 );
48 
49 //------------------------------------------------------------------------------
50 // 状态与寄存器
51 //   z1k1/z2k1/z3k1 : 上一拍(k)的观测状态，位宽 `WIDTH
52 //   y   : 测量值(= -e_in)，即取反后的误差
53 //   uk  : 上一拍控制量
54 //   c   : 状态误差反馈系数(默认 150，可由串口 A 帧修改)
55 //   k   : 反馈增益(默认 20，可由串口 B 帧修改)
56 //------------------------------------------------------------------------------
57 reg signed [`WIDTH-1: 0] z1k1;
58 reg signed [`WIDTH-1: 0] z2k1;
59 reg signed [`WIDTH-1: 0] z3k1;
60 reg signed [9:0] y;
61 reg signed [9:0] uk;
62 reg signed [9:0] c;
63 reg signed [10:0] k;
64 
65 //------------------------------------------------------------------------------
66 // 系数(定点数)：B01/B02/B03 为观测器增益，b0 为控制增益，n 未使用。
67 // 常数的实际比例由运算中的移位(>>>10 / >>>13)决定。
68 //------------------------------------------------------------------------------
69 parameter signed B01=21'd90;
70 parameter signed B02=21'd2700;
71 parameter signed B03=21'd27000;
72 parameter signed b0=16'd5000;
73 parameter signed n = 2'd1;
74 
75 //------------------------------------------------------------------------------
76 // (已废弃) 上电用 initial 直接赋 c/k 初值，现改在复位分支中赋值。
77 //------------------------------------------------------------------------------
78 //initial
79 //begin
80 // c=10'd150;
81 // k=11'd25;
82 //end
83 //
84 
85 //------------------------------------------------------------------------------
86 // [块 1] 状态打拍
87 //   zx_k1 <= zx_k0 : 把组合逻辑算出的 k+1 拍状态锁存为 k 拍状态(一级流水)
88 //   y  <= -e_in    : 误差取反(符号约定：观测器用 -e)
89 //   uk <= u_out    : 控制量延迟一拍，供 b0*u 项使用
```

```
// 复位时全部清零，保证上电从 0 状态开始收敛。
//------------------------------------------------------------------------------
always@(posedge ad_clk or negedge sys_rst_n)
    begin
        if(sys_rst_n==1'b0)
            begin
                z1k1 <= 'd0;
                z2k1 <= 'd0;
                z3k1 <= 'd0;
                y <= 'd0;
                uk <= 'd0;
            end
        else
            begin
                z1k1 <= z1k0; //z(k)=z(k+1)
                z2k1 <= z2k0;
                z3k1 <= z3k0;
                y <= -e_in;
                uk <= u_out;
            end
    end

//------------------------------------------------------------------------------
// [块 2] 参数锁存(c、k)
//   复位: c=150, k=20 (经验初值)
//   c_flag=1: c <= c_date
//   k_flag=1: k <= k_date
// 注意(设计隐患): 两条为 if/else-if 优先级结构，且 adda.v 中的 c_flag/k_flag
//   置 1 后不清零 => 一旦收到过 A 帧，c_flag 恒为 1，k 将永远无法再被更新。
//   建议把两个 flag 改为单周期脉冲，或改成两个独立的 if 分支。
//------------------------------------------------------------------------------
always@(posedge ad_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        begin
            c<=10'd150;
            k<=11'd20;
        end //初始化参数
    else if(c_flag==1'b1)
        c<=c_date;
    else if(k_flag==1'b1)
        k<=k_date;
    else
        begin
            c<=c;
            k<=k;
        end

//------------------------------------------------------------------------------
// [语句组 1] u*b0 * h 项
//   uk_b0   = uk * 5000            (20+16=36bit)
//   uk_b0_h = uk_b0 >>> 13         (等价于 *5000/8192, 约 *0.61)
//------------------------------------------------------------------------------
wire signed [`WIDTH + 16 - 1 : 0] uk_b0;
wire signed [`WIDTH + 16 - 1 : 0] uk_b0_h;
assign uk_b0 = uk * b0;
assign uk_b0_h = uk_b0 >>> 13;

```

```
147 //----------------------------------------------------------------------------
148 // [语句组 2] z2*h、z3*h 项: 右移 10 位 = 乘 1/1024 = 乘采样步长 h
149 //----------------------------------------------------------------------------
150 wire signed [`WIDTH - 1: 0] z2k1_h; //z(k)*h
151 wire signed [`WIDTH - 1: 0] z3k1_h;
152 assign z2k1_h = z2k1 >>> 10;
153 assign z3k1_h = z3k1 >>> 10;
154
155 //----------------------------------------------------------------------------
156 // [语句组 3] 观测误差 e = z1 - y (宽度 WIDTH+1 = 21bit)
157 //----------------------------------------------------------------------------
158 wire signed [`WIDTH - 1 + 1: 0] sub;
159 assign sub = z1k1 - y;
160
161 //----------------------------------------------------------------------------
162 // [语句组 3'] e 分别乘三个观测器增益 B01=90 / B02=2700 / B03=27000
163 //----------------------------------------------------------------------------
164 wire signed [`WIDTH - 1 + 1 + 21: 0] sub_B01;
165 wire signed [`WIDTH - 1 + 1 + 21: 0] sub_B02;
166 wire signed [`WIDTH - 1 + 1 + 21: 0] sub_B03;
167 assign sub_B01 = sub * B01; //B1e1,B2e1,B3e1
168 assign sub_B02 = sub * B02;
169 assign sub_B03 = sub * B03;
170
171 //----------------------------------------------------------------------------
172 // [语句组 4] 再右移 10 位, 得到 B0x*h*e 项
173 //----------------------------------------------------------------------------
174 wire signed [`WIDTH - 1 + 1 + 21: 0] sub_B01_h;
175 wire signed [`WIDTH - 1 + 1 + 21: 0] sub_B02_h;
176 wire signed [`WIDTH - 1 + 1 + 21: 0] sub_B03_h;
177 assign sub_B01_h = sub_B01 >>> 10; //Be1*h
178 assign sub_B02_h = sub_B02 >>> 10;
179 assign sub_B03_h = sub_B03 >>> 10;
180
181 //----------------------------------------------------------------------------
182 // [语句组 5] 三个观测方程的组合逻辑(对应文件头的差分方程组)
183 // 注意结果宽度为 44bit(声明 WIDTH+1+21+2), 高位为符号扩展位。
184 //----------------------------------------------------------------------------
185 wire signed [`WIDTH - 1 + 1 + 21 + 2: 0] z1k0_reg;
186 wire signed [`WIDTH - 1 + 1 + 21 + 2: 0] z2k0_reg;
187 wire signed [`WIDTH - 1 + 1 + 21 + 2: 0] z3k0_reg;
188 assign z1k0_reg = z1k1 + z2k1_h - sub_B01_h; //前向差分 ESO
189 assign z2k0_reg = z2k1 + z3k1_h - sub_B02_h + uk_b0_h;
190 assign z3k0_reg = z3k1 - sub_B03_h;
191
192 //----------------------------------------------------------------------------
193 // [语句组 6] 取低 22bit([21:0]) 驱动 z1k0/z2k0/z3k0, 并回灌到块 1 打拍。
194 // 直接取低 22 位属于'环绕'截断, 正常工作时高位为符号扩展, 不会溢出;
195 // 但若 z 状态发散, 这里会静默回绕(无饱和保护)。
196 //----------------------------------------------------------------------------
197 assign z1k0 = z1k0_reg[21: 0]; //z(k+1)
198 assign z2k0 = z2k0_reg[21: 0];
199 assign z3k0 = z3k0_reg[21: 0];
200
201 //----------------------------------------------------------------------------
202 // [语句组 7] 状态误差反馈: s = c*z1 + z2 (err1=c*z1, err2=c*z2, she=err1+z2)
203 //   注意 she 只声明了 33bit, 而 err1(32bit)+z2k0(22bit) 最大需 34bit,
```

```
//   极端情况下最高位会被截掉，属潜在的位宽不足点。
//----------------------------------------------------------------------------
wire signed [`WIDTH + 12 -1: 0] err1;
wire signed [`WIDTH + 12 -1: 0] err2;

assign err1 = z1k0 * c;
assign err2 = z2k0 * c;

wire signed [`WIDTH + 13 -1: 0] she ;
assign she = err1 + z2k0; //s=ce1+e2
//----------------------------------------------------------------------------
// [语句组 8] kshe = s * k (13+11 = 43bit 结果)
//----------------------------------------------------------------------------
wire signed [`WIDTH + 13 + 11 -1: 0] kshe;
assign kshe = she * k;

//----------------------------------------------------------------------------
// [语句组 9] 符号项 sgn: 设计意图是 s>0 取 +1，否则取 -1。
// 注意(设计隐患): sgn 声明为 5bit 有符号，但赋的 4'b1111 是无符号常量，
//   补零扩展后等于 +15，而不是 -1。若要得到 -1 应写 -5'sd1。
//----------------------------------------------------------------------------
wire signed [4 : 0] sgn;
assign sgn= (she>1'd0)?1'd1:4'b1111;

//----------------------------------------------------------------------------
// [语句组 10] v_1 = k*s + sgn + c*z2  (线性状态误差反馈 + 符号补偿)
//----------------------------------------------------------------------------
wire signed [`WIDTH + 22 -1: 0] v_1;
assign v_1 = kshe +sgn + err2;

//----------------------------------------------------------------------------
// [语句组 11] viout = v_1 - z3 : 减去观测到的'总扰动'，实现扰动补偿
//----------------------------------------------------------------------------
wire signed [`WIDTH + 21 -1: 0] viout;
assign viout = v_1 - z3k0; //ks+sgn+cz2-f

//----------------------------------------------------------------------------
// [语句组 12] 输出标度: 右移 15 位压缩动态范围，取低 10 位送 DAC(0~1023)。
//   右移 15 相当于除以 32768，目的是把大位宽的控制量映射到 10bit DAC 量程。
//----------------------------------------------------------------------------
wire signed [40 -1: 0] b;
assign b = viout >>> 15;
assign out_da = b[9 : 0];

//----------------------------------------------------------------------------
// 附注: 本模块共有 9 个乘法器(1 个 e*B、3 个 e*B0x、1 个 u*b0、2 个 z*c、1 个 s*k)，
// 综合后映射为 7 个 MULTALU27X18(见综合资源报告)，是设计中 DSP 的主要来源。
//----------------------------------------------------------------------------
endmodule
```

5 时钟分频模块 divider（src/divider.v）

5.1 文件信息

文件路径	FPGA/src/divider.v（注释后 82 行）
模块 / 实例	divider / divider_inst
功能	把 50MHz 系统时钟分频成 1kHz，输出两路同相方波作 AD/DA 转换时钟
参数	cnt_max=24999（顶层覆盖）

5.2 第 1 层：接口

端口	方向	位宽	说明
sys_clk	input	1	50MHz 输入时钟
sys_rst_n	input	1	异步复位，低有效
clk0	output	1	分频输出 0（接 ad_clk1）
clk1	output	1	分频输出 1（接 da_clk1）

5.3 第 2 层：内部结构

只有一个 15bit 计数器 cnt 和两个输出寄存器 clk0、clk1。整个模块是“计数—翻转”的最简偶数分频结构：

块	行号	功能
块 1	32	计数器 0∼cnt_max 循环
块 2	54	clk0 每计满翻转一次
块 3	66	clk1 与 clk0 同步翻转

5.4 第 3 层：语句块解析

5.4.1 块 1：分频计数器（32–52 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        cnt<=15'd0;
    else if(cnt==cnt_max)
        cnt<=15'd0;
    else
        cnt<=cnt+15'd1;
```

计数 0 到 24999 共 25000 个时钟，是一个周期为 25000 拍的“节拍发生器”。

5.4.2 块 2/3：输出翻转（54–80 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        clk0<=1'b0;
    else if(cnt==cnt_max)
        clk0<=~clk0;
    else
        clk0<=clk0;
```

clk0 与 clk1 的代码完全相同，都在计满时翻转，因此输出同频同相。

表 12: 分频关系

量	公式	数值
计数器周期	cnt_max+1	25000 拍
输出方波周期	2×25000 拍	50000 拍
输出频率	50MHz/50000	1 kHz
占空比	1:1	50%
输出抖动	一个 sys_clk 周期	20 ns

两点提示：
(1) 文件头注释与参数注释写“500 分频”，但实参是 24999，真实分频比是 50000，以代码为准；
(2) 用普通寄存器直接产生时钟并驱动 I/O 引脚，在 1kHz 下没有问题，但如果把频率提高到数十 MHz，建议改用 PLL（工程里的 Gowin_PLL 就是为此准备的）或“时钟使能”写法，并补充 create_generated_clock 约束。

5.5 完整注释源码

```
1  //============================================================================
2  // 文件: divider.v        模块: divider        实例名: divider_inst(adda 顶层)
3  // 功能: 对 50MHz 系统时钟做偶数分频，输出两路同相同频的方波，作为外部 AD/DA
4  //       芯片的转换(采样)时钟。
5  // 计算: cnt_max=24999 -> 计数 25000 个周期翻转一次 -> 分频比 50000,
6  //       50MHz / 50000 = 1kHz(注意: 原注释'500 分频'与实参不符，以 24999 为准)。
7  //============================================================================
8  module divider
9  #(
10 //----------------------------------------------------------------------------
11 // 参数: 分频计数器上限(可被上层例化时覆盖，adda 中传入 24999)
12 //----------------------------------------------------------------------------
13     parameter cnt_max=15'd24999   //500分频
14 )
15
16 (
17     input wire sys_clk,
18     input wire sys_rst_n,
19
```

```
    //output wire clk_flag
    output reg clk0,
    output reg clk1

);

//------------------------------------------------------------------------------
// 内部信号: 15bit 分频计数器, 0~cnt_max 循环
//------------------------------------------------------------------------------
reg [14:0] cnt;

//------------------------------------------------------------------------------
// [块 1] 分频计数器: 0..cnt_max 自由循环, 是两路输出的公共节拍。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        cnt<=15'd0;
    else if(cnt==cnt_max)
        cnt<=15'd0;
    else
        cnt<=cnt+15'd1;

//------------------------------------------------------------------------------
// (已废弃) 原设计输出单周期 clk_flag 作为'分频完成'脉冲, 现改为直接翻转时钟, 保留注释。
//------------------------------------------------------------------------------
/*always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        clk_flag<=1'b0;
    else if(cnt==cnt_max-1)
        clk_flag<=1'b1;
    else
        clk_flag<=1'b0;*/

//------------------------------------------------------------------------------
// [块 2] 输出时钟 0: 每计满一次翻转, 占空比 50%,
// 频率 = 50MHz/(2*(24999+1)) = 1kHz。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        clk0<=1'b0;
    else if(cnt==cnt_max)
        clk0<=~clk0;
    else
        clk0<=clk0;

//------------------------------------------------------------------------------
// [块 3] 输出时钟 1: 与 clk0 完全同步、同相, 供另一路(DA)使用。
// 说明: 用普通寄存器分频得到的时钟会走全局时钟网络, 建议在时序约束中为其
//       建立 create_generated_clock 或改用 PLL/时钟使能方式(见文档'改进建议')。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        clk1<=1'b0;
    else if(cnt==cnt_max)
        clk1<=~clk1;
    else
        clk1<=clk1;
```

```
77
78 //----------------------------------------------------------------------------
79 // 附注：该模块输出直接驱动 ad_clk1/da_clk1 引脚，未做 ODDR 处理，
80 // 1kHz 的低速场合无影响；若提高频率需考虑时钟输出相位与抖动。
81 //----------------------------------------------------------------------------
82 endmodule
```

6 按键消抖模块 key（src/key.v）

6.1 文件信息

文件路径	FPGA/src/key.v（注释后 61 行）
模块 / 实例	key / key_inst
功能	20ms 软件消抖，输出“按一次翻转一次”的电平
参数	CNT_MAX=999999

6.2 第 1 层：接口

端口	方向	位宽	说明
key_in	input	1	按键输入，1= 松开，0= 按下（低有效）
sys_clk	input	1	50MHz
sys_rst_n	input	1	异步复位，低有效
key_out	output	1	翻转式状态：每次有效按下翻转一次

6.3 第 3 层：语句块解析

6.3.1 块 1：消抖计时（29–44 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        cnt_20ms<=20'd0;
    else if(key_in==1'b1) //没按下保持 0
        cnt_20ms<=20'd0;
    else if(key_in==1'b0 && cnt_20ms==CNT_MAX) //非抖动确定按下，保持计数值
        cnt_20ms<=cnt_20ms;
    else
        cnt_20ms<=cnt_20ms+1'b1; //按下后计数 20ms
```

计数器的行为可以总结为三种情况：

条件	计数器动作	含义
key_in=1	清零	松开即重新计时（消抖关键）
按下且未计满	+1	累计按住时间
按下且已计满	保持	防止溢出，锁定“已确认按下”状态

由于松开立刻清零，按键抖动期间的短暂按下无法累计到 20ms，从而被过滤掉。

6.3.2 块 2：翻转输出（46–59 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        key_out <= 1'b1;
    else if(cnt_20ms == CNT_MAX - 1'b1)
        key_out <= ~key_out;
    else
        key_out <= key_out;
```

在“计满前一拍”（999999）翻转 key_out，此时按键已稳定按下约 20ms。因为计满后计数器保持 1000000 不再回到 999999，所以一次按住只会翻转一次；松开后计数器清零，下一次按下重新开始。复位值为 1（未按状态）。

表 13: 消抖时间

量	公式	数值
消抖计数	CNT_MAX+1	1 000 000 拍
消抖时间	10^6/50MHz	20.0 ms
可识别的最短按键	$>$ 20ms	20 ms

6.4 本章小结

这是一个标准的“计数消抖 + 翻转输出”实现，缺点是按键按住期间计数器停在 1000000，若松开时抖动会短暂清零/重新计数，但不影响已翻转的结果。若需要“长按连发”，可以在块 2 中增加“按住超过 1s 后周期翻转”的分支。

6.5 完整注释源码

```
1 //============================================================================
2 // 文件: key.v            模块: key            实例名: key_inst(adda 顶层)
3 // 功能: 机械按键消抖(20ms)并输出稳定的按键电平。
4 //   输入 key_in = 1 表示松开, 0 表示按下(低有效按键)。
5 //   未按下时计数器清零; 持续按下满 20ms 后 key_out 翻转一次并锁存。
6 //   因此 key_out 是'按一次变一次'的翻转式状态, 而不是电平跟随。
7 // 时钟: sys_clk = 50MHz
```

```
8  //============================================================================
9  module key
10 (
11     input wire key_in,
12     input wire sys_clk,
13     input wire sys_rst_n,
14
15     output reg key_out
16 );
17
18 //----------------------------------------------------------------------------
19 // 内部信号: cnt_20ms 为 20ms 消抖计数器(0~999999)
20 //----------------------------------------------------------------------------
21 reg [19:0] cnt_20ms;
22
23 //----------------------------------------------------------------------------
24 // 参数: CNT_MAX = 999999, 50MHz 下 (999999+1)/50MHz = 20ms
25 //----------------------------------------------------------------------------
26 parameter CNT_MAX=20'd999_999;
27
28 //----------------------------------------------------------------------------
29 // [块 1] 消抖计时
30 //   key_in=1(松开)      : 计数清零, 重新计时
31 //   按下且已计满 20ms   : 保持计数值(避免溢出)
32 //   按下但未满 20ms     : 继续 +1
33 // 由于松开即清零, 抖动期间的短脉冲无法计满 20ms, 从而实现消抖。
34 //----------------------------------------------------------------------------
35 always@(posedge sys_clk or negedge sys_rst_n)
36     if(sys_rst_n==1'b0)
37         cnt_20ms<=20'd0;
38     else if(key_in==1'b1) //没按下保持 0
39         cnt_20ms<=20'd0;
40     else if(key_in==1'b0 && cnt_20ms==CNT_MAX) //非抖动确定按下, 保持计数值
41         cnt_20ms<=cnt_20ms;
42     else
43         cnt_20ms<=cnt_20ms+1'b1; //按下后计数 20ms
44
45 //----------------------------------------------------------------------------
46 // [块 2] 翻转输出
47 // 计到 CNT_MAX-1 的那一拍翻转 key_out(此时已稳定按下约 20ms)。
48 // 注意: 计满后计数值保持在 CNT_MAX(块 1), 不会再次触发, 保证按一次只翻转一次。
49 //----------------------------------------------------------------------------
50 always@(posedge sys_clk or negedge sys_rst_n)
51     if(sys_rst_n == 1'b0)
52         key_out <= 1'b1;
53     else if(cnt_20ms == CNT_MAX - 1'b1)
54         key_out <= ~key_out;
55     else
56         key_out <= key_out;
57
58 //----------------------------------------------------------------------------
59 // 用法: adda 中用 key_out==0 判断'按键状态', 从而在两种 DA 输出之间切换。
60 //----------------------------------------------------------------------------
61 endmodule
```

7 串口接收模块 uart_rx（src/uart_rx.v）

7.1 文件信息

文件路径	FPGA/src/uart_rx.v（注释后 187 行）
模块 / 实例	uart_rx / uart_rx_inst
帧格式	8 数据位、1 停止位、无校验（8N1），LSB 先到
参数	UART_BPS=115200，CLK_FREQ=50 000 000
派生参数	BAUD_CNT_MAX=434

7.2 第 1 层：接口与参数

端口	方向	位宽	说明
sys_clk	input	1	50MHz
sys_rst_n	input	1	异步复位
rx	input	1	串口输入（异步信号）
po_data	output	8	收到的字节
po_flag	output	1	字节有效脉冲，宽 1 个时钟

分频参数计算：

$$\text{BAUD_CNT_MAX} = \left\lfloor \frac{50\,000\,000}{115\,200} \right\rfloor = 434, \qquad \text{实际波特率} = \frac{50\,000\,000}{434} = 115\,207 \text{ bps}, \tag{9}$$

误差约 +0.006%，远小于 UART 允许的 ±2%。

7.3 第 3 层：语句块解析

7.3.1 块 1–3：三级同步器（57–82 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_reg1<=1'b1;
    else
        rx_reg1<=rx;
```

三个完全相同的寄存器串联：rx→rx_reg1→rx_reg2 →rx_reg3。作用是让异步的串口输入在本时钟域内“稳定”下来，消除亚稳态。串口空闲为高电平，因此复位值取 1。

7.3.2 块 4：起始位检测（84–94 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        start_flag<=1'b0;
```

```
    else if((rx_reg3==1'b1)&&(rx_reg2==1'b0)&&(work_en==1'b0))
        start_flag<=1'b1;
    else
        start_flag<=1'b0;
```

在同步后的信号上检测**下降沿**（前两拍为 1、上一拍为 0），且当前空闲（work_en=0）时产生单周期起始标志。用 rx_reg2/rx_reg3 错开一拍判断，可避免把抖动误判为起始位。

7.3.3 块 5：接收忙标志（96–108 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        work_en<=1'b0;
    else if(start_flag==1'b1)
        work_en<=1'b1;
    else if ((bit_cnt==5'd8)&&(bit_flag==1'b1))
        work_en<=1'b0;
    else
        work_en<=work_en;
```

检出起始位后置忙，收满 8 个数据位（bit_cnt=8）后释放。忙标志有两个作用：禁止在接收过程中重复触发起始检测；作为 baud_cnt 的清零条件。

7.3.4 块 6/7：位内计数与中点采样（110–131 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        baud_cnt<=16'd0;
    else if((baud_cnt==BAUD_CNT_MAX-1)||(work_en==1'b0))
        baud_cnt<=16'd0;
    else
        baud_cnt<=baud_cnt+16'd1;

always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_flag<=1'b0;
    else if(baud_cnt==BAUD_CNT_MAX/2-1)
        bit_flag<=1'b1;
    else
        bit_flag<=1'b0;
```

baud_cnt 在 0~433 循环；当它等于 $434/2-1=216$ 时产生 bit_flag，即在每个位的 ** 正中间 ** 采样——这是 UART 接收抗时钟偏差的标准做法。

表 14: 接收时序

量	公式	数值
位周期	434 个时钟	8.68 μs
采样时刻	第 216 个时钟	位中点
一帧（起始 +8 数据 + 停止）	10 位	86.8 μs

7.3.5 块 8/9：位计数与数据移位（133–152 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_cnt<=5'b0;
    else if((bit_cnt==5'd8)&&(bit_flag==1'b1))
        bit_cnt<=5'b0;
    else if(bit_flag==1'b1)
        bit_cnt<=bit_cnt+1'b1;

always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_data<=8'b0;
    else if(( bit_cnt<=5'd8)&&(bit_cnt>=5'd1)&&(bit_flag==1'b1))
        rx_data<={rx_reg3,rx_data[7:1]};
```

bit_cnt=0 对应起始位中点（不采样），1∼8 分别对应 $d_0 \sim d_7$ 的中点。每次采样把 rx_reg3 塞到最高位、整体右移一位，因此先到的 d_0 最终落在 bit0，符合 UART 的 LSB-first 约定。

7.3.6 块 10–12：完成脉冲与输出（154–184 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_flag<=1'b0;
    else if((bit_cnt==5'd8)&&(bit_flag==1'b1))
        rx_flag<=1'b1;
    else
        rx_flag<=1'b0;

always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        po_data<=8'b0;
    else if(rx_flag==1'b1)
        po_data<=rx_data;

always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
```

```
        po_flag<=1'b0;
    else
        po_flag<=rx_flag;
```

第 8 个数据位采完后产生 rx_flag，把移位结果锁存到 po_data，再打一拍得到 po_flag，使“数据 + 有效”同拍输出给顶层组帧逻辑。

边界条件：本实现不检查停止位、不做帧错误检测，也不做接收 FIFO。若上位机连续高速发送（间隔小于 1 个位周期），可能因 work_en 释放晚于下一帧起始沿而丢字节。对“偶发指令帧”的应用（本工程）足够；若要连续数据流收发，应加 FIFO 与帧错误标志。

7.4 完整注释源码

```
//============================================================================
// 文件: uart_rx.v          模块: uart_rx          实例名: uart_rx_inst(adda 顶层)
// 功能: 标准 UART 接收器(8 数据位、1 停止位、无校验), 位宽 8bit。
// 参数: UART_BPS=115200, CLK_FREQ=50_000_000
//       位周期计数值 BAUD_CNT_MAX = 50e6/115200 = 434 (实际波特率 115207, 误差 0.006%)
// 时序: 1) 3 级同步器消除亚稳态;
//       2) 检测 rx 下降沿作为起始位;
//       3) 在每个位周期的中点(第 216 拍)采样, 抗边沿抖动;
//       4) 第 8 位采完后输出 1 拍 po_flag。
//============================================================================
module uart_rx
#(
//----------------------------------------------------------------------------
// 参数: 波特率与系统时钟频率(顶层例化时覆盖为 115200 / 50MHz)
//----------------------------------------------------------------------------
    parameter UART_BPS ='d115200,         //波特率9600
    parameter CLK_FREQ ='d50_000_000    //50MHz
)

(
    input wire sys_clk,
    input wire sys_rst_n,
    input wire rx,

    output reg [7:0] po_data,
    output reg po_flag
);

//----------------------------------------------------------------------------
// 派生参数: 每位对应的系统时钟数 = 50MHz/115200 = 434
//----------------------------------------------------------------------------
parameter BAUD_CNT_MAX =CLK_FREQ/UART_BPS;      //计数周期最大值=频率/波特率

//----------------------------------------------------------------------------
// 内部信号
//   rx_reg1..3 : 三级同步寄存(异步输入 rx -> 系统时钟域)
//   start_flag : 起始位检测脉冲(单周期)
//   work_en    : 接收忙标志(1=正在接收一帧)
//   baud_cnt   : 位内计数 0..433
//   bit_flag   : 位中点采样脉冲(单周期)
//   bit_cnt    : 位计数 0..8(0=起始位, 1~8=数据位)
//   rx_data    : 移位接收缓冲(LSB 先到)
```

```
//   rx_flag    : 一帧接收完成脉冲
//-----------------------------------------------------------------------------
reg rx_reg1;
reg rx_reg2;
reg rx_reg3;
reg start_flag;
reg work_en;
reg[15:0]baud_cnt;
reg bit_flag;
reg[4:0] bit_cnt;
reg[7:0] rx_data;
reg rx_flag;

//-----------------------------------------------------------------------------
// [块 1] 同步器第 1 级: 直接寄存异步的 rx 引脚
//-----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_reg1<=1'b1;
    else
        rx_reg1<=rx;

//-----------------------------------------------------------------------------
// [块 2] 同步器第 2 级
//-----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_reg2<=1'b1;
    else
        rx_reg2<=rx_reg1;

//-----------------------------------------------------------------------------
// [块 3] 同步器第 3 级: rx_reg3 为'滤波后'的稳定电平, 供采样使用
//-----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_reg3<=1'b1;
    else
        rx_reg3<=rx_reg2;

//-----------------------------------------------------------------------------
// [块 4] 起始位检测: rx_reg3=1(前一拍高) 且 rx_reg2=0(本拍低) 且空闲,
// 产生单周期 start_flag。用 rx_reg2 判断下降了沿是为了再打一拍、避开亚稳态。
//-----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        start_flag<=1'b0;
    else if((rx_reg3==1'b1)&&(rx_reg2==1'b0)&&(work_en==1'b0))
        start_flag<=1'b1;
    else
        start_flag<=1'b0;

//-----------------------------------------------------------------------------
// [块 5] 接收忙标志: 检出起始位后置 1, 收满 8 个数据位(bit_cnt=8)后清 0。
// 该标志同时给 baud_cnt 做清零使能, 保证空闲时计数器归零。
//-----------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
```

```
    if(sys_rst_n==1'b0)
        work_en<=1'b0;
    else if(start_flag==1'b1)
        work_en<=1'b1;
    else if ((bit_cnt==5'd8)&&(bit_flag==1'b1))
        work_en<=1'b0;
    else
        work_en<=work_en;

//------------------------------------------------------------------------------
// [块 6] 位内计数: work_en=0 或计满时清零, 否则 +1
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        baud_cnt<=16'd0;
    else if((baud_cnt==BAUD_CNT_MAX-1)||(work_en==1'b0))
        baud_cnt<=16'd0;
    else
        baud_cnt<=baud_cnt+16'd1;

//------------------------------------------------------------------------------
// [块 7] 位中点采样脉冲: 计到 BAUD_CNT_MAX/2-1 = 216 时产生,
// 即每个位的中间时刻, 是对 UART 允许 ±2% 时钟偏差的经典处理。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_flag<=1'b0;
    else if(baud_cnt==BAUD_CNT_MAX/2-1)
        bit_flag<=1'b1;
    else
        bit_flag<=1'b0;

//------------------------------------------------------------------------------
// [块 8] 位计数: 0 为起始位, 1~8 为 8 个数据位, 计满 8 后回 0
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_cnt<=5'b0;
    else if((bit_cnt==5'd8)&&(bit_flag==1'b1))
        bit_cnt<=5'b0;
    else if(bit_flag==1'b1)
        bit_cnt<=bit_cnt+1'b1;

//------------------------------------------------------------------------------
// [块 9] 数据移位: bit_cnt 1~8 的中点各采样一次, LSB 在前(右移拼接)
//   rx_data <= {rx_reg3, rx_data[7:1]}  => 先到的位落在 bit0。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        rx_data<=8'b0;
    else if(( bit_cnt<=5'd8)&&(bit_cnt>=5'd1)&&(bit_flag==1'b1))
        rx_data<={rx_reg3,rx_data[7:1]};

//------------------------------------------------------------------------------
// [块 10] 完成脉冲: 第 8 位采样时刻置 rx_flag(单周期)
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
```

```
157     if(sys_rst_n==1'b0)
158         rx_flag<=1'b0;
159     else if((bit_cnt==5'd8)&&(bit_flag==1'b1))
160         rx_flag<=1'b1;
161     else
162         rx_flag<=1'b0;
163 
164 //------------------------------------------------------------------------------
165 // [块 11] 输出数据锁存(仅在 rx_flag 时更新, 保证数据稳定 1 个位周期以上)
166 //------------------------------------------------------------------------------
167 always@(posedge sys_clk or negedge sys_rst_n)
168     if(sys_rst_n==1'b0)
169         po_data<=8'b0;
170     else if(rx_flag==1'b1)
171         po_data<=rx_data;
172 
173 //------------------------------------------------------------------------------
174 // [块 12] 输出标志打一拍, 使 po_flag 与 po_data 同拍有效
175 //------------------------------------------------------------------------------
176 always@(posedge sys_clk or negedge sys_rst_n)
177     if(sys_rst_n==1'b0)
178         po_flag<=1'b0;
179     else
180         po_flag<=rx_flag;
181 
182 //------------------------------------------------------------------------------
183 // 附注: 本模块不检测帧错误与溢出(无 stop bit 校验), 连续高速收发时若上位机
184 // 发送过快可能丢字节, 已有实现对本工程(指令帧偶发)足够。
185 //------------------------------------------------------------------------------
186 endmodule
```

8 串口发送模块 uart_tx（src/uart_tx.v）

8.1 文件信息

文件路径	FPGA/src/uart_tx.v（注释后 136 行）
模块 / 实例	uart_tx / uart_tx_inst
功能	一次发送 16bit（两帧 8N1 连续发出）
参数	UART_BPS=115200，CLK_FREQ=50MHz，BAUD_CNT_MAX=434

8.2 第 1 层：接口

端口	方向	位宽	说明
sys_clk	input	1	50MHz
sys_rst_n	input	1	异步复位
pi_data	input	16	待发送数据（低字节先发）
pi_flag	input	1	发送启动（本工程恒为 1，自由连续发送）
tx	output	1	串口输出，空闲为 1

8.3 帧结构

一次“发送”占 21 个位时隙，等于把 16bit 拆成两帧 8N1 背靠背发出：

表 15: 发送时隙表（bit_cnt）

时隙	输出	含义
0	0	第 1 帧起始位
1–8	pi_data[0..7]	第 1 字节（LSB 先发）
9、10	1、1	两个停止位（帧间空闲）
11	0	第 2 帧起始位
12–19	pi_data[8..15]	第 2 字节
20	1	停止位

一帧耗时 $21 \times 434/50\text{MHz} = 182.3\,\mu\text{s}$。由于顶层把 pi_flag 接成常量 1，work_en 在每个位时隙都会重新置位，发送器实际上**不停机**，串口线上是连续的“16bit 数据 + 1 个额外停止位”流。

8.4 第 3 层：语句块解析

8.4.1 块 1：忙标志（48–58 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        work_en<=1'b0;
    else if(pi_flag==1'b1)
        work_en<=1'b1;
    else if((bit_cnt==5'd20)&&(bit_flag==1'b1))
        work_en<=1'b0;
```

pi_flag 到来即启动;发到最后一个时隙（停止位）后自动结束。注意这里没有 else work_en<=work_en，但因为有复位分支且条件覆盖完整（两个 if 都不成立时保持），综合结果仍是寄存器（保持行为）。

8.4.2 块 2：位内计数（60–69 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        baud_cnt<=16'd0;
    else if((work_en==1'b0)||(baud_cnt==BAUD_CNT_MAX-1))
        baud_cnt<=16'd0;
    else if(work_en==1'b1)
        baud_cnt<=baud_cnt+16'd1;
```

空闲清零，忙时 0∼433 循环，给出 8.68μs 的位节拍。

8.4.3 块 3：位起始脉冲（71–81 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_flag<=1'b0;
    else if(baud_cnt==16'd1)
        bit_flag<=1'b1;
    else
        bit_flag<=1'b0;
```

与接收端不同，发送端在 ** 位起始 **（计数 =1）而不是位中点动作，这样 tx 的电平切换发生在每位边界，留出整整一个位周期的稳定时间。

8.4.4 块 4：位序号（83–92 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_cnt<=5'd0;
    else if((bit_cnt==5'd20)&&(bit_flag==1'b1))
        bit_cnt<=5'd0;
    else if((bit_flag==1'b1)&&(work_en==1'b1))
        bit_cnt<=bit_cnt+5'd1;
```

0∼20 循环，共 21 个时隙。

8.4.5 块 5：发送数据查表（94–134 行）

```
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        tx<=1'b1;
    else if(bit_flag==1'b1)
        case(bit_cnt)
            0:tx<=1'b0;
            1:tx<=pi_data[0];
            ...
            9:tx<=1'b1;
            10:tx<=1'b1;
            11:tx<=1'b0;
            12:tx<=pi_data[8];
            ...
            20:tx<=1'b1;
            default:tx<=1'b1;
        endcase
```

用 case 直接把“时隙号”映射成“电平”，等价于一个 21 级状态机，但不需要状态寄存器。每

个时隙开始时（bit_flag=1）更新一次 tx。

为什么先发低字节？ case 中 pi_data[0..7] 在前（时隙 1–8），pi_data[8..15] 在后（时隙 12–19）。顶层 pi_data = {ad_date[7:0], 6'h0, ad_date[9:8]} 中低字节正是 ad_date[7:0]，因此上位机按小端（低字节先到）解析即可还原 AD 码值。

8.5 完整注释源码

```
//================================================================================
// 文件: uart_tx.v          模块: uart_tx          实例名: uart_tx_inst(adda 顶层)
// 功能: 标准 UART 发送器, 一次发送 16bit(拆成两帧 8N1 连续发出)。
// 帧结构(共 21 个位时隙):
//   0      : 起始位 0
//   1 ~ 8  : 第 1 字节 data[7:0](LSB 先发), 9~10: 两个停止位
//   11     : 第 2 帧起始位 0
//   12 ~ 19: 第 2 字节 data[15:8], 20: 停止位
// 参数: UART_BPS=115200, CLK_FREQ=50MHz -> 每位 434 个时钟
// 特点: pi_flag 常为 1 时发送器自由连续发送(本工程就是这样用的)。
//================================================================================
module uart_tx
#(
//--------------------------------------------------------------------------------
// 参数: 波特率与系统时钟(顶层覆盖为 115200 / 50MHz)
//--------------------------------------------------------------------------------
    parameter UART_BPS ='d115200,
    parameter CLK_FREQ ='d50_000_000
)

(
    input wire sys_clk,
    input wire sys_rst_n,
    input wire[15:0] pi_data,
    input wire pi_flag,

    output reg tx
);

//--------------------------------------------------------------------------------
// 派生参数: 每位 434 个系统时钟
//--------------------------------------------------------------------------------
parameter BAUD_CNT_MAX =CLK_FREQ/UART_BPS;

//--------------------------------------------------------------------------------
// 内部信号
//   work_en : 发送忙标志
//   baud_cnt: 位内计数 0..(BAUD_CNT_MAX-1)
//   bit_flag: 位起始脉冲(每个位周期产生一次)
//   bit_cnt : 位序号 0..20
//--------------------------------------------------------------------------------
reg work_en;
reg [15:0] baud_cnt;
reg bit_flag;
reg [4:0] bit_cnt;

//--------------------------------------------------------------------------------
// [块 1] 忙标志: pi_flag 到来即启动; 发完第 20 个时隙(停止位)后自动结束。
```

```
// 由于 pi_flag 在 adda 中恒为 1，结束当拍又会被置 1，形成连续发送。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        work_en<=1'b0;
    else if(pi_flag==1'b1)
        work_en<=1'b1;
    else if((bit_cnt==5'd20)&&(bit_flag==1'b1))
        work_en<=1'b0;

//------------------------------------------------------------------------------
// [块 2] 位内计数：空闲清零；忙时 0..433 循环
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        baud_cnt<=16'd0;
    else if((work_en==1'b0)||(baud_cnt==BAUD_CNT_MAX-1))
        baud_cnt<=16'd0;
    else if(work_en==1'b1)
        baud_cnt<=baud_cnt+16'd1;

//------------------------------------------------------------------------------
// [块 3] 位起始脉冲：baud_cnt=1 时置 1，
// 即每一位的第一个时钟产生 bit_flag，tx 在此时切换到下一个位的电平。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_flag<=1'b0;
    else if(baud_cnt==16'd1)
        bit_flag<=1'b1;
    else
        bit_flag<=1'b0;

//------------------------------------------------------------------------------
// [块 4] 位序号：0..20 循环，每来一个位脉冲 +1
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        bit_cnt<=5'd0;
    else if((bit_cnt==5'd20)&&(bit_flag==1'b1))
        bit_cnt<=5'd0;
    else if((bit_flag==1'b1)&&(work_en==1'b1))
        bit_cnt<=bit_cnt+5'd1;

//------------------------------------------------------------------------------
// [块 5] 发送移位输出(状态机式的 case 查表)
//   时隙 0      : 0           起始位
//   时隙 1~8    : pi_data[0..7]
//   时隙 9,10   : 1           停止位(第 1 帧结束，留 2 位空闲)
//   时隙 11     : 0           第 2 帧起始位
//   时隙 12~19  : pi_data[8..15]
//   时隙 20     : 1           停止位
// 复位空闲时 tx=1(UART 空闲电平)。
//------------------------------------------------------------------------------
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n==1'b0)
        tx<=1'b1;
```

```
    else if(bit_flag==1'b1)
        case(bit_cnt)
            0:tx<=1'b0;
            1:tx<=pi_data[0];
            2:tx<=pi_data[1];
            3:tx<=pi_data[2];
            4:tx<=pi_data[3];
            5:tx<=pi_data[4];
            6:tx<=pi_data[5];
            7:tx<=pi_data[6];
            8:tx<=pi_data[7];
            9:tx<=1'b1;
            10:tx<=1'b1;
            11:tx<=1'b0;
            12:tx<=pi_data[8];
            13:tx<=pi_data[9];
            14:tx<=pi_data[10];
            15:tx<=pi_data[11];
            16:tx<=pi_data[12];
            17:tx<=pi_data[13];
            18:tx<=pi_data[14];
            19:tx<=pi_data[15];
            20:tx<=1'b1;
            default:tx<=1'b1;
        endcase

//------------------------------------------------------------------------------
// 附注: 20 个位时隙 = 2.08ms/帧(115200bps), 上位机按 16bit 小端解析即可还原
// adda 中拼接的 {ad_date[7:0], 6'h0, ad_date[9:8]}。
//------------------------------------------------------------------------------
endmodule
```

9 数码管扫描显示模块 seg（src/seg.v）

9.1 文件信息

文件路径	FPGA/src/seg.v（注释后 231 行）
模块 / 实例	seg / seg_inst
功能	把 10bit 二进制数转成 BCD 后，按“符号 + 百位 + 十位 + 个位”动态扫描显示
参数	CNT_MAX=23999（位扫描间隔）
子模块	bcd（bcd_8421）

9.2 第 1 层：接口

9.3 第 2 层：内部结构

9.4 第 3 层：语句块解析

9.4.1 例化：bcd（37–50 行）

端口	方向	位宽	说明
clk	input	1	50MHz
rst_n	input	1	异步复位
point	input	4	小数点位置掩码，1= 该位亮小数点
data	input	10	待显示数值（0∼1023）
seg_en	input	1	显示使能（本工程恒为 1）
sign	input	1	1= 负数，在最高位显示负号
sel	output	4	位选（低有效）
seg	output	8	段选（bit7= 小数点，其余 a∼g）

信号	位宽	功能
unit/ten/hun	4	BCD 个/十/百位
data_reg	16	4 个待显示 nibble（0∼9 数字，10= 负号，11= 空白）
cnt_1ms	16	位扫描间隔计数
flag_1ms	1	位切换脉冲
cnt_sel	3	当前扫描位 0∼5
sel_reg	6	一热码位选移位寄存器
data_disp	4	当前位要显示的数字
dot_disp	1	当前位的小数点

```
bcd bcd_8421(
    .clk(clk),
    .rst_n(rst_n),
    .data(data),
    .unit(unit),
    .ten(ten),
    .hun(hun)
);
```

把 10bit 二进制转成 3 位 BCD（见第 10 章）。由于只能表示 0∼999，1000∼1023 会丢失千位（显示成 0∼23）。

9.4.2 块 1：显示内容组装（71–98 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_reg<=1'b0;
    else if(((hun) || (point[2])) && (sign == 1'b1))
        data_reg <= {4'd10,hun,ten,unit};
    else if(((hun) || (point[2])) && (sign == 1'b0))
        data_reg <= {4'd11,hun,ten,unit};
```

```
    else if(((ten) || (point[1])) && (sign == 1'b1))
        data_reg <= {4'd11,4'd10,ten,unit};
    ...
    else
        data_reg <= {4'd11,4'd11,4'd11,unit};
```

这一组 if 实现“前导零消隐 + 负号插入”，共四种宽度 × 两种符号：

表 16: 显示模式

条件	data_reg	显示（自左向右）	例
百位非 0 或需小数点，负数	{10,hun,ten,unit}	负号 +3 位	-1.23
百位非 0 或需小数点，正数	{11,hun,ten,unit}	空 +3 位	1.23
十位非 0 或需小数点，负数	{11,10,ten,unit}	空 + 负号 +2 位	-4.5
十位非 0 或需小数点，正数	{11,11,ten,unit}	空 + 空 +2 位	4.5
仅个位，负数	{11,11,10,unit}	空 + 空 + 负号 +1 位	-7
其他	{11,11,11,unit}	仅最右一位	7

注意本工程 point=4'b0100,point[2]=1 恒成立,因此永远走“3 位显示”分支:显示为符号 . 百 十 个，小数点固定在百位（第 3 位），形如 1.23。

9.4.3 块 2/3：位扫描节拍（100–122 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt_1ms<=1'b0;
    else if(cnt_1ms==CNT_MAX)
        cnt_1ms<=1'b0;
    else
        cnt_1ms<=cnt_1ms+1'b1;

always @(posedge clk or negedge rst_n)
    if(!rst_n)
        flag_1ms<=1'b0;
    else if(cnt_1ms==CNT_MAX-1'b1)
        flag_1ms<=1'b1;
    else
        flag_1ms<=1'b0;
```

cnt_1ms 计 0∼23999（24000 拍），在计满前一拍产生 flag_1ms。24000/50MHz = 0.48ms（注释写“1ms”是按 25MHz 时钟估的）。

9.4.4 块 4/5：扫描位序号与一热码（124–152 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt_sel<=1'b0;
    else if(cnt_sel==3'b101 && flag_1ms)
        cnt_sel<=1'b0;
    else if(flag_1ms)
        cnt_sel<=cnt_sel+1'b1;
    else
        cnt_sel<=cnt_sel;

always @(posedge clk or negedge rst_n)
    if(!rst_n)
        sel_reg<=4'b000_000;
    else if(!cnt_sel && flag_1ms)
        sel_reg<=4'b0001;
    else if(flag_1ms)
        sel_reg<=sel_reg<<1;
    else
        sel_reg<=sel_reg;
```

cnt_sel 每 0.48ms 前进一步，0→5 循环；sel_reg 是 6bit 一热码，每拍左移一位。表 17 给出完整循环。

表 17: 位选循环（sel = ~sel_reg[3:0]）

cnt_sel	sel_reg	sel	效果
0	000001	1110	最右位（个位）亮
1	000010	1101	十位亮
2	000100	1011	百位亮（小数点）
3	001000	0111	符号位亮
4	010000	1111	全灭（空时隙）
5	100000	1111	全灭（空时隙）

一轮 6 个时隙共 $6 \times 0.48 = 2.88$ms，刷新率约 347Hz，人眼无闪烁。

9.4.5 块 6/7：当前位数字与小数点（154–188 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_disp<=1'b0;
    else if(seg_en && flag_1ms)
        case(cnt_sel)
            3'd0:data_disp<=data_reg[3:0];
            3'd1:data_disp<=data_reg[7:4];
```

```
            3'd2:data_disp<=data_reg[11:8];
            3'd3:data_disp<=data_reg[15:12];
            default:data_disp<=1'b0;
        endcase
    else
        data_disp<=data_disp;

always @(posedge clk or negedge rst_n)
    if(!rst_n)
        dot_disp<=1'b1;
    else if(flag_1ms)
        dot_disp<=point[cnt_sel];
    else
        dot_disp<=dot_disp;
```

cnt_sel=0~3 依次取 data_reg 的四个 nibble,4/5 取默认 0(对应空时隙)。小数点取 point[cnt_sel]；注意 point 只有 4 位而 cnt_sel 可达 5，越界索引在综合时被当作 0（仿真时可能读到 X)。

9.4.6 块 8：段码译码（190–215 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        seg<=8'b0000_0000;
    else
        case(data_disp)
            4'd0:seg<={dot_disp,7'b0111111};
            4'd1:seg<={dot_disp,7'b0000110};
            ...
            4'd10:seg<=8'b0100_0000;
            4'd11:seg<=8'b0000_0000;
            default:seg<=8'b0011_1111;
        endcase
```

段码按“bit0=a, bit1=b, ⋯, bit6=g, bit7=dp”排列，1 表示点亮（共阴数码管)：

9.4.7 块 9：位选输出（217–229 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        sel<=4'b0_000;
    else
        sel<=~sel_reg;
```

sel 为 4bit，而 sel_reg 为 6bit，赋值时取低 4 位取反。因此 cnt_sel=4/5 时 sel=4’b1111（全灭)，恰好把两个空时隙实现为“消隐”，功能上正确，但属于隐式依赖位宽截断——建议显式写成 sel

表 18: 段码表

数字	段码（不含 dp）	十六进制	点亮的段
0	0111111	0x3F	a b c d e f
1	0000110	0x06	b c
2	1011011	0x5B	a b d e g
3	1001111	0x4F	a b c d g
4	1100110	0x66	b c f g
5	1101101	0x6D	a c d f g
6	1111101	0x7D	a c d e f g
7	0000111	0x07	a b c
8	1111111	0x7F	全亮
9	1101111	0x6F	a b c d f g
10（负号）	1000000	0x40	只亮 g（中间一横）
11（空白）	0000000	0x00	全灭

`<= ~sel_reg[3:0]` 并注释意图。

9.5 本章小结

- 显示逻辑采用“软件译码 + 动态扫描”，6 个时隙中只有 4 个真正驱动数码管，另 2 个时隙用于降低亮度/消隐；
- 前导零消隐与负号插入由块 1 的优先级 `if` 链实现，逻辑清晰但分支较多，综合后是一组 4 选 1 的 mux；
- 10bit 数据（0~1023）超过 3 位 BCD 的表示范围，满码附近显示会回绕。

9.6 完整注释源码

```
//============================================================================
// 文件: seg.v            模块: seg            实例名: seg_inst(adda 顶层)
// 功能: 4~6 位数码管动态扫描显示。把 10bit AD 码值经 bcd 转成 BCD 码后,
//       按'负号/空白 + 百位 + 十位 + 个位'的组合显示, 并支持小数点。
// 扫描: 1ms 时间基准(CNT_MAX=23999, 50MHz 下 0.48ms/位) -> 6 位一轮,
//       刷新率约 347Hz, 人眼无闪烁。
//============================================================================
module seg
#(
//----------------------------------------------------------------------------
// 参数: 位扫描间隔计数上限, 23999+1 = 24000 个时钟 = 0.48ms
// (原注释写 1ms, 对应 25MHz 时钟; 本工程 50MHz, 实际 0.48ms)
//----------------------------------------------------------------------------
    parameter CNT_MAX=16'd23999
)

(
```

```
    input wire clk,
    input wire rst_n,
    input wire [3:0]point,
    input wire [9:0]data, //? 位宽
    input wire seg_en,
    input wire sign,

    output reg [3:0]sel,
    output reg [7:0]seg
);

//------------------------------------------------------------------------------
// BCD 转换结果: unit 个位, ten 十位, hun 百位(0~9)
//------------------------------------------------------------------------------
wire [3:0]unit;
wire [3:0]ten;
wire [3:0]hun;

//------------------------------------------------------------------------------
// [例化] bcd_8421: 10bit 二进制 -> 3 位 BCD(移位加 3 法, 约 22 拍完成)
// 注意: 10bit 最大 1023, 而 3 位 BCD 只能表示 999, 千位溢出会被丢弃,
//       因此 AD 满码附近(>=1000)显示会回绕(如 1020 显示为 020)。
//------------------------------------------------------------------------------
bcd bcd_8421(
    .clk(clk),
    .rst_n(rst_n),
    .data(data),

    .unit(unit),
    .ten(ten),
    .hun(hun)
 );

//------------------------------------------------------------------------------
// 内部寄存器
//   data_reg : 4 个 4bit 的待显示数字(每位一个 BCD/控制码)
//              取值 0~9 为数字, 10=负号, 11=空白
//   cnt_1ms  : 位扫描时间计数器
//   flag_1ms : 位切换(单周期)脉冲
//   cnt_sel  : 当前扫描位序号 0~5
//   sel_reg  : 6bit 循环移位的一热码, 经取反后作为位选
//   data_disp: 当前位要显示的数字
//   dot_disp : 当前位的小数点
//------------------------------------------------------------------------------
reg [15:0]data_reg;
reg [15:0]cnt_1ms;
reg flag_1ms;
reg [2:0]cnt_sel;
reg [5:0]sel_reg;
reg [3:0]data_disp;
reg dot_disp;

//------------------------------------------------------------------------------
// [块 1] 显示内容组装(4 个 nibble)
//   规则: 高位为 0(不含前导零)则压缩显示位数, 负号占用最左一位。
//   hun!=0 或需显示小数点(point[2]) -> 显示 3 位: {符号,hun,ten,unit}
//   ten!=0 或 point[1]              -> 显示 2 位: {11,符号,ten,unit}
```

```
//   unit 或 point[0]              -> 显示 1 位: {11,11,符号,unit}
//   sign=1 时该位填 4'd10(负号), sign=0 时填 4'd11(空白)。
//----------------------------------------------------------------------------
//控制数码管显示
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_reg<=1'b0;
    //若显示的十进制数的百位为非零数据或需显示小数点，则值显示 3 个数码管
    else if(((hun) || (point[2])) && (sign == 1'b1))//显示负号
        data_reg <= {4'd10,hun,ten,unit};//4'd10 我们定义为显示负号
    else if(((hun) || (point[2])) && (sign == 1'b0))
        data_reg <= {4'd11,hun,ten,unit};
    //若显示的十进制数的十位为非零数据或需显示小数点，则值显示 2 个数码管
    else if(((ten) || (point[1])) && (sign == 1'b1))
        data_reg <= {4'd11,4'd10,ten,unit};
    else if(((ten) || (point[1])) && (sign == 1'b0))
        data_reg <= {4'd11,4'd11,ten,unit};
    //若显示的十进制数的个位且需显示负号
    else if(((unit) || (point[0])) && (sign == 1'b1))
        data_reg <= {4'd11,4'd11,4'd10,unit};
    //若上面都不满足都只显示一位数码管
    else
        data_reg <= {4'd11,4'd11,4'd11,unit};

//----------------------------------------------------------------------------
// [块 2] 位扫描节拍计数器: 0..CNT_MAX 循环，0.48ms 一轮
//----------------------------------------------------------------------------
//计数器计数 1ms
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt_1ms<=1'b0;
    else if(cnt_1ms==CNT_MAX)
        cnt_1ms<=1'b0;
    else
        cnt_1ms<=cnt_1ms+1'b1;

//----------------------------------------------------------------------------
// [块 3] 节拍脉冲: cnt_1ms 计满前一拍产生单周期 flag_1ms
//----------------------------------------------------------------------------
//计数标志位
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        flag_1ms<=1'b0;
    else if(cnt_1ms==CNT_MAX-1'b1)
        flag_1ms<=1'b1;
    else
        flag_1ms<=1'b0;

//----------------------------------------------------------------------------
// [块 4] 位序号: 0->1->2->3->4->5->0 循环，每 0.48ms 前进一步
//----------------------------------------------------------------------------
//cnt_sel: 从 0 到 5 的循环，用于选择当前显示的数码管
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt_sel<=1'b0;
    else if(cnt_sel==3'b101 && flag_1ms)
        cnt_sel<=1'b0;
```

```
    else if(flag_1ms)
        cnt_sel<=cnt_sel+1'b1;
    else
        cnt_sel<=cnt_sel;

//-------------------------------------------------------------------------------
// [块 5] 位选一热码: cnt_sel=0 时置 1, 之后每拍左移一位
// sel_reg 为 6bit, 对应 6 个数码管; 实际只接 4 位(sel[3:0]), 高 2 位等效于
// 空白时隙, 用于降低平均亮度/消除残影。
//-------------------------------------------------------------------------------
//数码管位选信号寄存器
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        sel_reg<=4'b000_000;
    else if(!cnt_sel && flag_1ms)
        sel_reg<=4'b0001;
    else if(flag_1ms)
        sel_reg<=sel_reg<<1;
    else
        sel_reg<=sel_reg;

//-------------------------------------------------------------------------------
// [块 6] 当前位数字选择(多路选择器)
//   cnt_sel=0 -> data_reg[3:0]  (个位/最右)
//   cnt_sel=1 -> data_reg[7:4]  (十位)
//   cnt_sel=2 -> data_reg[11:8] (百位)
//   cnt_sel=3 -> data_reg[15:12](符号/空白)
//   cnt_sel=4,5 -> 0 (空时隙)
//-------------------------------------------------------------------------------
//控制数码管的位选信号, 使六个数码管轮流显示
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_disp<=1'b0;
    else if(seg_en && flag_1ms)
        case(cnt_sel)
            3'd0:data_disp<=data_reg[3:0];
            3'd1:data_disp<=data_reg[7:4];
            3'd2:data_disp<=data_reg[11:8];
            3'd3:data_disp<=data_reg[15:12];
            default:data_disp<=1'b0;
        endcase
    else
        data_disp<=data_disp;

//-------------------------------------------------------------------------------
// [块 7] 小数点: 取 point[cnt_sel]
// 说明: point 只有 4bit, 而 cnt_sel 可到 5, 越界时综合为默认(0)。
//-------------------------------------------------------------------------------
//dot_disp:小数点也高电平点亮
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        dot_disp<=1'b1;
    else if(flag_1ms)
        dot_disp<=point[cnt_sel];
    else
        dot_disp<=dot_disp;

```

```
189 //------------------------------------------------------------------------------
190 // [块 8] 段选译码(共阴/共阳取决于硬件，这里是'1 点亮'的段码表)
191 //   bit7 = 小数点，bit6..0 = g,f,e,d,c,b,a
192 //   0~9 为标准七段码；4'd10 -> 8'b0100_0000(只亮中间一横，作为负号)；
193 //   4'd11 -> 8'b0000_0000(全灭，空白位)。
194 //------------------------------------------------------------------------------
195 //控制数码管段选信号，显示数字
196 always @(posedge clk or negedge rst_n)
197     if(!rst_n)
198         seg<=8'b0000_0000;
199     else
200         case(data_disp)
201             4'd0:seg<={dot_disp,7'b0111111};
202             4'd1:seg<={dot_disp,7'b0000110};
203             4'd2:seg<={dot_disp,7'b1011011};
204             4'd3:seg<={dot_disp,7'b1001111};
205             4'd4:seg<={dot_disp,7'b1100110};
206             4'd5:seg<={dot_disp,7'b1101101};
207             4'd6:seg<={dot_disp,7'b1111101};
208             4'd7:seg<={dot_disp,7'b0000111};
209             4'd8:seg<={dot_disp,7'b1111111};
210             4'd9:seg<={dot_disp,7'b1101111};
211             4'd10:seg<=8'b0100_0000;
212             4'd11:seg<=8'b0000_0000;
213             default:seg<=8'b0011_1111;
214         endcase
215
216 //------------------------------------------------------------------------------
217 // [块 9] 位选输出：sel = ~sel_reg(低电平有效的共阴数码管位选)
218 // 注意 sel 只有 4bit，6bit 的 sel_reg 取反后被截断 -> cnt_sel=4/5 时
219 // sel=4'b1111(全部熄灭)，正好实现两个空时隙，属'误打误撞'的正确结果。
220 //------------------------------------------------------------------------------
221 //sel: 数码管位选信号赋值
222 always @(posedge clk or negedge rst_n)
223     if(!rst_n)
224         sel<=4'b0_000;
225     else
226         sel<=~sel_reg;
227
228 //------------------------------------------------------------------------------
229 // 附注：sel_reg 初值写 6'b000_000，而 sel 初值 4'b0_000，复位期间全部熄灭。
230 //------------------------------------------------------------------------------
231 endmodule
```

10 二进制转 BCD 模块 bcd（src/bcd.v）

10.1 文件信息

文件路径	FPGA/src/bcd.v（注释后 98 行）
模块 / 实例	bcd / bcd_8421（在 seg 内）
算法	双 dabble / 移位加 3（Shift-Add-3）
输入/输出	10bit 二进制 → 3 位 BCD（0~999）
转换周期	24 个时钟（50MHz 下 480ns），连续转换，无握手

10.2 第 1 层：接口

端口	方向	位宽	说明
clk	input	1	50MHz
rst_n	input	1	异步复位
data	input	10	待转换的二进制数
unit	output	4	个位 BCD
ten	output	4	十位 BCD
hun	output	4	百位 BCD

10.3 算法原理

“移位加 3”法与手算二进制转十进制的“每移一位检查是否超过 4”过程完全一致：把二进制从高位到低位逐位左移进 BCD 区；每移入一位之前，若某个 BCD 位 > 4，就先给它 $+3$（因为移位相当于乘 2，$+3$ 后再乘 2 恰好实现十进制进位）。

本模块把“加 3”和“移位”拆到相邻两个时钟（shift_flag=0/1 交替），因此每个二进制位需要 2 拍，10 位共 20 拍，加上装入与收尾共 **24 拍**。

10.4 第 3 层：语句块解析

10.4.1 块 1：相位计数（32–43 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        cnt_shift<=1'b0;
    else if(cnt_shift==5'd11 && shift_flag)
        cnt_shift<=1'b0;
    else if(shift_flag)
        cnt_shift<=cnt_shift+1'b1;
    else
        cnt_shift<=cnt_shift;
```

cnt_shift 只在 shift_flag=1 时步进，0~11 循环；11 之后回到 0 开始下一次转换。

10.4.2 块 2：移位/修正主过程（45–67 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_shift<=1'b0;
    else if(!cnt_shift)
        data_shift<={12'b0,data}; //目标 BCD 码三个数 12 位+10 位待转换数据
    else if(cnt_shift<=10 && (!shift_flag)) //低判断
        begin
            data_shift[13:10]<=(data_shift[13:10]>4)? (data_shift[13:10]+2'd3):(data_shift
    [13:10]);
            ...
        end
    else if(cnt_shift<=5'd10 && shift_flag) //高移位
        data_shift<=data_shift<<1;
    else
        data_shift<=data_shift;
```

data_shift 共 22bit：高 12bit 是三位 BCD，低 10bit 是待转换二进制。四个分支的优先级就是转换时序：cnt_shift=0 装入；1~10 在 flag=0 时做加 3 修正、flag=1 时整体左移；11 时保持，等待输出。

10.4.3 块 3：相位标志（69–76 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        shift_flag<=1'b0;
    else
        shift_flag<=~shift_flag;
```

每拍翻转，把“修正”和“移位”严格分开，避免组合加法与移位在同一拍竞争。

10.4.4 块 4：结果锁存（78–96 行）

```
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        begin
            unit<=1'b0;
            ten<=1'b0;
            hun<=1'b0;
        end
    else if(cnt_shift==5'd11)
        begin
```

```
            unit<=data_shift[13:10];
            ten<=data_shift[17:14];
            hun<=data_shift[21:18];
        end
```

相位到 11 时把 12bit BCD 拆成三路输出，供 seg 使用。

10.5 转换实例

以 $753 = 10'b10_1111_0001$ 为例（BCD 区初值 000，逐位移入）：

表 19: 753 的双 dabble 过程（每步已含“加 3 修正”与“左移并入一位”）

步	移入位	BCD（十六进制）	十进制读数
0（装入）	—	0x000	000
1	1	0x001	001
2	0	0x002	002
3	1	0x005	005
4	1	0x011	011（个位 5 加 3 后移位）
5	1	0x023	023
6	1	0x047	047
7	0	0x094	094（十位 7 加 3 后移位）
8	0	0x188	188
9	0	0x376	376
10	1	0x753	753（正确）

同理可算出 $1023 = 10'b11_1111_1111$ 的最终结果是 0x023，即**千位被丢弃**，显示为“023”——这正是第 9 章所述满码回绕的根源。

位数不足：10bit 数据最大 1023，需要 4 位 BCD（16bit）；本模块只保留 3 位（12bit），因此 1000~1023 会溢出。若希望数码管正确显示 0~1023，应把 data_shift 扩到 26bit（BCD 16bit + 数据 10bit），并增加第四组“加 3”判断与 thou 输出，同时把 seg 的 data_reg 扩到 5 个 nibble。

10.6 完整注释源码

```
1  //============================================================================
2  // 文件: bcd.v            模块: bcd            实例名: bcd_8421(seg 内部)
3  // 功能: 二进制转 BCD 码(双 dabble / 移位加 3 法), 10bit 输入 -> 3 位十进制。
4  // 结构: data_shift = {BCD 12bit, 二进制 10bit} 共 22bit;
5  //       每个二进制位做一次'判断加 3 -> 左移', 共 10 次, 最后从高 12 位取结果。
6  // 时序: 用 shift_flag 把'加 3'和'移位'拆到相邻两个时钟, 每次转换约 22 拍
7  //       (50MHz 下约 440ns), 转换是连续进行的, 不做忙闲握手。
8  // 局限: 只能表示 0~999, 输入 1000~1023 会丢失千位。
9  //============================================================================
10 module bcd
11 (
```

```
    input wire clk,
    input wire rst_n,
    input wire [9:0]data,

    output reg [3:0]unit,
    output reg [3:0]ten,
    output reg [3:0]hun
);

//------------------------------------------------------------------------------
// 内部信号
//   cnt_shift : 转换相位计数 0~11
//   data_shift: {BCD[21:10], 二进制[9:0]} 移位寄存器
//   shift_flag: 0=执行'加 3 修正', 1=执行'左移'
//------------------------------------------------------------------------------
reg [4:0]cnt_shift;
reg [21:0]data_shift;
reg shift_flag; //移位标志信号

//------------------------------------------------------------------------------
// [块 1] 相位计数: shift_flag=1 时步进, 到 11 后回 0
//------------------------------------------------------------------------------
always @(posedge clk or negedge rst_n) //0-11 循环
    if(!rst_n)
        cnt_shift<=1'b0;
    else if(cnt_shift==5'd11 && shift_flag)
        cnt_shift<=1'b0;
    else if(shift_flag)
        cnt_shift<=cnt_shift+1'b1;
    else
        cnt_shift<=cnt_shift;

//------------------------------------------------------------------------------
// [块 2] 移位/修正主过程
//   cnt_shift=0            : 装入 {12'b0, data}
//   cnt_shift<=10 且 flag=0: 三个 BCD 位分别'大于 4 则加 3'
//   cnt_shift<=10 且 flag=1: 整体左移一位
//   cnt_shift=11           : 保持, 等待块 4 输出
// 说明: 0 相位会持续 2 拍(flag 的两种取值), 因此数据被重复装入一次, 无害。
//------------------------------------------------------------------------------
always @(posedge clk or negedge rst_n)
    if(!rst_n)
        data_shift<=1'b0;
    else if(!cnt_shift)
        data_shift<={12'b0,data}; //目标 BCD 码三个数 12 位+10 位待转换数据
    else if(cnt_shift<=10 && (!shift_flag)) //低判断
        begin
            data_shift[13:10]<=(data_shift[13:10]>4)? (data_shift[13:10]+2'd3):(data_shift[13:10]); //大于 4 就
     加 3, 否则不变
            data_shift[17:14]<=(data_shift[17:14]>4)? (data_shift[17:14]+2'd3):(data_shift[17:14]);
            data_shift[21:18]<=(data_shift[21:18]>4)? (data_shift[21:18]+2'd3):(data_shift[21:18]);
        end
    else if(cnt_shift<=5'd10 && shift_flag) //高移位
        data_shift<=data_shift<<1;
    else
        data_shift<=data_shift;

```

```
68 //----------------------------------------------------------------------------
69 // [块 3] 相位标志：每个时钟翻转一次，与块 1 配合形成 加3/移位 交替节拍
70 //----------------------------------------------------------------------------
71 always @(posedge clk or negedge rst_n)
72     if(!rst_n)
73         shift_flag<=1'b0;
74     else
75         shift_flag<=~shift_flag; //移位标志信号
76 
77 //----------------------------------------------------------------------------
78 // [块 4] 结果锁存：第 11 相位把 12bit BCD 拆成 unit/ten/hun 输出
79 //----------------------------------------------------------------------------
80 always @(posedge clk or negedge rst_n)
81     if(!rst_n)
82         begin
83             unit<=1'b0;
84             ten<=1'b0;
85             hun<=1'b0;
86         end
87     else if(cnt_shift==5'd11)
88         begin
89             unit<=data_shift[13:10];
90             ten<=data_shift[17:14];
91             hun<=data_shift[21:18];
92         end
93 
94 //----------------------------------------------------------------------------
95 // 附注：三组 BCD 判断都在同一拍完成，组合路径较长(约 5 级比较+加法)，
96 // 50MHz 下时序裕量充足；若提高主频需做流水化处理。
97 //----------------------------------------------------------------------------
98 endmodule
```

11 PLL IP：Gowin_PLL（src/gowin_pll/gowin_pll.v）

11.1 文件信息

文件路径	FPGA/src/gowin_pll/gowin_pll.v（注释后 171 行）
模块名	Gowin_PLL
来源	Gowin IP Core Generator 生成（gowin_pll.ipc 为其配置）
状态	已加入工程，但顶层例化被注释 ⇒ **不参与综合**（PLL 占用 0/6）
原设计用途	产生 ad_clk1/da_clk1（6.25MHz），后被 divider 的 1kHz 取代

11.2 第 1 层：接口

```
module Gowin_PLL (clkout0, clkout1, clkin);
output clkout0;
output clkout1;
input clkin;
```

lock、clkout2..6、mdrdo 等是内部连线，未引出端口，综合时被裁剪。

端口	方向	位宽	说明
clkin	input	1	参考时钟，配置为 50MHz
clkout0	output	1	输出 0，配置为 6.25MHz
clkout1	output	1	输出 1，配置为 6.25MHz

11.3 第 2 层：结构与硬件参数

文件由三部分组成：

1. **端口与内部线**（31–48 行）：声明未使用的 PLL 输出/状态线；
2. **PLLA 原语例化**（49–77 行）：真正占用 GW5A 芯片 PLL 硬核的单元；
3. **defparam 参数区**（87–169 行）：约 80 条硬件参数。

11.3.1 PLLA 原语例化（49–77 行）

```
PLLA PLLA_inst (
    .LOCK(lock),
    .CLKOUT0(clkout0),
    .CLKOUT1(clkout1),
    .CLKOUT2(clkout2),
    ...
    .CLKIN(clkin),
    .CLKFB(gw_gnd),
    .RESET(gw_gnd),
    .PLLPWD(gw_gnd),
    ...
    .MDWDI({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd})
);
```

所有不需要的控制端口（复位、掉电、动态相移、扩频、MDIO 配置等）都接到常量 0（gw_gnd）。CLKFB 接地表示使用内部反馈。

11.3.2 关键 defparam（87 行起）

由此可算出：

$$f_{\mathrm{VCO}} = 50\mathrm{MHz} \times \frac{16}{1} = 800\mathrm{MHz}, \qquad f_{\mathrm{out}} = \frac{800\mathrm{MHz}}{128} = 6.25\mathrm{MHz}. \tag{10}$$

与 gowin_pll.ipc 中的 Clkout0ExpectedFrequency=6.25、Clkout1ExpectedFrequency=6.25 完全一致。

11.4 为什么它没有进入网表

adda.v 第 386 行附近的 PLL 例化被注释掉（/* ... */）：

表 20: PLL 主要参数

参数	值	含义
FCLKIN	"50"	输入频率 50MHz
IDIV_SEL	1	输入分频 = 1
FBDIV_SEL	1	反馈分频 = 1
MDIV_SEL	16	VCO 倍频 = 16
ODIV0_SEL / ODIV1_SEL	128 / 128	输出分频 = 128
CLKOUT0_EN/CLKOUT1_EN	TRUE	只启用两路输出
CLKOUT2_EN..6	FALSE	其余关闭
CLKFB_SEL	"INTERNAL"	内部反馈
SSC_EN	FALSE	关闭扩频时钟
DYN_PE0..6_SEL	FALSE	关闭动态相位调整

```
/*Gowin_PLL pll_inst(
    .clkout0(ad_clk1), //output clkout0
    .clkout1(da_clk1), //output clkout1
    .clkin(sys_clk) //input clkin
); */
```

原因很清楚：当前工程用 divider 输出 1kHz 作为 AD/DA 转换时钟（慢速演示），6.25MHz 的 PLL 输出用不上。综合资源报告里 PLL 为 0/6，也印证了这一点。

若要把 PLL 用起来：
(1) 取消 adda.v 中 pll_inst 的注释，并把 ad_clk1/da_clk1 改由 PLL 驱动；
(2) 删除或旁路 divider 例化（两者不能同时驱动同一输出）；
(3) 在时序约束里为 clkin 建立 create_clock -period 20，并为 PLL 输出加 create_generated_clock；
(4) 若下游逻辑仍按 sys_clk 设计，需要注意跨时钟域（6.25MHz 与 50MHz 不同源）。

11.5 完整注释源码

```
1  //============================================================================
2  //============================================================================
3  // 文件: gowin_pll.v       模块: Gowin_PLL      状态: 已加入工程但**未例化**
4  // 类型: Gowin IP Generator 生成的 PLL 高级模式(rPLL -> GW5A 的 PLLA 原语)封装
5  // 参数(来自 gowin_pll.ipc): CLKIN = 50MHz, IDIV = 1, MDIV = 16,
6  //   CLKOUT0 = CLKOUT1 = 6.25MHz (VCO = 50MHz x 16 = 800MHz, 800/128 = 6.25MHz)
7  // 用途: 原设计用它产生 ad_clk1/da_clk1; 后来改用 divider 的 1kHz 分频,
8  //       顶层中该例化被注释掉, 因此本文件不参与综合(PLL 占用 0/6)。
9  // 说明: 本文件由 IP 工具生成, 只可读, 不要手工修改参数; 改参数请在
10 //       Gowin IP Core Generator 中重新配置 gowin_pll.ipc 后再生成本文件。
11 //============================================================================
12 //Copyright (C)2014-2023 Gowin Semiconductor Corporation.
13 //All rights reserved.
14 //File Title: IP file
15 //Tool Version: V1.9.9 (64-bit)
```

```
//Part Number: GW5A-LV25UG324C2/I1
//Device: GW5A-25
//Device Version: A
//Created Time: Wed Oct 22 16:48:35 2025

//------------------------------------------------------------------------------
// 模块端口: clkin 输入; clkout0/clkout1 两路输出; (lock 等未引出)
//------------------------------------------------------------------------------
module Gowin_PLL (clkout0, clkout1, clkin);

output clkout0;
output clkout1;
input clkin;

//------------------------------------------------------------------------------
// 未被使用的 PLL 输出/状态线(悬空), 由综合工具裁剪
//------------------------------------------------------------------------------
wire lock;
wire clkout2;
wire clkout3;
wire clkout4;
wire clkout5;
wire clkout6;
wire clkfbout;
wire [7:0] mdrdo;
wire gw_gnd;

//------------------------------------------------------------------------------
// 常量 0: 把不需要的 PLL 控制端口(复位、掉电、动态移相等)全部接地
//------------------------------------------------------------------------------
assign gw_gnd = 1'b0;

//------------------------------------------------------------------------------
// [例化] GW5A 的 PLLA 原语: 这是真正占用芯片 PLL 硬核的单元
// .CLKFB 接 gnd 表示使用内部反馈(CLKFB_SEL="INTERNAL")
//------------------------------------------------------------------------------
PLLA PLLA_inst (
    .LOCK(lock),
    .CLKOUT0(clkout0),
    .CLKOUT1(clkout1),
    .CLKOUT2(clkout2),
    .CLKOUT3(clkout3),
    .CLKOUT4(clkout4),
    .CLKOUT5(clkout5),
    .CLKOUT6(clkout6),
    .CLKFBOUT(clkfbout),
    .MDRDO(mdrdo),
    .CLKIN(clkin),
    .CLKFB(gw_gnd),
    .RESET(gw_gnd),
    .PLLPWD(gw_gnd),
    .RESET_I(gw_gnd),
    .RESET_O(gw_gnd),
    .PSSEL({gw_gnd,gw_gnd,gw_gnd}),
    .PSDIR(gw_gnd),
    .PSPULSE(gw_gnd),
    .SSCPOL(gw_gnd),
```

```
    .SSCON(gw_gnd),
    .SSCMDSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
    .SSCMDSEL_FRAC({gw_gnd,gw_gnd,gw_gnd}),
    .MDCLK(gw_gnd),
    .MDOPC({gw_gnd,gw_gnd}),
    .MDAINC(gw_gnd),
    .MDWDI({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd})
);

//------------------------------------------------------------------------------
// [defparam 配置区] 下面每个 defparam 都是 IP 工具按 .ipc 生成的硬件参数
// 关键项: FCLKIN=50MHz, IDIV=1, FBDIV=1, MDIV=16, ODIV0/1=128
// 未启用的 CLKOUT2..6 全部 FALSE, 相位/占空比微调均为 0(默认)。
//------------------------------------------------------------------------------
defparam PLLA_inst.FCLKIN = "50";
defparam PLLA_inst.IDIV_SEL = 1;
defparam PLLA_inst.FBDIV_SEL = 1;
defparam PLLA_inst.ODIV0_SEL = 128;
defparam PLLA_inst.ODIV1_SEL = 128;
defparam PLLA_inst.ODIV2_SEL = 8;
defparam PLLA_inst.ODIV3_SEL = 8;
defparam PLLA_inst.ODIV4_SEL = 8;
defparam PLLA_inst.ODIV5_SEL = 8;
defparam PLLA_inst.ODIV6_SEL = 8;
defparam PLLA_inst.MDIV_SEL = 16;
defparam PLLA_inst.MDIV_FRAC_SEL = 0;
defparam PLLA_inst.ODIV0_FRAC_SEL = 0;
defparam PLLA_inst.CLKOUT0_EN = "TRUE";
defparam PLLA_inst.CLKOUT1_EN = "TRUE";
defparam PLLA_inst.CLKOUT2_EN = "FALSE";
defparam PLLA_inst.CLKOUT3_EN = "FALSE";
defparam PLLA_inst.CLKOUT4_EN = "FALSE";
defparam PLLA_inst.CLKOUT5_EN = "FALSE";
defparam PLLA_inst.CLKOUT6_EN = "FALSE";
defparam PLLA_inst.CLKFB_SEL = "INTERNAL";
defparam PLLA_inst.CLKOUT0_DT_DIR = 1'b1;
defparam PLLA_inst.CLKOUT1_DT_DIR = 1'b1;
defparam PLLA_inst.CLKOUT2_DT_DIR = 1'b1;
defparam PLLA_inst.CLKOUT3_DT_DIR = 1'b1;
defparam PLLA_inst.CLKOUT0_DT_STEP = 0;
defparam PLLA_inst.CLKOUT1_DT_STEP = 0;
defparam PLLA_inst.CLKOUT2_DT_STEP = 0;
defparam PLLA_inst.CLKOUT3_DT_STEP = 0;
defparam PLLA_inst.CLK0_IN_SEL = 1'b0;
defparam PLLA_inst.CLK0_OUT_SEL = 1'b0;
defparam PLLA_inst.CLK1_IN_SEL = 1'b0;
defparam PLLA_inst.CLK1_OUT_SEL = 1'b0;
defparam PLLA_inst.CLK2_IN_SEL = 1'b0;
defparam PLLA_inst.CLK2_OUT_SEL = 1'b0;
defparam PLLA_inst.CLK3_IN_SEL = 1'b0;
defparam PLLA_inst.CLK3_OUT_SEL = 1'b0;
defparam PLLA_inst.CLK4_IN_SEL = 2'b00;
defparam PLLA_inst.CLK4_OUT_SEL = 1'b0;
defparam PLLA_inst.CLK5_IN_SEL = 1'b0;
defparam PLLA_inst.CLK5_OUT_SEL = 1'b0;
defparam PLLA_inst.CLK6_IN_SEL = 1'b0;
defparam PLLA_inst.CLK6_OUT_SEL = 1'b0;
```

```
defparam PLLA_inst.DYN_DPA_EN = "FALSE";
defparam PLLA_inst.CLKOUT0_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT0_PE_FINE = 0;
defparam PLLA_inst.CLKOUT1_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT1_PE_FINE = 0;
defparam PLLA_inst.CLKOUT2_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT2_PE_FINE = 0;
defparam PLLA_inst.CLKOUT3_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT3_PE_FINE = 0;
defparam PLLA_inst.CLKOUT4_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT4_PE_FINE = 0;
defparam PLLA_inst.CLKOUT5_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT5_PE_FINE = 0;
defparam PLLA_inst.CLKOUT6_PE_COARSE = 0;
defparam PLLA_inst.CLKOUT6_PE_FINE = 0;
defparam PLLA_inst.DYN_PE0_SEL = "FALSE";
defparam PLLA_inst.DYN_PE1_SEL = "FALSE";
defparam PLLA_inst.DYN_PE2_SEL = "FALSE";
defparam PLLA_inst.DYN_PE3_SEL = "FALSE";
defparam PLLA_inst.DYN_PE4_SEL = "FALSE";
defparam PLLA_inst.DYN_PE5_SEL = "FALSE";
defparam PLLA_inst.DYN_PE6_SEL = "FALSE";
defparam PLLA_inst.DE0_EN = "FALSE";
defparam PLLA_inst.DE1_EN = "FALSE";
defparam PLLA_inst.DE2_EN = "FALSE";
defparam PLLA_inst.DE3_EN = "FALSE";
defparam PLLA_inst.DE4_EN = "FALSE";
defparam PLLA_inst.DE5_EN = "FALSE";
defparam PLLA_inst.DE6_EN = "FALSE";
defparam PLLA_inst.RESET_I_EN = "FALSE";
defparam PLLA_inst.RESET_O_EN = "FALSE";
defparam PLLA_inst.ICP_SEL = 6'bXXXXXX;
defparam PLLA_inst.LPF_RES = 3'bXXX;
defparam PLLA_inst.LPF_CAP = 2'b00;
defparam PLLA_inst.SSC_EN = "FALSE";

//--------------------------------------------------------------------------------
// 结束: 若要在本工程启用 PLL, 需在 adda.v 中取消 pll_inst 的注释,
// 并把 divider 移除(或将 PLL 输出作为 divider 的输入)。
//--------------------------------------------------------------------------------
endmodule //Gowin_PLL
```

12 FIFO IP：fifo_top（src/fifo_top/fifo_top.v）

12.1 文件信息

文件路径	FPGA/src/fifo_top/fifo_top.v（注释后 536 行，其中 445 行为加密数据）
模块名	fifo_top
来源	Gowin FIFO IP（配置见 fifo_top.ipc）
状态	已加入工程但**未例化**，不参与综合（BSRAM 占用 0%）

12.2 IP 配置（fifo_top.ipc）

表 21: FIFO 关键配置

配置项	值	含义
SIZE_W	10	数据位宽 10bit（与 AD 码值一致）
DEPTH_W	7	深度 $2^7 = 128$ 字
FWFT	true	首字直通（读使能即可出数）
ALEMPTY/ALFULL	false	不用几乎空/满标志
ECC	false	不开 ECC
RESET	false	无异步复位端口
Read_Write_Check_on_RAM	true	读写冲突检查
Synthesis_tool	GowinSynthesis	综合器

从配置可以反推它的用途：用 10bit 位宽、128 深度的双时钟 FIFO，把 AD 的采样数据从 ad_clk1 时钟域搬到 sys_clk 时钟域——这正是顶层被注释掉的 fifo_top_inst1 例化想做的事。

12.3 文件结构：加密 IP 的三段式

行号	内容	是否可读
1–22	版权头 + 本报告添加的注释	明文
29	`timescale 100 ps/100 ps	明文
30–38	`pragma protect begin_protected 等声明	明文
39–49	`pragma protect key_block（RSA 密钥块）	密文
50–479	`pragma protect data_block（AES-128-CFB）	密文
480	`pragma protect end_protected	明文
491–535	module fifo_top (...) 接口与内部例化	明文

12.3.1 加密声明段（30–49 行）

```
`pragma protect begin_protected
`pragma protect version="2.3"
`pragma protect author="default"
`pragma protect encrypt_agent="GOWIN"
`pragma protect encoding=(enctype="base64", line_length=76, bytes=256)
`pragma protect key_keyowner="GOWIN",key_keyname="GWK2023-09",key_method="rsa"
`pragma protect key_block
vUi58+GDvF3tn/uOD6R30LuygqDdwbKMl5gQq2yHYv2XJy1gjYOXchmWm3vQ4n6nTMfKPi2R95nA
...
```

这段是 IEEE 1735 风格的 IP 加密封装：**key_block** 用 RSA 封装 AES 密钥，**data_block** 是 AES-128-CFB 加密的 RTL 主体（24 416 字节，base64 每行 76 字符）。只有持有 Gowin 私钥的 EDA 工具能解密，第三方工具（例如安路 TD）只能看到明文接口。

12.3.2 明文模块声明（491–535 行）

```
module fifo_top (
  Data, WrClk, RdClk, WrEn, RdEn, Q, Empty, Full
);
input [9:0] Data;
input WrClk;
input RdClk;
input WrEn;
input RdEn;
output [9:0] Q;
output Empty;
output Full;
wire VCC; wire GND;
  \~fifo.fifo_top  fifo_inst (
    .RdClk(RdClk), .WrClk(WrClk),
    .WrEn(WrEn), .RdEn(RdEn),
    .Data(Data[9:0]),
    .Full(Full), .Empty(Empty), .Q(Q[9:0])
  );
  VCC VCC_cZ (.V(VCC));
  GND GND_cZ (.G(GND));
  GSR GSR (.GSRI(VCC));
endmodule
```

表 22: fifo_top 端口

端口	方向	位宽	说明
Data	input	10	写数据
WrClk	input	1	写时钟
RdClk	input	1	读时钟（可与写时钟异步）
WrEn	input	1	写使能
RdEn	input	1	读使能
Q	output	10	读数据
Empty	output	1	空标志
Full	output	1	满标志

内部还例化了三个全局原语：VCC/GND（常量）和 GSR（全局复位网络，GSRI=VCC 表示不复位）。这也是 Gowin IP 的常见写法。

注意：加密块内的 RTL 无法阅读、也无法添加注释；本报告只对加密块之外的明文部分加了注释，**没有改动任何密文字节**（如改动会导致 IP 解密校验失败）。若需要“可读可改”的 FIFO，建议用 IP 生成器重新生成一个非加密版本，或直接用 Verilog 手写一个同步/异步 FIFO。

12.4 完整注释源码（仅明文部分）

```
//============================================================================
//============================================================================
// 文件: fifo_top.v        模块: fifo_top       状态: 已加入工程但**未例化**
// 类型: Gowin FIFO IP(异步 FIFO, 10bit x 128 深度, FWFT 模式)
//       配置来自 fifo_top.ipc: SIZE_W=10, DEPTH_W=7, FWFT=true
// 重要: 本文件主体(第 21~462 行)是 `pragma protect 加密的 IP 数据块,
//       内容为 Gowin 加密的 RTL, 不可读也不可修改; 只有文件头的版权、
//       源文件索引表和文件尾的模块声明(第 464~504 行)是明文的。
// 本注释只加在加密块之外, 未改动任何受保护的字节。
// 用途: 原计划缓存 AD 采样数据(WrEn/RdEn/Q/Empty/Full), 顶层未例化。
//============================================================================
//
//Written by GowinSynthesis
//Tool Version "V1.9.9 (64-bit)"
//Fri Oct 24 16:45:53 2025

//Source file index table:
//file0 "\C:/Users/27429/Desktop/fpga2025/prj/project_2025_11_07/src/fifo_top/temp/FIFO/fifo_define.v"
//file1 "\C:/Users/27429/Desktop/fpga2025/prj/project_2025_11_07/src/fifo_top/temp/FIFO/fifo_parameter.v"
//file2 "\D:/GoWin_1_9_9/Gowin/Gowin_V1.9.9_x64/IDE/ipcore/FIFO/data/edc.v"
//file3 "\D:/GoWin_1_9_9/Gowin/Gowin_V1.9.9_x64/IDE/ipcore/FIFO/data/fifo.v"
//file4 "\D:/GoWin_1_9_9/Gowin/Gowin_V1.9.9_x64/IDE/ipcore/FIFO/data/fifo_top.v"
//----------------------------------------------------------------------------
// `timescale 与下面的 `pragma protect 段是 IP 加密载体:
//   - key_block: RSA 加密的密钥块(用于校验授权)
//   - data_block: AES-128-CFB 加密的 RTL 主体(base64 行长度 76)
// Gowin EDA 综合时解密; 第三方工具(如 TD)只能读到明文接口部分。
//----------------------------------------------------------------------------
`timescale 100 ps/100 ps
`pragma protect begin_protected
`pragma protect version="2.3"
`pragma protect author="default"
`pragma protect author_info="default"
`pragma protect encrypt_agent="GOWIN"
`pragma protect encrypt_agent_info="GOWIN Encrypt Version 2.3"

`pragma protect encoding=(enctype="base64", line_length=76, bytes=256)
`pragma protect key_keyowner="GOWIN",key_keyname="GWK2023-09",key_method="rsa"
`pragma protect key_block
vUi58+GDvF3tn/uOD6R30LuygqDdwbKMl5gQq2yHYv2XJy1gjYOXchmWm3vQ4n6nTMfKPi2R95nA
CBC47g831WaJlOmS3dIgCp/yMllKJftYzNGdAA4SVV4tkyPyb2xRVM62FpfbM3rsohLLj4ZMYyKs
+VJLIg8NgjhN/bmNa6ODxhM6z+zSMb7+letUIWUoZlDGqdj8J5qvMr8asmUpRDfQOsur2n8AKGuO
y9ggp641bOLeImuC1Dl9+Zp6cgCLwyEttyXlxhW+IFFloiyhLGVemqFk3pjK9VU5iSJgJA5DDv2f
5YzbBUUVJDVnC2+thClBIQOp+CFzbHmiKB/8ew==

`pragma protect encoding=(enctype="base64", line_length=76, bytes=24416)
`pragma protect data_keyowner="default-ip-vendor"
`pragma protect data_keyname="default-ip-key"
`pragma protect data_method="aes128-cfb"
```

```
`pragma protect data_block
lq/aZ0IbhdbsfnrdX8f5kcFn8xNrq0wSMy3VCgzQmBClVB/L2WwalmSwdjmPal0aFOc4EFyE43SB
UVzgwBP1y3ie/+qhu0qUqSNPICxx03fKx8mQlYBvDIUU4A/eOMREtYPW6+SmMNBGHM5No/BCan8+
1UhrQcCBCkFk4mBJVfM2PyDX3jSiCcBBkRmip9cPM+/RoRE8ZKpqlzlRBW8Mgaki9wjBLrwiOv1N
PN6zJESQ0s2Bkt8/CsJkJG+WCO37Qe0CYwiJYJFHv9Ciyp1uSV89LfIb6oyOAORlqscBILuJdSCc
JGst5EWqM3aKy1Iyz3CVutjXM1xAgjKH568US/Z4azR7+GnGHn0e8Ou69fQXROk8xl29ZWSWgCJL
kYEou3miHJE2Cw5+DHthlPtppTUlucNSO2rxxE+Gx68pLtF1OM5dPm2Cpgdim/0RASNe16LoDX8X
```

中间 57–479 行为 base64 密文（略）。明文接口部分如下：

```
`pragma protect end_protected
//------------------------------------------------------------------------------
// [明文接口] fifo_top: 异步 FIFO 顶层
//   Data[9:0] : 写数据(10bit)
//   WrClk/RdClk: 写/读时钟(可异步)
//   WrEn/RdEn : 写/读使能
//   Q[9:0]    : 读数据(FWFT 模式下读使能有效即出数)
//   Empty/Full: 空/满标志
// 内部: 例化加密模块 \~fifo.fifo_top(即 fifo.v 中的实现), 并驱动 VCC/GND/GSR
//       三个全局原语(GSR 为全局复位网络, GSRI=VCC 表示不复位)。
//------------------------------------------------------------------------------
module fifo_top (
  Data,
  WrClk,
  RdClk,
  WrEn,
  RdEn,
  Q,
  Empty,
  Full
)
;
input [9:0] Data;
input WrClk;
input RdClk;
input WrEn;
input RdEn;
output [9:0] Q;
output Empty;
output Full;
wire VCC;
wire GND;
  \~fifo.fifo_top  fifo_inst (
    .RdClk(RdClk),
    .WrClk(WrClk),
    .WrEn(WrEn),
    .RdEn(RdEn),
    .Data(Data[9:0]),
    .Full(Full),
    .Empty(Empty),
    .Q(Q[9:0])
);
  VCC VCC_cZ (
    .V(VCC)
);
  GND GND_cZ (
    .G(GND)
```

```
);
  GSR GSR (
    .GSRI(VCC)
);
//----------------------------------------------------------------------------
// 说明: endmodule 之后无其他逻辑; 若要在设计中使用, 请参考 fifo_top_tmp.v
// 中的例化模板, 并注意读写时钟域与使能的握手。
//----------------------------------------------------------------------------
endmodule /* fifo_top */
```

13 物理约束文件 project_2025_11_07.cst

13.1 文件信息

文件路径	FPGA/src/project_2025_11_07.cst（注释后 109 行）
类型	Gowin 物理约束（Physical Constraints），文本格式，行尾分号
作用	端口 → 器件引脚绑定 + I/O 电气属性
统计	30 条 IO_LOC + 30 条 IO_PORT，端口共 44 个

13.2 语法说明

Gowin 的物理约束只有两条核心命令：

```
IO_LOC  "端口名" 引脚号;
IO_PORT "端口名" 属性=值 ...;
```

关键字	含义	本例取值
IO_LOC	端口到引脚（或差分对的 P/N）的绑定	如"sys_clk" T9
PULL_MODE	内部上下拉	NONE（无上下拉）
DRIVE	输出驱动电流	8（8mA）
BANK_VCCIO	所在 Bank 的 I/O 电压	3.3（3.3V LC_MOS）

注意 IO_PORT 只作用于输出/双向端口的电气属性；输入端口虽然也写了 PULL_MODE/BANK_VCCIO，其中的 DRIVE 会被工具忽略。

13.3 引脚分配总表

13.3.1 用户约束的 30 个引脚

13.3.2 未约束、由工具自动分配的 12 个引脚

sel[3:0] 与 seg[7:0]（数码管位选/段选）在.cst 中**没有**约束。PnR 报告（pin.html）显示它们被自动布到了：

表 23: 时钟、复位、串口、按键、LED

端口	引脚	方向	说明
sys_clk	T9	in	50MHz 时钟（bank 4，专用时钟引脚）
sys_rst_n	A15	in	复位按键，低有效
rx	P18	in	串口接收
tx	N18	out	串口发送
key_in	B16	in	用户按键
led0	D14	out	状态指示
led6	C14	out	B 参数接收指示
led7	B9	out	A 参数接收指示
ad_clk1	H18	out	AD 转换时钟 1kHz
da_clk1	H13	out	DA 转换时钟 1kHz

表 24: DA 数据总线（ad_out[9:0]）

位	9	8	7	6	5	4	3	2	1	0
引脚	L13	F18	K13	G18	K14	D18	K16	G14	H14	C18

表 25: AD 数据总线（ad_date[9:0]）

位	9	8	7	6	5	4	3	2	1	0
引脚	C17	K15	F14	J13	D17	K12	G16	L12	F17	H17

端口	引脚	端口	引脚
seg[0]	U18	sel[0]	N16
seg[1]	L14	sel[1]	U17
seg[2]	P16	sel[2]	N14
seg[3]	V16	sel[3]	T17
seg[4]	P12		
seg[5]	N15		
seg[6]	V13		
seg[7]	R11		

风险：数码管引脚未写进约束文件，意味着每次重跑 PnR（尤其是更换版本、改动 unused_pin 策略后）都可能得到不同的引脚，最终码流上板后数码管可能不亮或乱码。由于工程已经生成过一版码流（当前引脚是那次 PnR 的结果），若要复现该板子的显示效果，应把上表 12 个引脚补写进.cst：

```
IO_LOC "seg[0]" U18;
IO_PORT "seg[0]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
...
IO_LOC "sel[3]" T17;
IO_PORT "sel[3]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
```

13.4 端口数与实际 Buffer 数的差异

PnR 报告给出“I/O Port 44、I/O Buf 42（14 输入 + 28 输出）”。差的 2 个是 ad_clk 与 da_clk：它们在 adda.v 中声明为输入端口，但整个设计**从未使用**，综合时被裁剪成悬空网络，因此没有生成输入 Buffer。这也解释了为什么.cst 中没有这两个端口的约束。

13.5 完整注释源码

```
//============================================================================
// 文件: project_2025_11_07.cst   类型: Gowin 物理约束(引脚/电平)
// 内容: 为 adda 顶层的 44 个端口分配 GW5A-25A(UBGA324) 的具体引脚,
//       并规定 I/O 电气属性(PULL_MODE / DRIVE / BANK_VCCIO)。
// 规则: IO_LOC  = 端口名 -> 器件引脚号(必须存在且不冲突);
//       IO_PORT = 电气属性, PULL_MODE=NONE 无上下拉, DRIVE=8mA 驱动,
//                 BANK_VCCIO=3.3 表示所在 Bank 的 IO 电压为 3.3V。
// 备注: 全部 44 个端口均工作在 3.3V LVCMOS; sys_clk 在 T9(专用时钟引脚)。
//============================================================================
//Copyright (C)2014-2023 Gowin Semiconductor Corporation.
//All rights reserved.
//File Title: Physical Constraints file
//Tool Version: V1.9.9 (64-bit)
//Part Number: GW5A-LV25UG324C2/I1
//Device: GW5A-25
//Device Version: A
//Created Time: Wed 10 22 16:26:14 2025

//----------------------------------------------------------------------------
// [LED 组] led7=B9 / led6=C14 / led0=D14, 3.3V 推挽输出, 8mA 驱动
// led0 为按键状态指示, led6/led7 为串口参数接收指示(见 adda.v)
//----------------------------------------------------------------------------
IO_LOC "led7" B9;
IO_PORT "led7" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "led6" C14;
IO_PORT "led6" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "led0" D14;
IO_PORT "led0" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
//----------------------------------------------------------------------------
// [串口发送] tx=N18 (115200bps, 8N1); 对应 rx=P18(见下方)
//----------------------------------------------------------------------------
IO_LOC "tx" N18;
IO_PORT "tx" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
//----------------------------------------------------------------------------
// [DA 数据总线] ad_out[9:0] -> L13/F18/K13/G18/K14/D18/K16/G14/H14/C18
// 注意: 比特顺序为 [9]->[0] 从上到下排列, 引脚均在 bank 0/1 的 3.3V 区。
```

```
//------------------------------------------------------------------------------
IO_LOC "ad_out[9]" L13;
IO_PORT "ad_out[9]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[8]" F18;
IO_PORT "ad_out[8]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[7]" K13;
IO_PORT "ad_out[7]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[6]" G18;
IO_PORT "ad_out[6]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[5]" K14;
IO_PORT "ad_out[5]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[4]" D18;
IO_PORT "ad_out[4]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[3]" K16;
IO_PORT "ad_out[3]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[2]" G14;
IO_PORT "ad_out[2]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[1]" H14;
IO_PORT "ad_out[1]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_out[0]" C18;
IO_PORT "ad_out[0]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
//------------------------------------------------------------------------------
// [AD/DA 转换时钟] da_clk1=H13, ad_clk1=H18(由 divider 产生 1kHz 方波)
//------------------------------------------------------------------------------
IO_LOC "da_clk1" H13;
IO_PORT "da_clk1" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "ad_clk1" H18;
IO_PORT "ad_clk1" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
//------------------------------------------------------------------------------
// [串口接收] rx=P18
//------------------------------------------------------------------------------
IO_LOC "rx" P18;
IO_PORT "rx" PULL_MODE=NONE BANK_VCCIO=3.3;
//------------------------------------------------------------------------------
// [按键输入] key_in=B16(内部无上下拉, 依赖板载上拉)
//------------------------------------------------------------------------------
IO_LOC "key_in" B16;
IO_PORT "key_in" PULL_MODE=NONE BANK_VCCIO=3.3;
//------------------------------------------------------------------------------
// [AD 数据总线] ad_date[9:0] -> C17/K15/F14/J13/D17/K12/G16/L12/F17/H17
// 这是外部 ADC 并行输出的 10bit 码值, 送 rtl(中值/电压) 与数码管显示
//------------------------------------------------------------------------------
IO_LOC "ad_date[9]" C17;
IO_PORT "ad_date[9]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[8]" K15;
IO_PORT "ad_date[8]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[7]" F14;
IO_PORT "ad_date[7]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[6]" J13;
IO_PORT "ad_date[6]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[5]" D17;
IO_PORT "ad_date[5]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[4]" K12;
IO_PORT "ad_date[4]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[3]" G16;
IO_PORT "ad_date[3]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[2]" L12;
```

```
IO_PORT "ad_date[2]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[1]" F17;
IO_PORT "ad_date[1]" PULL_MODE=NONE BANK_VCCIO=3.3;
IO_LOC "ad_date[0]" H17;
IO_PORT "ad_date[0]" PULL_MODE=NONE BANK_VCCIO=3.3;
//------------------------------------------------------------------------------
// [复位] sys_rst_n=A15, 低有效异步复位(按键复位)
//------------------------------------------------------------------------------
IO_LOC "sys_rst_n" A15;
IO_PORT "sys_rst_n" PULL_MODE=NONE BANK_VCCIO=3.3;
//------------------------------------------------------------------------------
// [系统时钟] sys_clk=T9, 50MHz 有源晶振输入, 走全局时钟网络
//------------------------------------------------------------------------------
IO_LOC "sys_clk" T9;
IO_PORT "sys_clk" PULL_MODE=NONE BANK_VCCIO=3.3;

//==============================================================================
// VGA 接口（由 src/vga/pins/scope_pins.csv 生成；引脚取自 PnR 报告
// All Package Pins 中 bank4 未使用的 8 个 IO, VCCIO=3.3V）
// HS/VS 直连显示器; RGB 经 270Ω(R1/G1/B1) + 560Ω(R0/G0/B0) 电阻网络
//==============================================================================
IO_LOC "vga_hs" T11;
IO_PORT "vga_hs" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_vs" T12;
IO_PORT "vga_vs" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_rgb[11]" U13;
IO_PORT "vga_rgb[11]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_rgb[10]" R10;
IO_PORT "vga_rgb[10]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_rgb[7]" U11;
IO_PORT "vga_rgb[7]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_rgb[6]" N12;
IO_PORT "vga_rgb[6]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_rgb[3]" V14;
IO_PORT "vga_rgb[3]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
IO_LOC "vga_rgb[2]" M11;
IO_PORT "vga_rgb[2]" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
```

14 设计审查、缺陷清单与改进建议

14.1 设计意图复原

综合代码与注释，可以还原出作者的设计意图：这是一块“AD 采样 → 观测/控制 → DA 输出”的 ADRC（自抗扰控制）教学实验板，核心链路为

AD 码值 → 去偏置 (-512) → ESO 观测/控制 → 偏移二进制 → DAC；
同一路 AD 数据同时送数码管显示、并每 $182\mu s$ 通过串口上报；
上位机用“Axxx/Bxxx”两字节帧在线修改 ESO 的 c、k；
按键切换“直通 AD 回环”与“输出控制量”两种工作模式。

14.2 缺陷与风险清单

编号	位置	现象	后果 / 建议
P1	adda.v 块 7/8 + eso.v 块 2	c_flag/k_flag 置 1 后不清零，而 ESO 参数更新为 if/else-if 优先级结构	收到过 A 帧后 k 参数永久无法更新。建议两个标志改为单周期脉冲，或 ESO 内改为两个独立 if
P2	eso.v 语句组 9	sgn 是 5bit signed，却赋 4'b1111（补零后 = +15）	符号项实际加 +15 而非 −1，控制量存在固定偏置。建议写 -5'sd1 或 she>0 ? 5'sd1 : -5'sd1
P3	eso.v 语句组 7	she 仅 33bit，cz_1(32bit) + z_2(22bit) 最大需 34bit	极端值截断丢符号位。建议加宽到 34bit
P4	bcd.v	10bit 输入只用 3 位 BCD（0∼999）	AD 码 ≥1000 时显示回绕（1023→023）。建议扩展为 4 位 BCD（26bit 移位寄存器）
P5	adda.v 块 3/4/例化 6	pi_flag 产生但无负载；uart_tx 的 pi_flag 固定 1	26ms 定时逻辑被综合删除；串口自由连续发送，占用 100% 串口带宽。若需定速上报应改接 pi_flag
P6	seg.v 块 5/块 9	sel_reg 6bit 而 sel 4bit，靠截断实现两个空时隙	功能正确但可读性差，建议显式取 sel_reg[3:0] 或把位选设计成 4 位一轮
P7	seg.v 块 7	point[cnt_sel] 中 cnt_sel 可达 5，而 point 只有 4 位	越界索引，仿真读 X、综合按 0，建议 point 改成 6 位或对索引做保护
P8	divider.v	普通寄存器输出直接当时钟用；注释“500 分频”与实际 50000 分频不符	高频下时钟质量差；注释误导。建议改用 PLL 或时钟使能，并修正注释
P9	adda.v 端口	ad_clk、da_clk 输入端口从未使用	悬空端口被裁剪，浪费 2 个引脚约束意图，建议删除
P10	adda.v 106 行	用 initial 初始化 po_flag2	可综合但可移植性差，建议改到所在时序块的复位分支

编号	位置	现象	后果 / 建议
P11	.cst	sel/seg 共 12 个引脚未约束，被自动布局	重新 PnR 后数码管引脚可能变化，建议补全约束（见第 13 章）
P12	工程文件列表	gowin_pll.v、fifo_top.v 未例化却参与工程	增加编译时间、误导资源评估，建议移出工程或标注

14.3 重点问题详解

14.3.1 P1：参数更新优先级导致 k 失效

```
// adda.v 块 7/8
k_flag<=1'b1;   // 置 1
// c_flag<=1'b0;  // 被注释：永不清零

// eso.v 块 2
if(c_flag==1'b1)      c<=c_date;
else if(k_flag==1'b1) k<=k_date;
```

时序上：若上位机先发 A 帧，c_flag 变 1 并保持；之后发 B 帧，k_flag=1，但 else if 分支永远进不去。只有在**从未收到 A 帧**的情况下 k 才能更新。修复示例：

```
// 方案一：标志脉冲化(顶层)
else if((rx_date[15:12]==4'hB)&&(po_flag3==1'b1))
    begin
        led6<=~led6;
        k_flag<=1'b1;      // 只在解析当拍为 1
        k_date<=rx_date[11:0];
    end
else
    k_flag<=1'b0;          // 下一拍自动清 0

// 方案二：ESO 内改为并行 if
if(c_flag) c<=c_date;
if(k_flag) k<=k_date;
```

14.3.2 P2：符号项的位宽陷阱

assign sgn = (she>0) ? 1'd1 : 4'b1111; 中，4'b1111 是**无符号**常量，赋给 5bit 有符号寄存器时按“补零”扩展为 5'b01111=+15。正确写法：

```
assign sgn = (she > 0) ? 5'sd1 : -5'sd1;
```

14.3.3 P4：BCD 位宽不足

详见第 10 章的转换实例：1023 的最终 BCD 结果是 0x023。修复需要把 data_shift 扩到 26bit、增加第 4 组“加 3”判断与 thou 输出，并把 seg 的 data_reg 扩到 5 个 nibble。

14.4 工程性改进建议

1. **补时序约束**：当前工程没有.sdc（Timing Constraints File 为“—”），50MHz 主时钟没有任何 create_clock。建议至少加 create_clock -name sys_clk -period 20 -waveform {0 10} [get_ports sys_clk]，并为 ad_clk1 加生成时钟约束；
2. **复位处理**：异步复位、异步释放存在恢复/移除时间风险，建议加“复位同步器”（两级触发器同步释放）；
3. **时钟域**：ESO 用 1kHz 分频时钟，属“派生时钟”用法。建议改为 sys_clk + 时钟使能（每 50000 拍使能一次），可避免时钟树布线与时序分析问题；
4. **参数集中**：把 50_000_000、115200、分频比等提到一个 define.vh 或用参数传递，避免三处手工同步；
5. **串口协议**：建议加帧头/校验（如 0xAA55 + 累加和），并给上位机一份字节序说明；
6. **文档与工程一致性**：把未例化的 IP 从工程移除，或建一个“参考”文件夹，避免后人误以为 PLL/FIFO 参与工作。

14.5 上板验证清单

验证项	现象	判据
时钟与复位	led7 初始为 1	复位释放后 LED 常亮/常灭一致
数码管显示	显示 3 位十进制	与输入码值对应（≥1000 会回绕）
按键切换	按一次 LED0 变一次	ad_out 在直通/控制量之间切换
串口上报	每 182μs 一帧	115200 8N1，16bit 小端
串口调参	发 A/B 帧	LED7/LED6 翻转；注意 P1 的 k 失效问题
DA 输出	示波器看 ad_out	直通模式下与 AD 输入反相

15 VGA 示波器子系统：架构与配置

15.1 概览与仓库地址

在原有 AD/DA 采集设计之上，本工程新增了一个 640×480 的 VGA 波形显示子系统（下称“VGA 示波器”），代码位于 FPGA/src/vga/。设计遵循“底层（HAL）/ 抽象（图形）/ 应用 / 顶层”四层结构，并把**引脚表、视频参数、特性开关、运行时参数**全部外置到配置文件或参数表里，RTL 中不含任何板级信息。

本工程全部源码、文档与本报告托管在：

https://github.com/starfruitsz/FPGA

表 27: 仓库目录结构

路径	内容
FPGA/src/	原 AD/DA 工程 11 个源文件 + vga/ 14 个新源文件
FPGA/src/vga/README.md	分层、开关、命令、时序、资源说明
FPGA/src/vga/pins/	引脚表 scope_pins.csv + 生成脚本 gen_cst.py
FPGA/project_2025_11_07.gprj	Gowin 工程（26 条文件记录）
FPGA_TD/	TD 容器工程与验证工程
doc/	本报告 LaTeX 源码
tools/	注释/端口检查/字模/预览生成脚本

15.2 分层结构与文件清单

数据通路为：外部 ADC 码值 ad_date 与 DAC 输出 ad_out 分别进入两个采样缓存，按可配置速率循环写入；显示时每个像素列读取相邻两个采样点，映射为 y 坐标后用一条竖线连接，形成波形。整条链路没有帧缓存。

15.3 特性开关

四个特性各自对应一个 localparam，集中在 adda_scope_top.v 顶部，改一行即可开关（0 = 关闭，1 = 开启）：

```
localparam integer FEAT_TRIG   = 1;   // 特性1 触发对齐
localparam integer FEAT_MEAS   = 1;   // 特性2 自动测量
localparam integer FEAT_SCROLL = 1;   // 特性3 滚动显示
localparam integer FEAT_UART   = 1;   // 特性4 串口在线配置
```

表 28: VGA 示波器的四层结构

层	文件	职责
HAL	hal/vga_timing.v	参数化行/场时序、同步极性、pix_ce 像素节拍
HAL	hal/vram_sdp.v	1 写 1 读 RAM 封装（推断 BSRAM）
HAL	hal/font_rom.v	8×8 点阵字模（分布式 ROM）
ABS	abs/grid_layer.v	网格与零电平轴
ABS	abs/trace_layer.v	单通道波形（相邻采样连竖线）
ABS	abs/text_layer.v	2 行 × 32 列文本 + 5 个十六进制数字域
ABS	abs/compositor.v	图层优先级合成（文字 > CH1 > CH0 > 网格）
APP	app/scope_capture.v	采样分频、写指针、触发对齐
APP	app/scope_meas.v	峰峰值与过零计数（自动测量）
APP	app/scope_cmd.v	ASCII 命令解析（串口在线配置）
APP	app/scope_app.v	时基/触发/滚动寄存器与按键仲裁
APP	app/scope_ui.v	按键去抖、短按/长按识别
TOP	vga_scope_top.v	参数集中处、特性开关、像素流水线对齐
TOP	adda_scope_top.v	原 adda + 示波器 + 串口配置（新 Top）

表 29: 特性开关与关闭后的行为

开关	特性	关闭后的行为
FEAT_TRIG	触发对齐：检测电平穿越，命中时写指针归零	自由采集，周期信号左右轻微漂移
FEAT_MEAS	自动测量：帧内统计 min/max 与过零次数	测量模块整块不例化，第二行显示 0
FEAT_SCROLL	滚动显示：帧起始锁存写指针，读地址 base−640+px	读地址退化为 px，只显示前 640 点
FEAT_UART	串口在线配置：独立第二路 uart_rx	cfg_* 被忽略，仅保留按键

15.4 IO（引脚）配置方式

15.4.1 引脚表格式

引脚全部外置在 FPGA/src/vga/pins/scope_pins.csv 中，每行描述一个物理引脚：

```
# 端口名, 引脚, 方向, 驱动mA(输入填 0), 上下拉, Bank电压, 说明
sys_clk,T9,in,0,NONE,3.3,50MHz 晶振
ad_date[3],G16,in,0,NONE,3.3,ADC D3
vga_hs,T11,out,8,NONE,3.3,VGA HSYNC
vga_rgb[11],U13,out,8,NONE,3.3,R1 -> 270R
```

端口名支持任意位（如 ad_date[3]、vga_rgb[11]），方向为 in/out；输入行不需要驱动电流。

15.4.2 生成约束

改完 CSV 后执行：

```
python pins/gen_cst.py        # 生成 project_2025_11_07_scope.cst
```

生成器会把每行翻译成一对 Gowin 约束语句：

```
IO_LOC  "vga_hs" T11;
IO_PORT "vga_hs" PULL_MODE=NONE DRIVE=8 BANK_VCCIO=3.3;
```

生成结果可整体替换工程约束，或只把需要的行合并进 FPGA/src/project_2025_11_07.cst。本工程已把 8 条 VGA 约束合并进该文件，工程约束共 38 个端口（30 个原有 + 8 个 VGA），端口与引脚均唯一、无冲突。

15.4.3 本工程实际分配的 VGA 引脚

这 8 个引脚不是随意指定的，而是从 PnR 报告（impl/pnr/ 下的 pin.html）的 **All Package Pins** 表中，筛选 Signal="-"、LVCMOS33、bank 4 的空闲 IO 得到（脚本见 tools/ 下的 list_free_pins.py），并确认与现有 30 个端口及自动布局的数码管引脚无重叠：

表 30: VGA 引脚分配（全部 bank 4，VCCIO = 3.3V）

端口	引脚	Site	接法
vga_hs	T11	IOB45[B]	DE-15 脚 13
vga_vs	T12	IOB43[A]	DE-15 脚 14
vga_rgb[11]（R1）	U13	IOB50[A]	串联 270Ω
vga_rgb[10]（R0）	R10	IOB35[A]	串联 560Ω
vga_rgb[7]（G1）	U11	IOB39[A]	串联 270Ω
vga_rgb[6]（G0）	N12	IOB52[A]	串联 560Ω
vga_rgb[3]（B1）	V14	IOB54[B]	串联 270Ω
vga_rgb[2]（B0）	M11	IOB48[A]	串联 560Ω

RGB 采用“2 bit/色 + 电阻网络”的简易 DAC：每通道高 bit 串 270Ω、低 bit 串 560Ω，输出端分别接 75Ω 负载（显示器端），可得到 64 色。HS/VS 直接由 3.3V LVCMOS 驱动，绝大多数显示器可识别。

15.5 视频格式配置

分辨率与时序全部是 vga_scope_top.v 的 parameter，默认 640×480：

表 31: 三种可选视频格式

模式	H 可见/前肩/同步/后肩	V 可见/前肩/同步/后肩	极性	像素时钟
640×480@59.5	640/16/96/48	480/10/2/33	−/−	25MHz（50MHz/2）
800×600@60	800/40/128/88	600/1/4/23	+/+	40MHz
1024×768@60	1024/24/136/160	768/3/6/29	−/−	65MHz

像素节拍由 PIX_DIV 参数控制：640×480 取 2，即 50MHz 二分频得到 25MHz，**不需要 PLL**；800×600 与 1024×768 需要 40/65MHz，可复用工程中已有的 Gowin_PLL IP（50MHz 输入，1024×768 取 VCO = 50×13 = 650MHz、ODIV = 10）。换分辨率时需同步调整绘图区参数：

参数	默认值	含义
PLOT_H	448	绘图区高度（16 的倍数，便于取模）
AXIS_Y	224	零电平轴位置（约 PLOT_H/2）
TEXT_Y0	448	文本区起始行
GRID_STEP	64	网格间距（建议取 2 的幂）

15.6 运行时配置

15.6.1 按键

操作	效果
短按 key_in（<800ms）	时基 8 档循环：1k/5k/25k/100k/500k/5M/50M S/s
长按（≥800ms）	运行 / 冻结

15.6.2 串口命令（特性 4）

串口与原有二进制 A/B 参数帧**共用同一个 rx 引脚**，但由独立的第二路 uart_rx 接收，因此互不影响。命令为 ASCII 文本，格式 < 字母 >< 十六进制数 ><CR/LF>：

非十六进制字节会被当作命令字母；二进制 A/B 帧中不含回车符，因此不会误提交。若串口写入与按键同一拍生效，串口优先。

15.6.3 显示内容

文本层固定两行，其中 5 个 4 位十六进制数字域由运行值填充：

表 32: 串口命令表

命令	例子	含义
T<hex4>	T0063	设置采样分频 tb_div（约 500 kS/s）
R<0/1>	R0	冻结 / 运行
A<0/1>	A1	触发使能
L<hex3>	L200	触发电平（0∼3FF）
E<0/1>	E1	触发边沿：0 = 上升沿，1 = 下降沿
S<0/1>	S1	滚动显示使能

行	内容
第 0 行	AD=######## U=######## TB=########（AD 输入 / DAC 输出 / 时基分频）
第 1 行	VP=######## FR=########（峰峰值 / 过零次数 × 60Hz 近似频率）

15.7 显示原理与资源占用

每个像素周期依次完成：读采样 RAM（同步读，1 拍延迟）→ 相邻两点各自映射 y 坐标 → 用竖线填满两点之间 → 在像素周期后半拍求值、pix_ce 时打入输出寄存器；hs/vs/de 固定延迟 2 拍（正好 1 像素）以保持对齐。坐标映射为

$$y = (H_{\text{plot}} - 1) - \left\lfloor \frac{s \times 7}{16} \right\rfloor, \tag{11}$$

即 10bit 采样（0∼1023）映射到 0∼447 的绘图区；网格 64×64，零轴位于 $y = 224$。

表 33: 资源估计（GW5A-25A：23040 LUT / 28 DSP / 6 PLL）

模块	LUT（约）	备注
时序 / 网格 / 合成	250	计数器与比较
字模 + 文本	600	38 字模 × 8 行 + 5 个数字域
双通道波形	200	4 个 10×7 乘法
采集 ×2	200	分频 + 触发比较
测量 + 命令解析	250	关闭对应特性可省去
按键 / 配置	200	长计数器
合计	≈1700	占 7.4%；BSRAM 2 块，DSP 0

15.8 验证状态与限制

- TD 6.0.3 前端分析：23 个源文件通过（PRJ-1401），无语法错误；
- 自写端口连接检查：23 个模块、26 处例化，0 个未定义端口；

- 渲染数学已用 Python 复现并出图（图 4）；
- 逻辑综合、时序与上板验证需在 Gowin EDA 与实板完成。

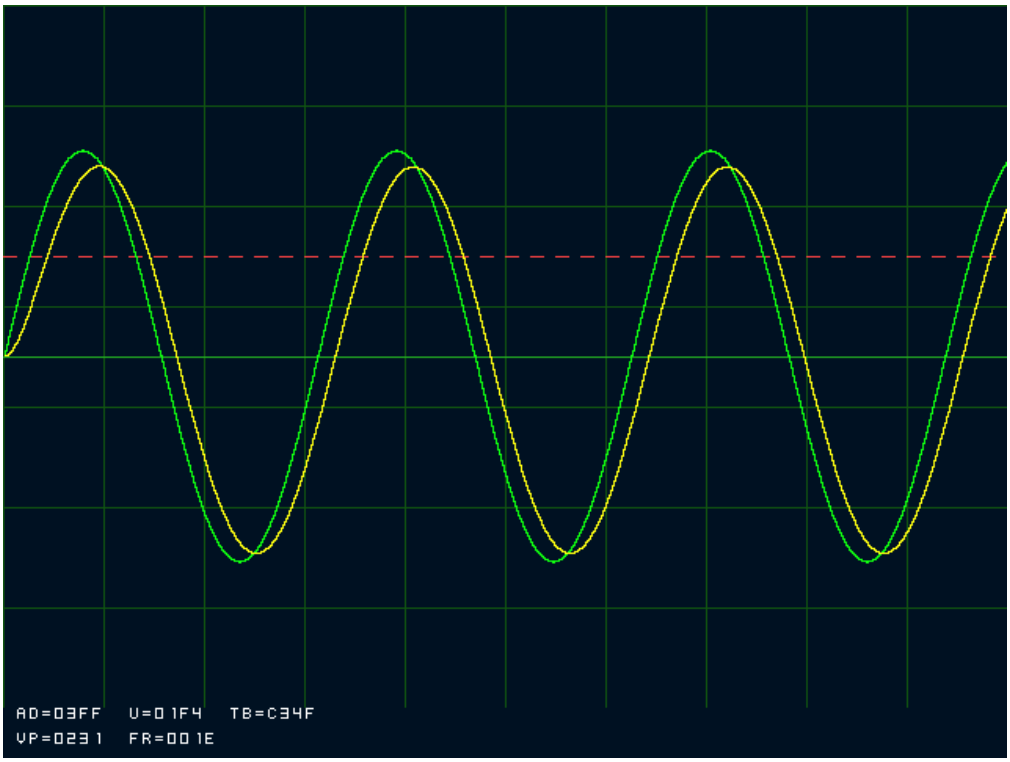


图 4: 640×480 显示效果（按 RTL 相同的映射关系复现）：双通道波形、64×64 网格、零电平轴、红色虚线为触发电平，底部两行为运行值。

已知限制：（1）触发为“指针归零”式对齐，没有预触发深度与单次捕获；（2）频率由帧内过零数 ×60 得到，分辨率 60Hz，且阈值固定为 512；（3）滚动模式每帧更新一次基准，极高采样率下帧间会有跳变；（4）串口命令为 ASCII 文本，极端情况下可能与二进制帧混淆；（5）文本为固定模板，暂不支持任意字符串。

A 附录

A.1 附录 A：文件清单与统计

全部注释只增加以 // 开头的整行注释，未改动任何代码行。经脚本比对，注释前后去掉注释后的 Verilog 词法内容**完全一致**（17 个文件全部通过）。

A.2 附录 B：模块与关键信号速查

A.3 附录 C：复现步骤

A.3.1 C.1 用安路 TD 打开（代码浏览用容器工程）

高云.gprj 无法被 TD 直接打开，本报告用 TD 的 Tcl 接口创建了一个同名容器工程：

文件	模块	原始行数	注释后行数
src/adda.v	adda	261	417
src/rtl.v	rtl	80	143
src/eso.v	eso	140	252
src/divider.v	divider	50	82
src/key.v	key	32	61
src/seg.v	seg	153	231
src/bcd.v	bcd	62	98
src/uart_rx.v	uart_rx	116	187
src/uart_tx.v	uart_tx	84	136
src/gowin_pll/gowin_pll.v	Gowin_PLL	137	171
src/fifo_top/fifo_top.v	fifo_top	504	536
src/project_2025_11_07.cst	约束	69	109
src/gowin_pll/gowin_pll_tmp.v	模板	19	30
src/fifo_top/fifo_top_tmp.v	模板	24	34

模块	输入	输出
adda	ad_date, key_in, rx, sys_clk, sys_rst_n	ad_out, sel, seg, tx, led*, *_clk1
rtl	ad_data	sign, volt
eso	e_in, u_out, c/k	out_da, z1k0..z3k0
divider	sys_clk	clk0, clk1
key	key_in	key_out
seg	data, point, sign	sel, seg
bcd	data[9:0]	unit, ten, hun
uart_rx	rx	po_data, po_flag
uart_tx	pi_data, pi_flag	tx

```
$env:PATH  # 需能找到 D:\FPGA\bin\td_commands_prompt.exe
cd C:/Users/Administrator/Desktop/task/FPGA_TD
# 通过 stdin 喂给 TD 的 Tcl 解释器:
create_project -family ph1 -device PH1A60GEG324 project_2025_11_07_TD .
import_device ph1_60.db -package PH1A60GEG324
add_files -list ./files.list
exit
```

随后直接双击 project_2025_11_07_TD.al 或 td.exe project_2025_11_07_TD.al 即可打开，工程只引用../FPGA/src 下的源文件，不复制。注意该工程只能浏览/编辑代码，不能用安路器件综合本设计。

A.3.2 C.2 编译本报告

```
cd C:/Users/Administrator/Desktop/task/doc
xelatex main.tex
xelatex main.tex      # 第二遍生成目录与交叉引用
```

需要 MiKTeX（含 ctex 宏包与中文字体）。文档引用../FPGA/src/*.v 源文件，移动目录结构后需同步修改相对路径。

A.4 附录 D：术语表

术语	含义
ADRC	自抗扰控制（Active Disturbance Rejection Control）
ESO	扩张状态观测器（Extended State Observer），估计状态与总扰动
TD	安路科技开发工具 TangDynasty
GW5A	高云 Arora V 系列 FPGA（22nm 工艺）
8N1	UART 8 数据位、无校验、1 停止位
FWFT	First-Word Fall-Through，FIFO 首字直通模式
BCD	二进制编码十进制
LSB/MSB	最低有效位 / 最高有效位